

Directory structure:

```
xai-org-x-algorithm/
├── README.md
├── CODE_OF_CONDUCT.md
├── LICENSE
├── candidate-pipeline/
│   ├── candidate_pipeline.rs
│   ├── filter.rs
│   ├── hydrator.rs
│   ├── lib.rs
│   ├── query_hydrator.rs
│   ├── scorer.rs
│   ├── selector.rs
│   ├── side_effect.rs
│   └── source.rs
├── home-mixer/
│   ├── lib.rs
│   ├── main.rs
│   ├── server.rs
│   └── candidate_hydrators/
│       ├── core_data_candidate_hydrator.rs
│       ├── gizmoduck_hydrator.rs
│       ├── in_network_candidate_hydrator.rs
│       ├── mod.rs
│       ├── subscription_hydrator.rs
│       ├── vf_candidate_hydrator.rs
│       └── video_duration_candidate_hydrator.rs
│   └── candidate_pipeline/
│       ├── candidate.rs
│       ├── candidate_features.rs
│       ├── mod.rs
│       ├── phoenix_candidate_pipeline.rs
│       ├── query.rs
│       └── query_features.rs
│   └── filters/
│       ├── age_filter.rs
│       ├── author_socialgraph_filter.rs
│       ├── core_data_hydration_filter.rs
│       ├── dedup_conversation_filter.rs
│       ├── drop_duplicates_filter.rs
│       ├── ineligible_subscription_filter.rs
│       ├── mod.rs
│       ├── muted_keyword_filter.rs
│       ├── previously_seen_posts_filter.rs
│       ├── previously_served_posts_filter.rs
│       ├── retweet_deduplication_filter.rs
│       ├── self_tweet_filter.rs
│       └── vf_filter.rs
│   └── query_hydrators/
│       ├── mod.rs
│       ├── user_action_seq_query_hydrator.rs
│       └── user_features_query_hydrator.rs
│   └── scorers/
│       ├── author_diversity_scorer.rs
│       ├── mod.rs
│       ├── oon_scorer.rs
│       ├── phoenix_scorer.rs
│       └── weighted_scorer.rs
│   └── selectors/
│       ├── mod.rs
│       └── top_k_score_selector.rs
│   └── side_effects/
│       ├── cache_request_info_side_effect.rs
│       └── mod.rs
```

```

├── sources/
│   ├── mod.rs
│   ├── phoenix_source.rs
│   └── thunder_source.rs
├── phoenix/
│   ├── README.md
│   ├── grok.py
│   ├── pyproject.toml
│   ├── recsys_model.py
│   ├── recsys_retrieval_model.py
│   ├── run_ranker.py
│   ├── run_retrieval.py
│   ├── runners.py
│   ├── test_recsys_model.py
│   └── test_recsys_retrieval_model.py
├── thunder/
│   ├── deserializer.rs
│   ├── kafka_utils.rs
│   ├── lib.rs
│   ├── main.rs
│   ├── thunder_service.rs
│   └── kafka/
│       ├── mod.rs
│       ├── tweet_events_listener.rs
│       ├── tweet_events_listener_v2.rs
│       └── utils.rs
└── posts/
    ├── mod.rs
    └── post_store.rs

```

Files Content:

=====

FILE: README.md

=====

X For You Feed Algorithm

This repository contains the core recommendation system powering the "For You" feed on X. It combines in-network content (from accounts you follow) with out-of-network content (discovered through ML-based retrieval) and ranks everything using a Grok-based transformer model.

> **Note:** The transformer implementation is ported from the [Grok-1 open source release] (<https://github.com/xai-org/grok-1>) by xAI, adapted for recommendation system use cases.

Table of Contents

- [Overview] (#overview)
- [System Architecture] (#system-architecture)
- [Components] (#components)
 - [Home Mixer] (#home-mixer)
 - [Thunder] (#thunder)
 - [Phoenix] (#phoenix)
 - [Candidate Pipeline] (#candidate-pipeline)
- [How It Works] (#how-it-works)
 - [Pipeline Stages] (#pipeline-stages)
 - [Scoring and Ranking] (#scoring-and-ranking)
 - [Filtering] (#filtering)
- [Key Design Decisions] (#key-design-decisions)
- [License] (#license)

Overview

The For You feed algorithm retrieves, ranks, and filters posts from two sources:

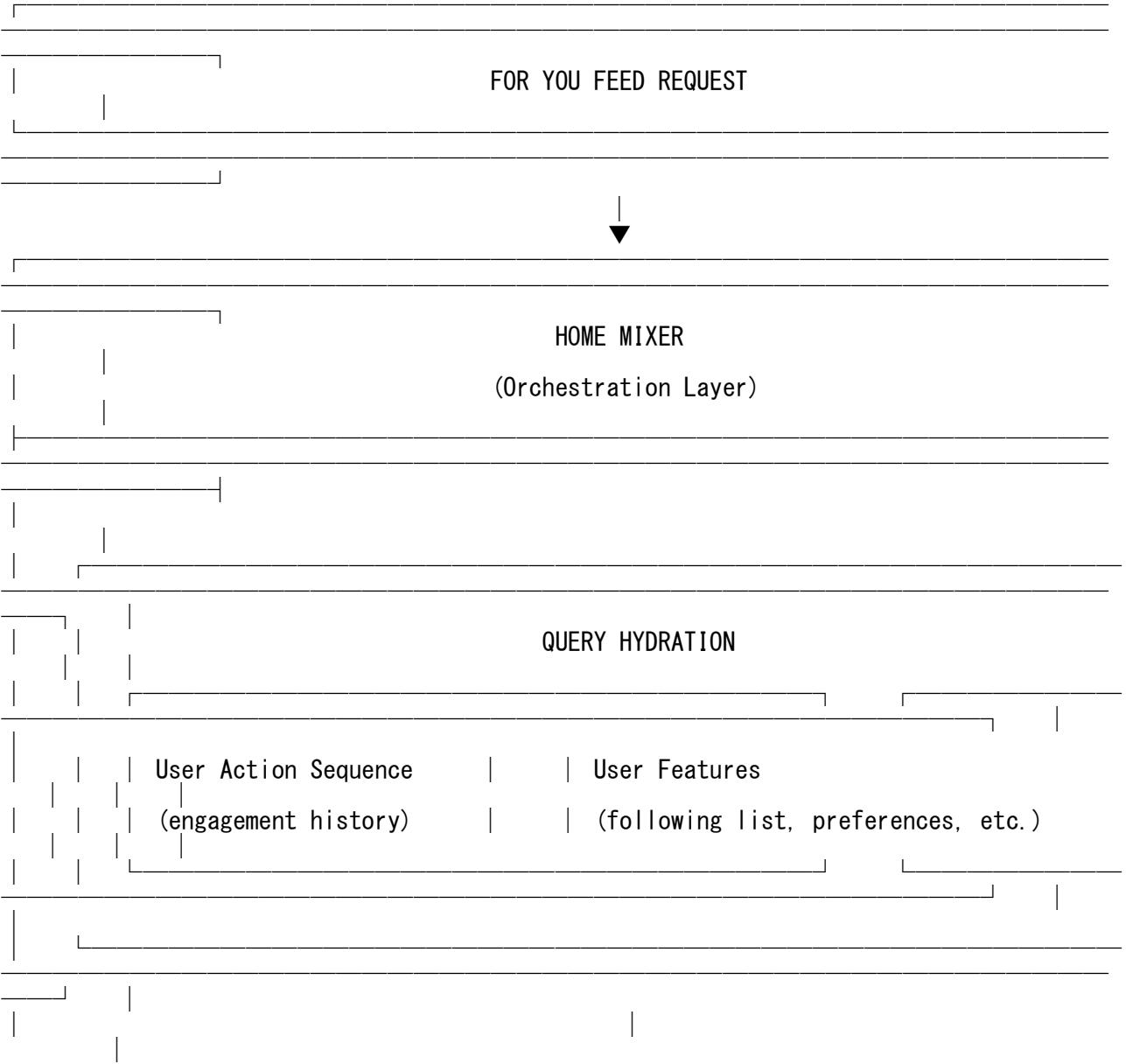
- 1. **In-Network (Thunder)**: Posts from accounts you follow
- 2. **Out-of-Network (Phoenix Retrieval)**: Posts discovered from a global corpus

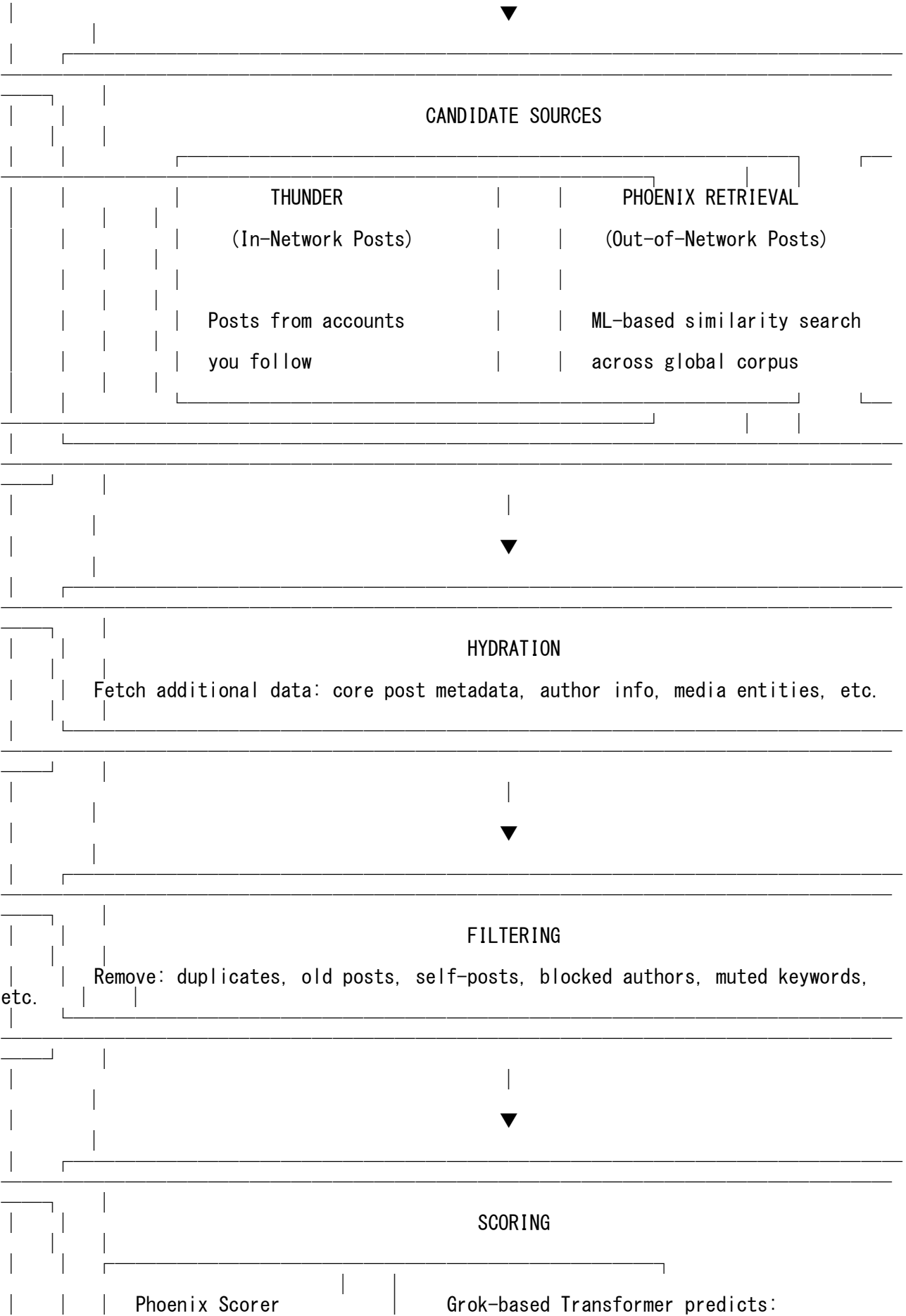
Both sources are combined and ranked together using **Phoenix**, a Grok-based transformer model that predicts engagement probabilities for each post. The final score is a weighted combination of these predicted engagements.

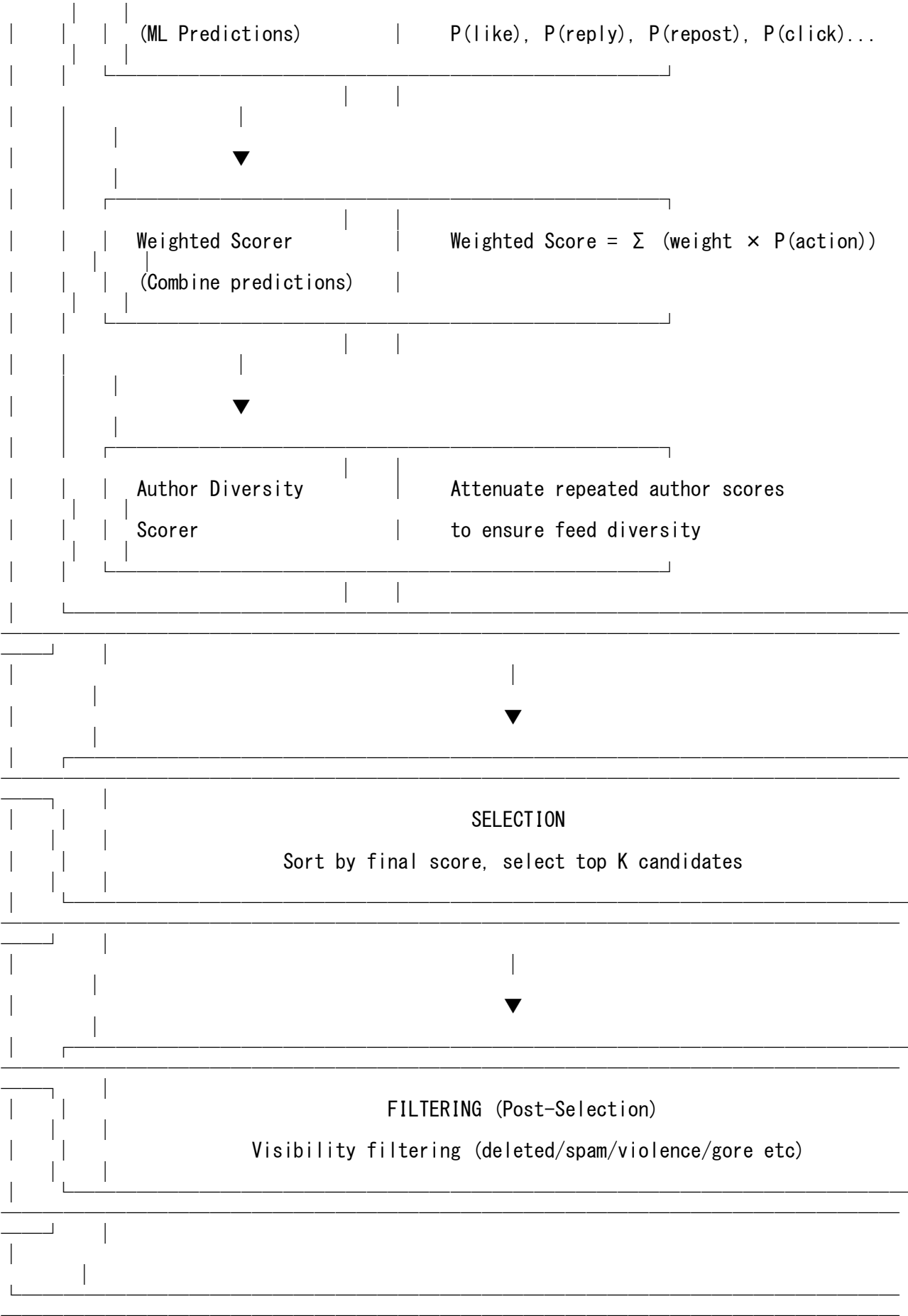
We have eliminated every single hand-engineered feature and most heuristics from the system. The Grok-based transformer does all the heavy lifting by understanding your engagement history (what you liked, replied to, shared, etc.) and using that to determine what content is relevant to you.

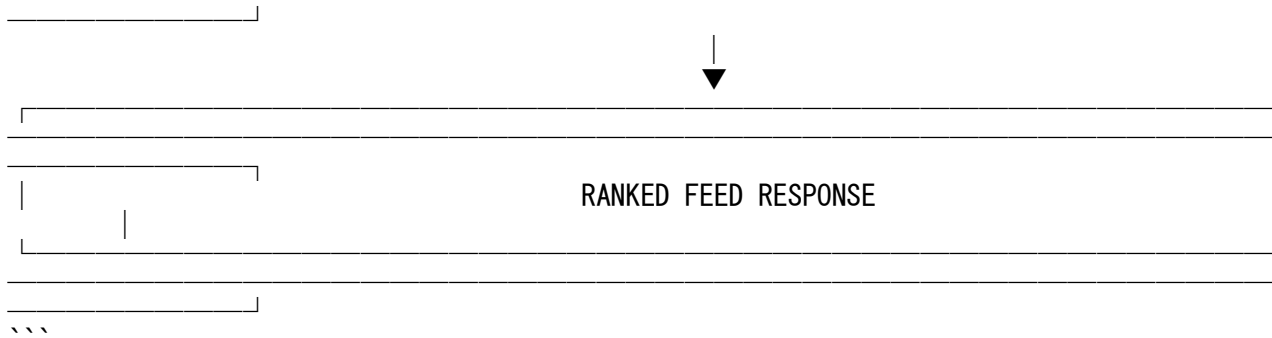
System Architecture

...









Components

Home Mixer

****Location:**** [``home-mixer/``](home-mixer/)

The orchestration layer that assembles the For You feed. It leverages the ``CandidatePipeline`` framework with the following stages:

Stage	Description
Query Hydrators	Fetch user context (engagement history, following list)
Sources	Retrieve candidates from Thunder and Phoenix
Hydrators	Enrich candidates with additional data
Filters	Remove ineligible candidates
Scorers	Predict engagement and compute final scores
Selector	Sort by score and select top K
Post-Selection Filters	Final visibility and dedup checks
Side Effects	Cache request info for future use

The server exposes a gRPC endpoint (``ScoredPostsService``) that returns ranked posts for a given user.

Thunder

****Location:**** [``thunder/``](thunder/)

An in-memory post store and realtime ingestion pipeline that tracks recent posts from all users. It:

- Consumes post create/delete events from Kafka
- Maintains per-user stores for original posts, replies/reposts, and video posts
- Serves "in-network" post candidates from accounts the requesting user follows
- Automatically trims posts older than the retention period

Thunder enables sub-millisecond lookups for in-network content without hitting an external database.

Phoenix

****Location:**** [``phoenix/``](phoenix/)

The ML component with two main functions:

1. Retrieval (Two-Tower Model)

Finds relevant out-of-network posts:

- ****User Tower****: Encodes user features and engagement history into an embedding
- ****Candidate Tower****: Encodes all posts into embeddings
- ****Similarity Search****: Retrieves top-K posts via dot product similarity

2. Ranking (Transformer with Candidate Isolation)

Predicts engagement probabilities for each candidate:

- Takes user context (engagement history) and candidate posts as input
- Uses special attention masking so candidates cannot attend to each other
- Outputs probabilities for each action type (like, reply, repost, click, etc.)

See [``phoenix/README.md``] (`phoenix/README.md`) for detailed architecture documentation.

Candidate Pipeline

****Location**** [``candidate-pipeline/``] (`candidate-pipeline/`)

A reusable framework for building recommendation pipelines. Defines traits for:

Trait	Purpose
<code>`Source`</code>	Fetch candidates from a data source
<code>`Hydrator`</code>	Enrich candidates with additional features
<code>`Filter`</code>	Remove candidates that shouldn't be shown
<code>`Scorer`</code>	Compute scores for ranking
<code>`Selector`</code>	Sort and select top candidates
<code>`SideEffect`</code>	Run async side effects (caching, logging)

The framework runs sources and hydrators in parallel where possible, with configurable error handling and logging.

How It Works

Pipeline Stages

1. ****Query Hydration****: Fetch the user's recent engagements history and metadata (eg. following list)
2. ****Candidate Sourcing****: Retrieve candidates from:
 - ****Thunder****: Recent posts from followed accounts (in-network)
 - ****Phoenix Retrieval****: ML-discovered posts from the global corpus (out-of-network)
3. ****Candidate Hydration****: Enrich candidates with:
 - Core post data (text, media, etc.)
 - Author information (username, verification status)
 - Video duration (for video posts)
 - Subscription status
4. ****Pre-Scoring Filters****: Remove posts that are:
 - Duplicates
 - Too old
 - From the viewer themselves
 - From blocked/muted accounts
 - Containing muted keywords
 - Previously seen or recently served
 - Ineligible subscription content
5. ****Scoring****: Apply multiple scorers sequentially:
 - ****Phoenix Scorer****: Get ML predictions from the Phoenix transformer model
 - ****Weighted Scorer****: Combine predictions into a final relevance score

- **Author Diversity Scorer**: Attenuate repeated author scores for diversity
- **00N Scorer**: Adjust scores for out-of-network content

6. **Selection**: Sort by score and select the top K candidates

7. **Post-Selection Processing**: Final validation of post candidates to be served

Scoring and Ranking

The Phoenix Grok-based transformer model predicts probabilities for multiple engagement types:

...

Predictions:

_____	P(favorite)
_____	P(reply)
_____	P(repost)
_____	P(quote)
_____	P(click)
_____	P(profile_click)
_____	P(video_view)
_____	P(photo_expand)
_____	P(share)
_____	P(dwell)
_____	P(follow_author)
_____	P(not_interested)
_____	P(block_author)
_____	P(mute_author)
_____	P(report)

...

The **Weighted Scorer** combines these into a final score:

...

Final Score = $\sum (\text{weight}_i \times P(\text{action}_i))$

Positive actions (like, repost, share) have positive weights. Negative actions (block, mute, report) have negative weights, pushing down content the user would likely dislike.

Filtering

Filters run at two stages:

Pre-Scoring Filters:

Filter	Purpose
\DropDuplicatesFilter\	Remove duplicate post IDs
\CoreDataHydrationFilter\	Remove posts that failed to hydrate core metadata
\AgeFilter\	Remove posts older than threshold
\SelfpostFilter\	Remove user's own posts
\RepostDeduplicationFilter\	Dedupe reposts of same content
\IneligibleSubscriptionFilter\	Remove paywalled content user can't access
\PreviouslySeenPostsFilter\	Remove posts user has already seen
\PreviouslyServedPostsFilter\	Remove posts already served in session
\MutedKeywordFilter\	Remove posts with user's muted keywords
\AuthorSocialgraphFilter\	Remove posts from blocked/muted authors

Post-Selection Filters:

Filter	Purpose
`VFFilter`	Remove posts that are deleted/spam/violence/gore etc.
`DedupConversationFilter`	Deduplicate multiple branches of the same conversation thread

Key Design Decisions

1. No Hand-Engineered Features

The system relies entirely on the Grok-based transformer to learn relevance from user engagement sequences. No manual feature engineering for content relevance. This significantly reduces the complexity in our data pipelines and serving infrastructure.

2. Candidate Isolation in Ranking

During transformer inference, candidates cannot attend to each other—only to the user context. This ensures the score for a post doesn't depend on which other posts are in the batch, making scores consistent and cacheable.

3. Hash-Based Embeddings

Both retrieval and ranking use multiple hash functions for embedding lookup

4. Multi-Action Prediction

Rather than predicting a single "relevance" score, the model predicts probabilities for many actions.

5. Composable Pipeline Architecture

The `candidate-pipeline` crate provides a flexible framework for building recommendation pipelines with:

- Separation of pipeline execution and monitoring from business logic
- Parallel execution of independent stages and graceful error handling
- Easy addition of new sources, hydrations, filters, and scorers

License

This project is licensed under the Apache License 2.0. See LICENSE for details.

=====

FILE: CODE_OF_CONDUCT.md

=====

Be excellent to each other.

=====

FILE: LICENSE

=====

Apache License
Version 2.0, January 2004
<http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.
3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable

(except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:
 - (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
 - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
 - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
 - (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.
6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor,

except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. **Disclaimer of Warranty.** Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.
8. **Limitation of Liability.** In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.
9. **Accepting Warranty or Additional Liability.** While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

```
=====
FILE: candidate-pipeline/candidate_pipeline.rs
=====
```

```
use crate::filter::Filter;
use crate::hydrator::Hydrator;
use crate::query_hydrator::QueryHydrator;
use crate::scorer::Scorer;
use crate::selector::Selector;
use crate::side_effect::{SideEffect, SideEffectInput};
use crate::source::Source;
use futures::future::join_all;
use log::{error, info, warn};
use std::sync::Arc;
use tonic::async_trait;

#[derive(Copy, Clone, Debug)]
pub enum PipelineStage {
    QueryHydrator,
    Source,
    Hydrator,
    PostSelectionHydrator,
    Filter,
    PostSelectionFilter,
    Scorer,
}

pub struct PipelineResult<Q, C> {
    pub retrieved_candidates: Vec<C>,
    pub filtered_candidates: Vec<C>,
    pub selected_candidates: Vec<C>,
    pub query: Arc<Q>,
}

/// Provides a stable request identifier for logging/tracing.
pub trait HasRequestId {
    fn request_id(&self) -> &str;
}

#[async_trait]
pub trait CandidatePipeline<Q, C>: Send + Sync
where
    Q: HasRequestId + Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    fn query_hydrators(&self) -> &[Box<dyn QueryHydrator<Q>>];
    fn sources(&self) -> &[Box<dyn Source<Q, C>>];
    fn hydrators(&self) -> &[Box<dyn Hydrator<Q, C>>];
    fn filters(&self) -> &[Box<dyn Filter<Q, C>>];
    fn scorers(&self) -> &[Box<dyn Scorer<Q, C>>];
    fn selector(&self) -> &dyn Selector<Q, C>;
    fn post_selection_hydrators(&self) -> &[Box<dyn Hydrator<Q, C>>];
    fn post_selection_filters(&self) -> &[Box<dyn Filter<Q, C>>];
    fn side_effects(&self) -> Arc<Vec<Box<dyn SideEffect<Q, C>>>>;
    fn result_size(&self) -> usize;

    async fn execute(&self, query: Q) -> PipelineResult<Q, C> {
        let hydrated_query = self.hydrate_query(query).await;

        let candidates = self.fetch_candidates(&hydrated_query).await;

        let hydrated_candidates = self.hydrate(&hydrated_query, candidates).await;

        let (kept_candidates, mut filtered_candidates) = self
```

```

        .filter(&hydrated_query, hydrated_candidates.clone())
        .await;

let scored_candidates = self.score(&hydrated_query, kept_candidates).await;

let selected_candidates = self.select(&hydrated_query, scored_candidates);

let post_selection_hydrated_candidates = self
    .hydrate_post_selection(&hydrated_query, selected_candidates)
    .await;

let (mut final_candidates, post_selection_filtered_candidates) = self
    .filter_post_selection(&hydrated_query, post_selection_hydrated_candidates)
    .await;
filtered_candidates.extend(post_selection_filtered_candidates);

final_candidates.truncate(self.result_size());

let arc_hydrated_query = Arc::new(hydrated_query);
let input = Arc::new(SideEffectInput {
    query: arc_hydrated_query.clone(),
    selected_candidates: final_candidates.clone(),
});
self.run_side_effects(input);

PipelineResult {
    retrieved_candidates: hydrated_candidates,
    filtered_candidates: filtered_candidates,
    selected_candidates: final_candidates,
    query: arc_hydrated_query,
}
}

/// Run all query hydrators in parallel and merge results into the query.
async fn hydrate_query(&self, query: Q) -> Q {
    let request_id = query.request_id().to_string();
    let hydrators: Vec<_> = self
        .query_hydrators()
        .iter()
        .filter(|h| h.enable(&query))
        .collect();
    let hydrate_futures = hydrators.iter().map(|h| h.hydrate(&query));
    let results = join_all(hydrate_futures).await;

    let mut hydrated_query = query;
    for (hydrator, result) in hydrators.iter().zip(results) {
        match result {
            Ok(hydrated) => {
                hydrator.update(&mut hydrated_query, hydrated);
            }
            Err(err) => {
                error!(
                    "request_id={} stage={:?} component={} failed: {}",
                    request_id,
                    PipelineStage::QueryHydrator,
                    hydrator.name(),
                    err
                );
            }
        }
    }
    hydrated_query
}

```

```

    /// Run all candidate sources in parallel and collect results.
    async fn fetch_candidates(&self, query: &Q) -> Vec<C> {
        let request_id = query.request_id().to_string();
        let sources: Vec<_> = self.sources().iter().filter(|s|
s.enable(query)).collect();
        let source_futures = sources.iter().map(|s| s.get_candidates(query));
        let results = join_all(source_futures).await;

        let mut collected = Vec::new();
        for (source, result) in sources.iter().zip(results) {
            match result {
                Ok(mut candidates) => {
                    info!(
                        "request_id={} stage={:?} component={} fetched {} candidates",
                        request_id,
                        PipelineStage::Source,
                        source.name(),
                        candidates.len()
                    );
                    collected.append(&mut candidates);
                }
                Err(err) => {
                    error!(
                        "request_id={} stage={:?} component={} failed: {}",
                        request_id,
                        PipelineStage::Source,
                        source.name(),
                        err
                    );
                }
            }
        }
        collected
    }

    /// Run all candidate hydrators in parallel and merge results into candidates.
    async fn hydrate(&self, query: &Q, candidates: Vec<C>) -> Vec<C> {
        self.run_hydrators(query, candidates, self.hydrators(),
PipelineStage::Hydrator)
            .await
    }

    /// Run post-selection candidate hydrators in parallel and merge results into
    candidates.
    async fn hydrate_post_selection(&self, query: &Q, candidates: Vec<C>) -> Vec<C> {
        self.run_hydrators(
            query,
            candidates,
            self.post_selection_hydrators(),
            PipelineStage::PostSelectionHydrator,
        )
            .await
    }

    /// Shared helper to hydrate with a provided hydrator list.
    async fn run_hydrators(
        &self,
        query: &Q,
        mut candidates: Vec<C>,
        hydrators: &[Box<dyn Hydrator<Q, C>>],
        stage: PipelineStage,
    ) -> Vec<C> {
        let request_id = query.request_id().to_string();
        let hydrators: Vec<_> = hydrators.iter().filter(|h| h.enable(query)).collect();

```

```

let expected_len = candidates.len();
let hydrate_futures = hydrators.iter().map(|h| h.hydrate(query, &candidates));
let results = join_all(hydrate_futures).await;
for (hydrator, result) in hydrators.iter().zip(results) {
    match result {
        Ok(hydrated) => {
            if hydrated.len() == expected_len {
                hydrator.update_all(&mut candidates, hydrated);
            } else {
                warn!(
                    "request_id={} stage={:?} component={} skipped:
length_mismatch expected={} got={}",
                    request_id,
                    stage,
                    hydrator.name(),
                    expected_len,
                    hydrated.len()
                );
            }
        }
        Err(err) => {
            error!(
                "request_id={} stage={:?} component={} failed: {}",
                request_id,
                stage,
                hydrator.name(),
                err
            );
        }
    }
}
candidates
}

/// Run all filters sequentially. Each filter partitions candidates into kept and
removed.
async fn filter(&self, query: &Q, candidates: Vec<C>) -> (Vec<C>, Vec<C>) {
    self.run_filters(query, candidates, self.filters(), PipelineStage::Filter)
        .await
}

/// Run post-scoring filters sequentially on already-scored candidates.
Vec<C>) {
    self.run_filters(
        query,
        candidates,
        self.post_selection_filters(),
        PipelineStage::PostSelectionFilter,
    )
    .await
}

// Shared helper to run filters sequentially from a provided filter list.
async fn run_filters(
    &self,
    query: &Q,
    mut candidates: Vec<C>,
    filters: &[Box<dyn Filter<Q, C>>],
    stage: PipelineStage,
) -> (Vec<C>, Vec<C>) {
    let request_id = query.request_id().to_string();
    let mut all_removed = Vec::new();
    for filter in filters.iter().filter(|f| f.enable(query)) {

```

```

        let backup = candidates.clone();
        match filter.filter(query, candidates).await {
            Ok(result) => {
                candidates = result.kept;
                all_removed.extend(result.removed);
            }
            Err(err) => {
                error!(
                    "request_id={} stage={:?} component={} failed: {}",
                    request_id,
                    stage,
                    filter.name(),
                    err
                );
                candidates = backup;
            }
        }
    }
    info!(
        "request_id={} stage={:?} kept {}, removed {}",
        request_id,
        stage,
        candidates.len(),
        all_removed.len()
    );
    (candidates, all_removed)
}

/// Run all scorers sequentially and apply their results to candidates.
async fn score(&self, query: &Q, mut candidates: Vec<C>) -> Vec<C> {
    let request_id = query.request_id().to_string();
    let expected_len = candidates.len();
    for scorer in self.scorers().iter().filter(|s| s.enable(query)) {
        match scorer.score(query, &candidates).await {
            Ok(scored) => {
                if scored.len() == expected_len {
                    scorer.update_all(&mut candidates, scored);
                } else {
                    warn!(
                        "request_id={} stage={:?} component={} skipped:
length_mismatch expected={} got={}",
                        request_id,
                        PipelineStage::Scorer,
                        scorer.name(),
                        expected_len,
                        scored.len()
                    );
                }
            }
            Err(err) => {
                error!(
                    "request_id={} stage={:?} component={} failed: {}",
                    request_id,
                    PipelineStage::Scorer,
                    scorer.name(),
                    err
                );
            }
        }
    }
    candidates
}

/// Select (sort/truncate) candidates using the configured selector

```

```

fn select(&self, query: &Q, candidates: Vec<C>) -> Vec<C> {
    if self.selector().enable(query) {
        self.selector().select(query, candidates)
    } else {
        candidates
    }
}

// Run all side effects in parallel
fn run_side_effects(&self, input: Arc<SideEffectInput<Q, C>>) {
    let side_effects = self.side_effects();
    tokio::spawn(async move {
        let futures = side_effects
            .iter()
            .filter(|se| se.enable(input.query.clone()))
            .map(|se| se.run(input.clone()));
        let _ = join_all(futures).await;
    });
}
}

```

```

=====
FILE: candidate-pipeline/filter.rs
=====

```

```

use std::any::Any, type_name_of_val};
use tonic::async_trait;

```

```

use crate::util;

```

```

pub struct FilterResult<C> {
    pub kept: Vec<C>,
    pub removed: Vec<C>,
}

```

```

/// Filters run sequentially and partition candidates into kept and removed sets
#[async_trait]

```

```

pub trait Filter<Q, C>: Any + Send + Sync

```

```

where

```

```

    Q: Clone + Send + Sync + 'static,

```

```

    C: Clone + Send + Sync + 'static,

```

```

{

```

```

    /// Decide if this filter should run for the given query

```

```

    fn enable(&self, _query: &Q) -> bool {

```

```

        true
    }

```

```

    /// Filter candidates by evaluating each against some criteria.

```

```

    /// Returns a FilterResult containing kept candidates (which continue to the next
stage)

```

```

    /// and removed candidates (which are excluded from further processing).

```

```

    async fn filter(&self, query: &Q, candidates: Vec<C>) -> Result<FilterResult<C>,
String>;

```

```

    /// Returns a stable name for logging/metrics.

```

```

    fn name(&self) -> &'static str {

```

```

        util::short_type_name(type_name_of_val(self))
    }

```

```

}

```

```

=====

```

FILE: candidate-pipeline/hydrator.rs

```
=====
use crate::util;
use std::any::{Any, type_name_of_val};
use tonic::async_trait;

// Hydrators run in parallel and update candidate fields
#[async_trait]
pub trait Hydrator<Q, C>: Any + Send + Sync
where
    Q: Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    /// Decide if this hydrator should run for the given query
    fn enable(&self, _query: &Q) -> bool {
        true
    }

    /// Hydrate candidates by performing async operations.
    /// Returns candidates with this hydrator's fields populated.
    ///
    /// IMPORTANT: The returned vector must have the same candidates in the same order
as the input.
    /// Dropping candidates in a hydrator is not allowed - use a filter stage instead.
    async fn hydrate(&self, query: &Q, candidates: &[C]) -> Result<Vec<C>, String>;

    /// Update a single candidate with the hydrated fields.
    /// Only the fields this hydrator is responsible for should be copied.
    fn update(&self, candidate: &mut C, hydrated: C);

    /// Update all candidates with the hydrated fields from `hydrated`.
    /// Default implementation iterates and calls `update` for each pair.
    fn update_all(&self, candidates: &mut [C], hydrated: Vec<C>) {
        for (c, h) in candidates.iter_mut().zip(hydrated) {
            self.update(c, h);
        }
    }

    fn name(&self) -> &'static str {
        util::short_type_name(type_name_of_val(self))
    }
}
=====
```

FILE: candidate-pipeline/lib.rs

```
=====
pub mod candidate_pipeline;
pub mod filter;
pub mod hydrator;
pub mod query_hydrator;
pub mod scorer;
pub mod selector;
pub mod side_effect;
pub mod source;
pub mod util;
=====
```

FILE: candidate-pipeline/query_hydrator.rs

```
=====
use std::any::{Any, type_name_of_val};
=====
```

```

use tonic::async_trait;

use crate::util;

#[async_trait]
pub trait QueryHydrator<Q>: Any + Send + Sync
where
    Q: Clone + Send + Sync + 'static,
{
    /// Decide if this query hydrator should run for the given query
    fn enable(&self, _query: &Q) -> bool {
        true
    }

    /// Hydrate the query by performing async operations.
    /// Returns a new query with this hydrator's fields populated.
    async fn hydrate(&self, query: &Q) -> Result<Q, String>;

    /// Update the query with the hydrated fields.
    /// Only the fields this hydrator is responsible for should be copied.
    fn update(&self, query: &mut Q, hydrated: Q);

    fn name(&self) -> &'static str {
        util::short_type_name(type_name_of_val(self))
    }
}

```

```

=====
FILE: candidate-pipeline/scorer.rs
=====

```

```

use crate::util;
use std::any::type_name_of_val;
use tonic::async_trait;

/// Scorers update candidate fields (like a score field) and run sequentially
#[async_trait]
pub trait Scorer<Q, C>: Send + Sync
where
    Q: Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    /// Decide if this scorer should run for the given query
    fn enable(&self, _query: &Q) -> bool {
        true
    }

    /// Score candidates by performing async operations.
    /// Returns candidates with this scorer's fields populated.
    ///
    /// IMPORTANT: The returned vector must have the same candidates in the same order
    as the input.
    /// Dropping candidates in a scorer is not allowed - use a filter stage instead.
    async fn score(&self, query: &Q, candidates: &[C]) -> Result<Vec<C>, String>;

    /// Update a single candidate with the scored fields.
    /// Only the fields this scorer is responsible for should be copied.
    fn update(&self, candidate: &mut C, scored: C);

    /// Update all candidates with the scored fields from `scored`.
    /// Default implementation iterates and calls `update` for each pair.
    fn update_all(&self, candidates: &mut [C], scored: Vec<C>) {
        for (c, s) in candidates.iter_mut().zip(scored) {

```

```

        self.update(c, s);
    }
}

fn name(&self) -> &'static str {
    util::short_type_name(type_name_of_val(self))
}
}

```

```

=====
FILE: candidate-pipeline/selector.rs
=====

```

```

use crate::util;
use std::any::type_name_of_val;

pub trait Selector<Q, C>: Send + Sync
where
    Q: Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    /// Default selection: sort and truncate based on provided configs
    fn select(&self, _query: &Q, candidates: Vec<C>) -> Vec<C> {
        let mut sorted = self.sort(candidates);
        if let Some(limit) = self.size() {
            sorted.truncate(limit);
        }
        sorted
    }

    /// Decide if this selector should run for the given query
    fn enable(&self, _query: &Q) -> bool {
        true
    }

    /// Extract the score from a candidate to use for sorting.
    fn score(&self, candidate: &C) -> f64;

    /// Sort candidates by their scores in descending order.
    fn sort(&self, candidates: Vec<C>) -> Vec<C> {
        let mut sorted = candidates;
        sorted.sort_by(|a, b| {
            self.score(b)
                .partial_cmp(&self.score(a))
                .unwrap_or(std::cmp::Ordering::Equal)
        });
        sorted
    }

    /// Optionally provide a size to select. Defaults to no truncation if not
    overridden.
    fn size(&self) -> Option<usize> {
        None
    }

    fn name(&self) -> &'static str {
        util::short_type_name(type_name_of_val(self))
    }
}

```

FILE: candidate-pipeline/side_effect.rs

```
=====
use crate::util;
use std::any::type_name_of_val;
use std::sync::Arc;
use tonic::async_trait;

// A side-effect is an action run that doesn't affect the pipeline result from being
// returned
#[derive(Clone)]
pub struct SideEffectInput<Q, C> {
    pub query: Arc<Q>,
    pub selected_candidates: Vec<C>,
}

#[async_trait]
pub trait SideEffect<Q, C>: Send + Sync
where
    Q: Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    /// Decide if this side-effect should be run
    fn enable(&self, _query: Arc<Q>) -> bool {
        true
    }

    async fn run(&self, input: Arc<SideEffectInput<Q, C>>) -> Result<(), String>;

    fn name(&self) -> &'static str {
        util::short_type_name(type_name_of_val(self))
    }
}
=====
```

FILE: candidate-pipeline/source.rs

```
=====
use std::any::{Any, type_name_of_val};
use tonic::async_trait;

use crate::util;

#[async_trait]
pub trait Source<Q, C>: Any + Send + Sync
where
    Q: Clone + Send + Sync + 'static,
    C: Clone + Send + Sync + 'static,
{
    /// Decide if this source should run for the given query
    fn enable(&self, _query: &Q) -> bool {
        true
    }

    async fn get_candidates(&self, query: &Q) -> Result<Vec<C>, String>;

    fn name(&self) -> &'static str {
        util::short_type_name(type_name_of_val(self))
    }
}
=====
```

FILE: home-mixer/lib.rs

```
mod candidate_hydrators;
mod candidate_pipeline;
pub mod clients; // Excluded from open source release for security reasons
mod filters;
pub mod params; // Excluded from open source release for security reasons
mod query_hydrators;
pub mod scorers;
mod selectors;
mod server;
mod side_effects;
mod sources;
pub mod util; // Excluded from open source release for security reasons

pub use server::HomeMixerServer;
```

FILE: home-mixer/main.rs

```
use clap::Parser;
use log::info;
use std::time::Duration;

use tonic::codec::CompressionEncoding;
use tonic::service::RoutesBuilder;
use tonic_reflection::server::Builder;

use xai_home_mixer_proto as pb;
use xai_http_server::{CancellationToken, GrpcConfig, HttpServer};

use xai_home_mixer::HomeMixerServer;
use xai_home_mixer::params;

#[derive(Parser, Debug)]
#[command(about = "HomeMixer gRPC Server")]
struct Args {
    #[arg(long)]
    grpc_port: u16,
    #[arg(long)]
    metrics_port: u16,
    #[arg(long)]
    reload_interval_minutes: u64,
    #[arg(long)]
    chunk_size: usize,
}

#[xai_stats_macro::main(name = "home-mixer")]
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let args = Args::parse();
    xai_init_utils::init().log();
    xai_init_utils::init().rustls();
    info!(
        "Starting server with gRPC port: {}, metrics port: {}, reload interval: {} minutes, chunk size: {}",
        args.grpc_port, args.metrics_port, args.reload_interval_minutes,
        args.chunk_size,
    );

    // Create the service implementation
    let service = HomeMixerServer::new().await;
```

```

// Keep a reference to stats_receiver before service is moved
let reflection_service = Builder::configure()
    .register_encoded_file_descriptor_set(pb::FILE_DESCRIPTOR_SET)
    .build_v1()?;

let mut grpc_routes = RoutesBuilder::default();

grpc_routes.add_service(
    pb::scored_posts_service_server::ScoredPostsServiceServer::new(service)
        .max_decoding_message_size(params::MAX_GRPC_MESSAGE_SIZE)
        .max_encoding_message_size(params::MAX_GRPC_MESSAGE_SIZE)
        .accept_compressed(CompressionEncoding::Gzip)
        .accept_compressed(CompressionEncoding::Zstd)
        .send_compressed(CompressionEncoding::Gzip)
        .send_compressed(CompressionEncoding::Zstd),
);

grpc_routes.add_service(reflection_service);

let grpc_config = GrpcConfig::new(args.grpc_port, grpc_routes.routes());

let http_router = axum::Router::default();

let mut server = HttpServer::new(
    args.metrics_port,
    http_router,
    Some(grpc_config),
    CancellationToken::new(),
    Duration::from_secs(20),
)
.await?;

server.set_readiness(true);
info!("Server ready");
server.wait_for_termination().await;
info!("Server shutdown complete");
Ok(())
}

```

```

=====
FILE: home-mixer/server.rs
=====

```

```

use crate::candidate_pipeline::candidate::CandidateHelpers;
use crate::candidate_pipeline::phoenix_candidate_pipeline::PhoenixCandidatePipeline;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use log::info;
use std::sync::Arc;
use std::time::Instant;
use tonic::{Request, Response, Status};
use xai_candidate_pipeline::candidate_pipeline::CandidatePipeline;
use xai_home_mixer_proto as pb;
use xai_home_mixer_proto::{ScoredPost, ScoredPostsResponse};

pub struct HomeMixerServer {
    phx_candidate_pipeline: Arc<PhoenixCandidatePipeline>,
}

impl HomeMixerServer {
    pub async fn new() -> Self {
        HomeMixerServer {
            phx_candidate_pipeline: Arc::new(PhoenixCandidatePipeline::prod().await),
        }
    }
}

```

```

    }
}

#[tonic::async_trait]
impl pb::scored_posts_service_server::ScoredPostsService for HomeMixerServer {
    #[xai_stats_macro::receive_stats]
    async fn get_scored_posts(
        &self,
        request: Request<pb::ScoredPostsQuery>,
    ) -> Result<Response<ScoredPostsResponse>, Status> {
        let proto_query = request.into_inner();

        if proto_query.viewer_id == 0 {
            return Err(Status::invalid_argument("viewer_id must be specified"));
        }

        let start = Instant::now();
        let query = ScoredPostsQuery::new(
            proto_query.viewer_id,
            proto_query.client_app_id,
            proto_query.country_code,
            proto_query.language_code,
            proto_query.seen_ids,
            proto_query.served_ids,
            proto_query.in_network_only,
            proto_query.is_bottom_request,
            proto_query.bloom_filter_entries,
        );
        info!("Scored Posts request - request_id {}", query.request_id);
        let pipeline_result = self.phx_candidate_pipeline.execute(query).await;

        let scored_posts: Vec<ScoredPost> = pipeline_result
            .selected_candidates
            .into_iter()
            .map(|candidate| {
                let screen_names = candidate.get_screen_names();
                ScoredPost {
                    tweet_id: candidate.tweet_id as u64,
                    author_id: candidate.author_id,
                    retweeted_tweet_id: candidate.retweeted_tweet_id.unwrap_or(0),
                    retweeted_user_id: candidate.retweeted_user_id.unwrap_or(0),
                    in_reply_to_tweet_id: candidate.in_reply_to_tweet_id.unwrap_or(0),
                    score: candidate.score.unwrap_or(0.0) as f32,
                    in_network: candidate.in_network.unwrap_or(false),
                    served_type: candidate.served_type.map(|t| t as
i32).unwrap_or_default(),
                    last_scored_timestamp_ms: candidate.last_scored_at_ms.unwrap_or(0),
                    prediction_request_id:
candidate.prediction_request_id.unwrap_or(0),
                    ancestors: candidate.ancestors,
                    screen_names,
                    visibility_reason: candidate.visibility_reason.map(|r| r.into()),
                }
            })
            .collect();

        info!(
            "Scored Posts response - request_id {} - {} posts ({} ms)",
            pipeline_result.query.request_id,
            scored_posts.len(),
            start.elapsed().as_millis()
        );
        Ok(Response::new(ScoredPostsResponse { scored_posts }))
    }
}

```

```
}
```

```
=====
FILE: home-mixer/candidate_hydrators/core_data_candidate_hydrator.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::tweet_entity_service_client::TESClient;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;

pub struct CoreDataCandidateHydrator {
    pub tes_client: Arc<dyn TESClient + Send + Sync>,
}

impl CoreDataCandidateHydrator {
    pub async fn new(tes_client: Arc<dyn TESClient + Send + Sync>) -> Self {
        Self { tes_client }
    }
}

#[async_trait]
impl Hydrator<ScoredPostsQuery, PostCandidate> for CoreDataCandidateHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let client = &self.tes_client;

        let tweet_ids = candidates.iter().map(|c| c.tweet_id).collect::<Vec<_>>();

        let post_features = client.get_tweet_core_datas(tweet_ids.clone()).await;
        let post_features = post_features.map_err(|e| e.to_string())?;

        let mut hydrated_candidates = Vec::with_capacity(candidates.len());
        for tweet_id in tweet_ids {
            let post_features = post_features.get(&tweet_id);
            let core_data = post_features.and_then(|x| x.as_ref());
            let text = core_data.map(|x| x.text.clone());
            let hydrated = PostCandidate {
                author_id: core_data.map(|x| x.author_id).unwrap_or_default(),
                retweeted_user_id: core_data.and_then(|x| x.source_user_id),
                retweeted_tweet_id: core_data.and_then(|x| x.source_tweet_id),
                in_reply_to_tweet_id: core_data.and_then(|x| x.in_reply_to_tweet_id),
                tweet_text: text.unwrap_or_default(),
                ..Default::default()
            };
            hydrated_candidates.push(hydrated);
        }

        Ok(hydrated_candidates)
    }
}

fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
    candidate.retweeted_user_id = hydrated.retweeted_user_id;
    candidate.retweeted_tweet_id = hydrated.retweeted_tweet_id;
    candidate.in_reply_to_tweet_id = hydrated.in_reply_to_tweet_id;
    candidate.tweet_text = hydrated.tweet_text;
}
```

```
}
```

```
=====
FILE: home-mixer/candidate_hydrators/gizmoduck_hydrator.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::gizmoduck_client::GizmoduckClient;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;

pub struct GizmoduckCandidateHydrator {
    pub gizmoduck_client: Arc<dyn GizmoduckClient + Send + Sync>,
}

impl GizmoduckCandidateHydrator {
    pub async fn new(gizmoduck_client: Arc<dyn GizmoduckClient + Send + Sync>) -> Self {
        Self { gizmoduck_client }
    }
}

#[async_trait]
impl Hydrator<ScoredPostsQuery, PostCandidate> for GizmoduckCandidateHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let client = &self.gizmoduck_client;

        let author_ids: Vec<_> = candidates.iter().map(|c| c.author_id).collect();
        let author_ids: Vec<_> = author_ids.iter().map(|&x| x as i64).collect();
        let retweet_user_ids: Vec<_> = candidates.iter().map(|c|
c.retweeted_user_id).collect();
        let retweet_user_ids: Vec<_> = retweet_user_ids.iter().flatten().collect();
        let retweet_user_ids: Vec<_> = retweet_user_ids.iter().map(|&&x| x as
i64).collect();

        let mut user_ids_to_fetch = Vec::with_capacity(author_ids.len() +
retweet_user_ids.len());
        user_ids_to_fetch.extend(author_ids);
        user_ids_to_fetch.extend(retweet_user_ids);
        user_ids_to_fetch.dedup();

        let users = client.get_users(user_ids_to_fetch).await;
        let users = users.map_err(|e| e.to_string())?;

        let mut hydrated_candidates = Vec::with_capacity(candidates.len());

        for candidate in candidates {
            let user = users
                .get(&(candidate.author_id as i64))
                .and_then(|user| user.as_ref());
            let user_counts = user.and_then(|user| user.user.as_ref().map(|u|
&u.counts));
            let user_profile = user.and_then(|user| user.user.as_ref().map(|u|
&u.profile));

            let author_followers_count: Option<i32> =
```

```

        user_counts.map(|x| x.followers_count).map(|x| x as i32);
        let author_screen_name: Option<String> = user_profile.map(|x|
x.screen_name.clone());

        let retweet_user = candidate
            .retweeted_user_id
            .and_then(|retweeted_user_id| users.get(&(retweeted_user_id as i64)))
            .and_then(|user| user.as_ref());
        let retweet_profile =
            retweet_user.and_then(|user| user.user.as_ref().map(|u| &u.profile));
        let retweeted_screen_name: Option<String> =
            retweet_profile.map(|x| x.screen_name.clone());

        let hydrated = PostCandidate {
            author_followers_count,
            author_screen_name,
            retweeted_screen_name,
            ..Default::default()
        };
        hydrated_candidates.push(hydrated);
    }

    Ok(hydrated_candidates)
}

fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
    candidate.author_followers_count = hydrated.author_followers_count;
    candidate.author_screen_name = hydrated.author_screen_name;
    candidate.retweeted_screen_name = hydrated.retweeted_screen_name;
}
}

```

```

=====
FILE: home-mixer/candidate_hydrators/in_network_candidate_hydrator.rs
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::collections::HashSet;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;

pub struct InNetworkCandidateHydrator;

#[async_trait]
impl Hydrator<ScoredPostsQuery, PostCandidate> for InNetworkCandidateHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let viewer_id = query.user_id as u64;
        let followed_ids: HashSet<u64> = query
            .user_features
            .followed_user_ids
            .iter()
            .copied()
            .map(|id| id as u64)
            .collect();

        let hydrated_candidates = candidates
            .iter()

```

```

        .map(|candidate| {
            let is_self = candidate.author_id == viewer_id;
            let is_in_network = is_self ||
followed_ids.contains(&candidate.author_id);
            PostCandidate {
                in_network: Some(is_in_network),
                ..Default::default()
            }
        })
        .collect();

    Ok(hydrated_candidates)
}

fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
    candidate.in_network = hydrated.in_network;
}
}

```

```

=====
FILE: home-mixer/candidate_hydrators/mod.rs
=====

```

```

pub mod core_data_candidate_hydrator;
pub mod gizmoduck_hydrator;
pub mod in_network_candidate_hydrator;
pub mod subscription_hydrator;
pub mod vf_candidate_hydrator;
pub mod video_duration_candidate_hydrator;

```

```

=====
FILE: home-mixer/candidate_hydrators/subscription_hydrator.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::tweet_entity_service_client::TESClient;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;

pub struct SubscriptionHydrator {
    pub tes_client: Arc<dyn TESClient + Send + Sync>,
}

impl SubscriptionHydrator {
    pub async fn new(tes_client: Arc<dyn TESClient + Send + Sync>) -> Self {
        Self { tes_client }
    }
}

#[async_trait]
impl Hydrator<ScoredPostsQuery, PostCandidate> for SubscriptionHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let client = &self.tes_client;

        let tweet_ids = candidates.iter().map(|c| c.tweet_id).collect::<Vec<_>>();
    }
}

```

```

        let post_features =
client.get_subscription_author_ids(tweet_ids.clone()).await;
        let post_features = post_features.map_err(|e| e.to_string())?;

        let mut hydrated_candidates = Vec::with_capacity(candidates.len());
        for tweet_id in tweet_ids {
            let post_features = post_features.get(&tweet_id);
            let subscription_author_id = post_features.and_then(|x| *x);
            let hydrated = PostCandidate {
                subscription_author_id,
                ..Default::default()
            };
            hydrated_candidates.push(hydrated);
        }

        Ok(hydrated_candidates)
    }

    fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
        candidate.subscription_author_id = hydrated.subscription_author_id;
    }
}

```

=====

FILE: home-mixer/candidate_hydrators/vf_candidate_hydrator.rs

=====

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use futures::future::join;
use std::collections::HashMap;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;
use xai_twittercontext_proto::GetTwitterContextViewer;
use xai_twittercontext_proto::TwitterContextViewer;
use xai_visibility_filtering::models::FilteredReason;
use xai_visibility_filtering::vf_client::SafetyLevel;
use xai_visibility_filtering::vf_client::SafetyLevel:: {TimelineHome,
TimelineHomeRecommendations};
use xai_visibility_filtering::vf_client::VisibilityFilteringClient;

pub struct VFCandidateHydrator {
    pub vf_client: Arc<dyn VisibilityFilteringClient + Send + Sync>,
}

impl VFCandidateHydrator {
    pub async fn new(vf_client: Arc<dyn VisibilityFilteringClient + Send + Sync>) ->
Self {
    Self { vf_client }
}

    async fn fetch_vf_results(
        client: &Arc<dyn VisibilityFilteringClient + Send + Sync>,
        tweet_ids: Vec<i64>,
        safety_level: SafetyLevel,
        for_user_id: i64,
        context: Option<TwitterContextViewer>,
    ) -> Result<HashMap<i64, Option<FilteredReason>>, String> {
        if tweet_ids.is_empty() {
            return Ok(HashMap::new());
        }
    }
}

```

```

        client
            .get_result(tweet_ids, safety_level, for_user_id, context)
            .await
            .map_err(|e| e.to_string())
    }
}

#[async_trait]
impl Hydrator<ScoredPostsQuery, PostCandidate> for VFCandidateHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let context = query.get_viewer();
        let user_id = query.user_id;
        let client = &self.vf_client;

        let mut in_network_ids = Vec::new();
        let mut oon_ids = Vec::new();
        for candidate in candidates.iter() {
            if candidate.in_network.unwrap_or(false) {
                in_network_ids.push(candidate.tweet_id);
            } else {
                oon_ids.push(candidate.tweet_id);
            }
        }

        let in_network_future = Self::fetch_vf_results(
            client,
            in_network_ids,
            TimelineHome,
            user_id,
            context.clone(),
        );

        let oon_future = Self::fetch_vf_results(
            client,
            oon_ids,
            TimelineHomeRecommendations,
            user_id,
            context,
        );

        let (in_network_result, oon_result) = join(in_network_future,
oon_future).await;
        let mut result: HashMap<i64, Option<FilteredReason>> = HashMap::new();
        result.extend(in_network_result?);
        result.extend(oon_result?);

        let mut hydrated_candidates = Vec::with_capacity(candidates.len());
        for candidate in candidates {
            let visibility_reason = result.get(&candidate.tweet_id);
            let visibility_reason = visibility_reason.unwrap_or(&None);
            let hydrated = PostCandidate {
                visibility_reason: visibility_reason.clone(),
                ..Default::default()
            };
            hydrated_candidates.push(hydrated);
        }
        Ok(hydrated_candidates)
    }
}

```

```

    fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
        candidate.visibility_reason = hydrated.visibility_reason;
    }
}

```

```

=====
FILE: home-mixer/candidate_hydrators/video_duration_candidate_hydrator.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::candidate_features::MediaInfo;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::tweet_entity_service_client::TESClient;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::hydrator::Hydrator;

```

```

pub struct VideoDurationCandidateHydrator {
    pub tes_client: Arc<dyn TESClient + Send + Sync>,
}

```

```

impl VideoDurationCandidateHydrator {
    pub async fn new(tes_client: Arc<dyn TESClient + Send + Sync>) -> Self {
        Self { tes_client }
    }
}

```

```

#[async_trait]

```

```

impl Hydrator<ScoredPostsQuery, PostCandidate> for VideoDurationCandidateHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let client = &self.tes_client;

        let tweet_ids = candidates.iter().map(|c| c.tweet_id).collect::<Vec<_>>();

        let post_features = client.get_tweet_media_entities(tweet_ids.clone()).await;
        let post_features = post_features.map_err(|e| e.to_string())?;

        let mut hydrated_candidates = Vec::with_capacity(candidates.len());
        for tweet_id in tweet_ids {
            let post_features = post_features.get(&tweet_id);
            let media_entities = post_features.and_then(|x| x.as_ref());

            let video_duration_ms = media_entities.and_then(|entities| {
                entities.iter().find_map(|entity| {
                    if let Some(MediaInfo::VideoInfo(video_info)) = &entity.media_info
                {
                    Some(video_info.duration_millis)
                } else {
                    None
                }
            })
        });

            let hydrated = PostCandidate {
                video_duration_ms,
                ..Default::default()
            };
        }
    }
}

```

```

        hydrated_candidates.push(hydrated);
    }

    Ok(hydrated_candidates)
}

fn update(&self, candidate: &mut PostCandidate, hydrated: PostCandidate) {
    candidate.video_duration_ms = hydrated.video_duration_ms;
}
}

```

```

=====
FILE: home-mixer/candidate_pipeline/candidate.rs
=====

```

```

use std::collections::HashMap;
use xai_home_mixer_proto as pb;
use xai_visibility_filtering::models as vf;

#[derive(Clone, Debug, Default)]
pub struct PostCandidate {
    pub tweet_id: i64,
    pub author_id: u64,
    pub tweet_text: String,
    pub in_reply_to_tweet_id: Option<u64>,
    pub retweeted_tweet_id: Option<u64>,
    pub retweeted_user_id: Option<u64>,
    pub phoenix_scores: PhoenixScores,
    pub prediction_request_id: Option<u64>,
    pub last_scored_at_ms: Option<u64>,
    pub weighted_score: Option<f64>,
    pub score: Option<f64>,
    pub served_type: Option<pb::ServedType>,
    pub in_network: Option<bool>,
    pub ancestors: Vec<u64>,
    pub video_duration_ms: Option<i32>,
    pub author_followers_count: Option<i32>,
    pub author_screen_name: Option<String>,
    pub retweeted_screen_name: Option<String>,
    pub visibility_reason: Option<vf::FilteredReason>,
    pub subscription_author_id: Option<u64>,
}

```

```

#[derive(Clone, Debug, Default)]
pub struct PhoenixScores {
    pub favorite_score: Option<f64>,
    pub reply_score: Option<f64>,
    pub retweet_score: Option<f64>,
    pub photo_expand_score: Option<f64>,
    pub click_score: Option<f64>,
    pub profile_click_score: Option<f64>,
    pub vqv_score: Option<f64>,
    pub share_score: Option<f64>,
    pub share_via_dm_score: Option<f64>,
    pub share_via_copy_link_score: Option<f64>,
    pub dwell_score: Option<f64>,
    pub quote_score: Option<f64>,
    pub quoted_click_score: Option<f64>,
    pub follow_author_score: Option<f64>,
    pub not_interested_score: Option<f64>,
    pub block_author_score: Option<f64>,
    pub mute_author_score: Option<f64>,
    pub report_score: Option<f64>,
}

```

```

    // Continuous actions
    pub dwell_time: Option<f64>,
}

pub trait CandidateHelpers {
    fn get_screen_names(&self) -> HashMap<u64, String>;
}

impl CandidateHelpers for PostCandidate {
    fn get_screen_names(&self) -> HashMap<u64, String> {
        let mut screen_names = HashMap::<u64, String>::new();
        if let Some(author_screen_name) = self.author_screen_name.clone() {
            screen_names.insert(self.author_id, author_screen_name);
        }
        if let (Some(retweeted_screen_name), Some(retweeted_user_id)) =
            (self.retweeted_screen_name.clone(), self.retweeted_user_id)
        {
            screen_names.insert(retweeted_user_id, retweeted_screen_name);
        }
        screen_names
    }
}

```

=====

FILE: home-mixer/candidate_pipeline/candidate_features.rs

=====

```

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct PureCoreData {
    pub author_id: u64,
    pub text: String,
    pub source_tweet_id: Option<u64>,
    pub source_user_id: Option<u64>,
    pub in_reply_to_tweet_id: Option<u64>,
    pub in_reply_to_user_id: Option<u64>,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct ExclusiveTweetControl {
    pub conversation_author_id: i64,
}

pub type MediaEntities = Vec<MediaEntity>;

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct MediaEntity {
    pub media_info: Option<MediaInfo>,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "camelCase")]
pub enum MediaInfo {
    VideoInfo(VideoInfo),
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct VideoInfo {

```

```

    pub duration_millis: i32,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct Share {
    pub source_tweet_id: u64,
    pub source_user_id: u64,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct Reply {
    pub in_reply_to_tweet_id: Option<u64>,
    pub in_reply_to_user_id: u64,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct GizmoduckUserCounts {
    pub followers_count: u32,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct GizmoduckUserProfile {
    pub screen_name: String,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct GizmoduckUser {
    pub user_id: u64,
    pub profile: GizmoduckUserProfile,
    pub counts: GizmoduckUserCounts,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct GizmoduckUserResult {
    pub user: Option<GizmoduckUser>,
}

```

```

=====
FILE: home-mixer/candidate_pipeline/mod.rs
=====

```

```

pub mod candidate;
pub mod candidate_features;
pub mod phoenix_candidate_pipeline;
pub mod query;
pub mod query_features;

```

```

=====
FILE: home-mixer/candidate_pipeline/phoenix_candidate_pipeline.rs
=====

```

```

use
crate::candidate_hydrators::core_data_candidate_hydrator::CoreDataCandidateHydrator;
use crate::candidate_hydrators::gizmoduck_hydrator::GizmoduckCandidateHydrator;
use
crate::candidate_hydrators::in_network_candidate_hydrator::InNetworkCandidateHydrator;

```

```

use crate::candidate_hydrators::subscription_hydrator::SubscriptionHydrator;
use crate::candidate_hydrators::vf_candidate_hydrator::VFCandidateHydrator;
use
crate::candidate_hydrators::video_duration_candidate_hydrator::VideoDurationCandidateHydrator;
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::gizmoduck_client::{GizmoduckClient, ProdGizmoduckClient};
use crate::clients::phoenix_prediction_client::{
    PhoenixPredictionClient, ProdPhoenixPredictionClient,
};
use crate::clients::phoenix_retrieval_client::{
    PhoenixRetrievalClient, ProdPhoenixRetrievalClient,
};
use crate::clients::s2s::{S2S_CHAIN_PATH, S2S_CRT_PATH, S2S_KEY_PATH};
use crate::clients::socialgraph_client::SocialGraphClient;
use crate::clients::strato_client::{ProdStratoClient, StratoClient};
use crate::clients::thunder_client::ThunderClient;
use crate::clients::tweet_entity_service_client::{ProdTESClient, TESClient};
use crate::clients::uas_fetcher::UserActionSequenceFetcher;
use crate::filters::age_filter::AgeFilter;
use crate::filters::author_socialgraph_filter::AuthorSocialgraphFilter;
use crate::filters::core_data_hydration_filter::CoreDataHydrationFilter;
use crate::filters::dedup_conversation_filter::DedupConversationFilter;
use crate::filters::drop_duplicates_filter::DropDuplicatesFilter;
use crate::filters::ineligible_subscription_filter::IneligibleSubscriptionFilter;
use crate::filters::muted_keyword_filter::MutedKeywordFilter;
use crate::filters::previously_seen_posts_filter::PreviouslySeenPostsFilter;
use crate::filters::previously_served_posts_filter::PreviouslyServedPostsFilter;
use crate::filters::retweet_deduplication_filter::RetweetDeduplicationFilter;
use crate::filters::self_tweet_filter::SelfTweetFilter;
use crate::filters::vf_filter::VFFilter;
use crate::params;
use crate::query_hydrators::user_action_seq_query_hydrator::UserActionSeqQueryHydrator;
use crate::query_hydrators::user_features_query_hydrator::UserFeaturesQueryHydrator;
use crate::scorers::author_diversity_scorer::AuthorDiversityScorer;
use crate::scorers::oon_scorer::OONScorer;
use crate::scorers::phoenix_scorer::PhoenixScorer;
use crate::scorers::weighted_scorer::WeightedScorer;
use crate::selectors::TopKScoreSelector;
use crate::side_effects::cache_request_info_side_effect::CacheRequestInfoSideEffect;
use crate::sources::phoenix_source::PhoenixSource;
use crate::sources::thunder_source::ThunderSource;
use std::sync::Arc;
use std::time::Duration;
use tonic::async_trait;
use xai_candidate_pipeline::candidate_pipeline::CandidatePipeline;
use xai_candidate_pipeline::filter::Filter;
use xai_candidate_pipeline::hydrator::Hydrator;
use xai_candidate_pipeline::query_hydrator::QueryHydrator;
use xai_candidate_pipeline::scorer::Scorer;
use xai_candidate_pipeline::selector::Selector;
use xai_candidate_pipeline::side_effect::SideEffect;
use xai_candidate_pipeline::source::Source;
use xai_visibility_filtering::vf_client::{
    ProdVisibilityFilteringClient, VisibilityFilteringClient,
};

pub struct PhoenixCandidatePipeline {
    query_hydrators: Vec<Box<dyn QueryHydrator<ScoredPostsQuery>>>,
    sources: Vec<Box<dyn Source<ScoredPostsQuery, PostCandidate>>>,
    hydrators: Vec<Box<dyn Hydrator<ScoredPostsQuery, PostCandidate>>>,
    filters: Vec<Box<dyn Filter<ScoredPostsQuery, PostCandidate>>>,
    scorers: Vec<Box<dyn Scorer<ScoredPostsQuery, PostCandidate>>>,

```

```

selector: TopKScoreSelector,
post_selection_hydrators: Vec<Box<dyn Hydrator<ScoredPostsQuery, PostCandidate>>>,
post_selection_filters: Vec<Box<dyn Filter<ScoredPostsQuery, PostCandidate>>>,
side_effects: Arc<Vec<Box<dyn SideEffect<ScoredPostsQuery, PostCandidate>>>>,
}

impl PhoenixCandidatePipeline {
    async fn build_with_clients(
        uas_fetcher: Arc<UserActionSequenceFetcher>,
        phoenix_client: Arc<dyn PhoenixPredictionClient + Send + Sync>,
        phoenix_retrieval_client: Arc<dyn PhoenixRetrievalClient + Send + Sync>,
        thunder_client: Arc<ThunderClient>,
        strato_client: Arc<dyn StratoClient + Send + Sync>,
        tes_client: Arc<dyn TESClient + Send + Sync>,
        gizmoduck_client: Arc<dyn GizmoduckClient + Send + Sync>,
        vf_client: Arc<dyn VisibilityFilteringClient + Send + Sync>,
    ) -> PhoenixCandidatePipeline {
        // Query Hydrators
        let query_hydrators: Vec<Box<dyn QueryHydrator<ScoredPostsQuery>>> = vec![
            Box::new(UserActionSeqQueryHydrator::new(uas_fetcher)),
            Box::new(UserFeaturesQueryHydrator {
                strato_client: strato_client.clone(),
            }),
        ];

        // Sources
        let phoenix_source = Box::new(PhoenixSource {
            phoenix_retrieval_client,
        });
        let thunder_source = Box::new(ThunderSource { thunder_client });
        let sources: Vec<Box<dyn Source<ScoredPostsQuery, PostCandidate>>> =
            vec![phoenix_source, thunder_source];

        // Hydrators
        let hydrators: Vec<Box<dyn Hydrator<ScoredPostsQuery, PostCandidate>>> = vec![
            Box::new(InNetworkCandidateHydrator),
            Box::new(CoreDataCandidateHydrator::new(tes_client.clone()).await),
            Box::new(VideoDurationCandidateHydrator::new(tes_client.clone()).await),
            Box::new(SubscriptionHydrator::new(tes_client.clone()).await),
            Box::new(GizmoduckCandidateHydrator::new(gizmoduck_client).await),
        ];

        // Filters
        let filters: Vec<Box<dyn Filter<ScoredPostsQuery, PostCandidate>>> = vec![
            Box::new(DropDuplicatesFilter),
            Box::new(CoreDataHydrationFilter),
            Box::new(AgeFilter::new(Duration::from_secs(params::MAX_POST_AGE))),
            Box::new(SelfTweetFilter),
            Box::new(RetweetDeduplicationFilter),
            Box::new(IneligibleSubscriptionFilter),
            Box::new(PreviouslySeenPostsFilter),
            Box::new(PreviouslyServedPostsFilter),
            Box::new(MutedKeywordFilter::new()),
            Box::new(AuthorSocialgraphFilter),
        ];

        // Scorers
        let phoenix_scorer = Box::new(PhoenixScorer { phoenix_client });
        let weighted_scorer = Box::new(WeightedScorer);
        let author_diversity_scorer = Box::new(AuthorDiversityScorer::default());
        let oon_scorer = Box::new(OONScorer);
        let scorers: Vec<Box<dyn Scorer<ScoredPostsQuery, PostCandidate>>> = vec![
            phoenix_scorer,
            weighted_scorer,

```

```

        author_diversity_scorer,
        oon_scorer,
    ];

    // Selector
    let selector = TopKScoreSelector;

    // Post-selection hydrators
    let post_selection_hydrators: Vec<Box<dyn Hydrator<ScoredPostsQuery,
PostCandidate>>>> =
        vec![Box::new(VFCandidateHydrator::new(vf_client.clone()).await)];

    // Post-selection filters
    let post_selection_filters: Vec<Box<dyn Filter<ScoredPostsQuery,
PostCandidate>>>> =
        vec![Box::new(VFFilter), Box::new(DedupConversationFilter)];

    // Side Effects
    let side_effects: Arc<Vec<Box<dyn SideEffect<ScoredPostsQuery,
PostCandidate>>>> =
        Arc::new(vec![Box::new(CacheRequestInfoSideEffect { strato_client })]);

    PhoenixCandidatePipeline {
        query_hydrators,
        hydrators,
        filters,
        sources,
        scorers,
        selector,
        post_selection_hydrators,
        post_selection_filters,
        side_effects,
    }
}

pub async fn prod() -> PhoenixCandidatePipeline {
    let uas_fetcher =
        Arc::new(UserActionSequenceFetcher::new().expect("Failed to create UAS
fetcher"));
    let _sgs_client = Arc::new(SocialGraphClient::new());
    let phoenix_client = Arc::new(
        ProdPhoenixPredictionClient::new()
            .await
            .expect("Failed to create Phoenix prediction client"),
    );
    let phoenix_retrieval_client = Arc::new(
        ProdPhoenixRetrievalClient::new()
            .await
            .expect("Failed to create Phoenix retrieval client"),
    );
    let thunder_client = Arc::new(ThunderClient::new().await);
    let strato_client = Arc::new(
        ProdStratoClient::new()
            .await
            .expect("Failed to create Strato client"),
    );
    let tes_client = Arc::new(
        ProdTESClient::new()
            .await
            .expect("Failed to create TES client"),
    );
    let gizmoduck_client = Arc::new(
        ProdGizmoduckClient::new()
            .await

```

```

        .expect("Failed to create Gizmoduck client"),
    );
    let vf_client = Arc::new(
        ProdVisibilityFilteringClient::new(
            S2S_CHAIN_PATH.clone(),
            S2S_CRT_PATH.clone(),
            S2S_KEY_PATH.clone()
        )
        .await
        .expect("Failed to create VF client"),
    );
    PhoenixCandidatePipeline::build_with_clients(
        uas_fetcher,
        phoenix_client,
        phoenix_retrieval_client,
        thunder_client,
        strato_client,
        tes_client,
        gizmoduck_client,
        vf_client,
    )
    .await
}

#[async_trait]
impl CandidatePipeline<ScoredPostsQuery, PostCandidate> for PhoenixCandidatePipeline {
    fn query_hydrators(&self) -> &[Box<dyn QueryHydrator<ScoredPostsQuery>>] {
        &self.query_hydrators
    }

    fn sources(&self) -> &[Box<dyn Source<ScoredPostsQuery, PostCandidate>>] {
        &self.sources
    }

    fn hydrators(&self) -> &[Box<dyn Hydrator<ScoredPostsQuery, PostCandidate>>] {
        &self.hydrators
    }

    fn filters(&self) -> &[Box<dyn Filter<ScoredPostsQuery, PostCandidate>>] {
        &self.filters
    }

    fn scorers(&self) -> &[Box<dyn Scorer<ScoredPostsQuery, PostCandidate>>] {
        &self.scorers
    }

    fn selector(&self) -> &dyn Selector<ScoredPostsQuery, PostCandidate> {
        &self.selector
    }

    fn post_selection_hydrators(&self) -> &[Box<dyn Hydrator<ScoredPostsQuery,
PostCandidate>>] {
        &self.post_selection_hydrators
    }

    fn post_selection_filters(&self) -> &[Box<dyn Filter<ScoredPostsQuery,
PostCandidate>>] {
        &self.post_selection_filters
    }

    fn side_effects(&self) -> Arc<Vec<Box<dyn SideEffect<ScoredPostsQuery,
PostCandidate>>>> {
        Arc::clone(&self.side_effects)
    }
}

```

```

    fn result_size(&self) -> usize {
        params::RESULT_SIZE
    }
}

```

```

=====
FILE: home-mixer/candidate_pipeline/query.rs
=====

```

```

use crate::candidate_pipeline::query_features::UserFeatures;
use crate::util::request_util::generate_request_id;
use xai_candidate_pipeline::candidate_pipeline::HasRequestId;
use xai_home_mixer_proto::ImpressionBloomFilterEntry;
use xai_twittercontext_proto::{GetTwitterContextViewer, TwitterContextViewer};

```

```

#[derive(Clone, Default, Debug)]

```

```

pub struct ScoredPostsQuery {
    pub user_id: i64,
    pub client_app_id: i32,
    pub country_code: String,
    pub language_code: String,
    pub seen_ids: Vec<i64>,
    pub served_ids: Vec<i64>,
    pub in_network_only: bool,
    pub is_bottom_request: bool,
    pub bloom_filter_entries: Vec<ImpressionBloomFilterEntry>,
    pub user_action_sequence: Option<xai_recsys_proto::UserActionSequence>,
    pub user_features: UserFeatures,
    pub request_id: String,
}

```

```

impl ScoredPostsQuery {
    pub fn new(
        user_id: i64,
        client_app_id: i32,
        country_code: String,
        language_code: String,
        seen_ids: Vec<i64>,
        served_ids: Vec<i64>,
        in_network_only: bool,
        is_bottom_request: bool,
        bloom_filter_entries: Vec<ImpressionBloomFilterEntry>,
    ) -> Self {
        let request_id = format!("{}", generate_request_id(), user_id);
        Self {
            user_id,
            client_app_id,
            country_code,
            language_code,
            seen_ids,
            served_ids,
            in_network_only,
            is_bottom_request,
            bloom_filter_entries,
            user_action_sequence: None,
            user_features: UserFeatures::default(),
            request_id,
        }
    }
}

```

```

impl GetTwitterContextViewer for ScoredPostsQuery {

```

```

    fn get_viewer(&self) -> Option<TwitterContextViewer> {
        Some(TwitterContextViewer {
            user_id: self.user_id,
            client_application_id: self.client_app_id as i64,
            request_country_code: self.country_code.clone(),
            request_language_code: self.language_code.clone(),
            ..Default::default()
        })
    }
}

impl HasRequestId for ScoredPostsQuery {
    fn request_id(&self) -> &str {
        &self.request_id
    }
}

```

```

=====
FILE: home-mixer/candidate_pipeline/query_features.rs
=====

```

```

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Default)]
#[serde(rename_all = "camelCase")]
pub struct UserFeatures {
    pub muted_keywords: Vec<String>,
    pub blocked_user_ids: Vec<i64>,
    pub muted_user_ids: Vec<i64>,
    pub followed_user_ids: Vec<i64>,
    pub subscribed_user_ids: Vec<i64>,
}

```

```

=====
FILE: home-mixer/filters/age_filter.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::util::snowflake;
use std::time::Duration;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

/// Filter that removes tweets older than a specified duration.
pub struct AgeFilter {
    pub max_age: Duration,
}

impl AgeFilter {
    pub fn new(max_age: Duration) -> Self {
        Self { max_age }
    }

    fn is_within_age(&self, tweet_id: i64) -> bool {
        snowflake::duration_since_creation_opt(tweet_id)
            .map(|age| age <= self.max_age)
            .unwrap_or(false)
    }
}

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for AgeFilter {

```

```

    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let (kept, removed): (Vec<_>, Vec<_>) = candidates
            .into_iter()
            .partition(|c| self.is_within_age(c.tweet_id));

        Ok(FilterResult { kept, removed })
    }
}

```

FILE: home-mixer/filters/author_socialgraph_filter.rs

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

// Remove candidates that are blocked or muted by the viewer
pub struct AuthorSocialgraphFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for AuthorSocialgraphFilter {
    async fn filter(
        &self,
        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let viewer_blocked_user_ids = query.user_features.blocked_user_ids.clone();
        let viewer_muted_user_ids = query.user_features.muted_user_ids.clone();

        if viewer_blocked_user_ids.is_empty() && viewer_muted_user_ids.is_empty() {
            return Ok(FilterResult {
                kept: candidates,
                removed: Vec::new(),
            });
        }

        let mut kept: Vec<PostCandidate> = Vec::new();
        let mut removed: Vec<PostCandidate> = Vec::new();

        for candidate in candidates {
            let author_id = candidate.author_id as i64;
            let muted = viewer_muted_user_ids.contains(&author_id);
            let blocked = viewer_blocked_user_ids.contains(&author_id);
            if muted || blocked {
                removed.push(candidate);
            } else {
                kept.push(candidate);
            }
        }

        Ok(FilterResult { kept, removed })
    }
}

```

FILE: home-mixer/filters/core_data_hydration_filter.rs

```
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

pub struct CoreDataHydrationFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for CoreDataHydrationFilter {
    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let (kept, removed) = candidates
            .into_iter()
            .partition(|c| c.author_id != 0 && !c.tweet_text.trim().is_empty());
        Ok(FilterResult { kept, removed })
    }
}
```

FILE: home-mixer/filters/dedup_conversation_filter.rs

```
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::collections::HashMap;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

/// Keeps only the highest-scored candidate per branch of a conversation tree
pub struct DedupConversationFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for DedupConversationFilter {
    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let mut kept: Vec<PostCandidate> = Vec::new();
        let mut removed: Vec<PostCandidate> = Vec::new();
        let mut best_per_convo: HashMap<u64, (usize, f64)> = HashMap::new();

        for candidate in candidates {
            let conversation_id = get_conversation_id(&candidate);
            let score = candidate.score.unwrap_or(0.0);

            if let Some((kept_idx, best_score)) =
                best_per_convo.get_mut(&conversation_id) {
                if score > *best_score {
                    let previous = std::mem::replace(&mut kept[*kept_idx], candidate);
                    removed.push(previous);
                    *best_score = score;
                } else {
                    removed.push(candidate);
                }
            } else {
                let idx = kept.len();
                best_per_convo.insert(conversation_id, (idx, score));
            }
        }

        Ok(FilterResult { kept, removed })
    }
}
```

```

        kept.push(candidate);
    }
}

Ok(FilterResult { kept, removed })
}

fn get_conversation_id(candidate: &PostCandidate) -> u64 {
    candidate
        .ancestors
        .iter()
        .copied()
        .min()
        .unwrap_or(candidate.tweet_id as u64)
}

```

```

=====
FILE: home-mixer/filters/drop_duplicates_filter.rs
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::collections::HashSet;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

pub struct DropDuplicatesFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for DropDuplicatesFilter {
    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let mut seen_ids = HashSet::new();
        let mut kept = Vec::new();
        let mut removed = Vec::new();

        for candidate in candidates {
            if seen_ids.insert(candidate.tweet_id) {
                kept.push(candidate);
            } else {
                removed.push(candidate);
            }
        }

        Ok(FilterResult { kept, removed })
    }
}

```

```

=====
FILE: home-mixer/filters/ineligible_subscription_filter.rs
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::collections::HashSet;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

```

```
/// Filters out subscription-only posts from authors the viewer is not subscribed to.
pub struct IneligibleSubscriptionFilter;
```

```
#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for IneligibleSubscriptionFilter {
    async fn filter(
        &self,
        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let subscribed_user_ids: HashSet<u64> = query
            .user_features
            .subscribed_user_ids
            .iter()
            .map(|id| *id as u64)
            .collect();

        let (kept, removed): (Vec<_>, Vec<_>) =
            candidates
                .into_iter()
                .partition(|candidate| match candidate.subscription_author_id {
                    Some(author_id) => subscribed_user_ids.contains(&author_id),
                    None => true,
                });

        Ok(FilterResult { kept, removed })
    }
}
```

```
=====
FILE: home-mixer/filters/mod.rs
=====
```

```
pub mod age_filter;
pub mod author_socialgraph_filter;
pub mod core_data_hydration_filter;
pub mod dedup_conversation_filter;
pub mod drop_duplicates_filter;

pub mod ineligible_subscription_filter;
pub mod muted_keyword_filter;
pub mod previously_seen_posts_filter;
pub mod previously_served_posts_filter;
pub mod retweet_deduplication_filter;
pub mod self_tweet_filter;
pub mod vf_filter;
```

```
=====
FILE: home-mixer/filters/muted_keyword_filter.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::filter::Filter, FilterResult;
use xai_post_text::MatchTweetGroup, TokenSequence, TweetTokenizer, UserMutes;

pub struct MutedKeywordFilter {
    pub tokenizer: Arc<TweetTokenizer>,
}
```

```

impl MutedKeywordFilter {
    pub fn new() -> Self {
        let tokenizer = TweetTokenizer::new();
        Self {
            tokenizer: Arc::new(tokenizer),
        }
    }
}

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for MutedKeywordFilter {
    #[xai_stats_macro::receive_stats]
    async fn filter(
        &self,
        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let muted_keywords = query.user_features.muted_keywords.clone();

        if muted_keywords.is_empty() {
            return Ok(FilterResult {
                kept: candidates,
                removed: vec![],
            });
        }

        let tokenized = muted_keywords.iter().map(|k| self.tokenizer.tokenize(k));
        let token_sequences: Vec<TokenSequence> = tokenized.collect::<Vec<_>>();
        let user_mutes = UserMutes::new(token_sequences);
        let matcher = MatchTweetGroup::new(user_mutes);

        let mut kept = Vec::new();
        let mut removed = Vec::new();

        for candidate in candidates {
            let tweet_text_token_sequence =
self.tokenizer.tokenize(&candidate.tweet_text);
            if matcher.matches(&tweet_text_token_sequence) {
                // Matches muted keywords - should be removed/filtered out
                removed.push(candidate);
            } else {
                // Does not match muted keywords - keep it
                kept.push(candidate);
            }
        }

        Ok(FilterResult { kept, removed })
    }
}

```

```

=====
FILE: home-mixer/filters/previously_seen_posts_filter.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::util::bloom_filter::BloomFilter;
use crate::util::candidates_util::get_related_post_ids;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

/// Filter out previously seen posts using a Bloom Filter and
/// the seen IDs sent in the request directly from the client

```

```

pub struct PreviouslySeenPostsFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for PreviouslySeenPostsFilter {
    async fn filter(
        &self,
        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let bloom_filters = query
            .bloom_filter_entries
            .iter()
            .map(BloomFilter::from_entry)
            .collect::<Vec<_>>();

        let (removed, kept): (Vec<_>, Vec<_>) = candidates.into_iter().partition(|c| {
            get_related_post_ids(c).iter().any(|&post_id| {
                query.seen_ids.contains(&post_id)
                || bloom_filters
                    .iter()
                    .any(|filter| filter.may_contain(post_id))
            })
        });

        Ok(FilterResult { kept, removed })
    }
}

```

```

=====
FILE: home-mixer/filters/previously_served_posts_filter.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::util::candidates_util::get_related_post_ids;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

pub struct PreviouslyServedPostsFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for PreviouslyServedPostsFilter {
    fn enable(&self, query: &ScoredPostsQuery) -> bool {
        query.is_bottom_request
    }

    async fn filter(
        &self,
        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let (removed, kept): (Vec<_>, Vec<_>) = candidates.into_iter().partition(|c| {
            get_related_post_ids(c)
                .iter()
                .any(|id| query.served_ids.contains(id))
        });

        Ok(FilterResult { kept, removed })
    }
}

```

```
=====
FILE: home-mixer/filters/retweet_deduplication_filter.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use std::collections::HashSet;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

/// Deduplicates retweets, keeping only the first occurrence of a tweet
/// (whether as an original or as a retweet).
pub struct RetweetDeduplicationFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for RetweetDeduplicationFilter {
    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let mut seen_tweet_ids: HashSet<u64> = HashSet::new();
        let mut kept = Vec::new();
        let mut removed = Vec::new();

        for candidate in candidates {
            match candidate.retweeted_tweet_id {
                Some(retweeted_id) => {
                    // Remove if we've already seen this tweet (as original or retweet)
                    if seen_tweet_ids.insert(retweeted_id) {
                        kept.push(candidate);
                    } else {
                        removed.push(candidate);
                    }
                }
                None => {
                    // Mark this original tweet ID as seen so retweets of it get
                    filtered
                    seen_tweet_ids.insert(candidate.tweet_id as u64);
                    kept.push(candidate);
                }
            }
        }

        Ok(FilterResult { kept, removed })
    }
}
```

```
=====
FILE: home-mixer/filters/self_tweet_filter.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};

/// Filter that removes tweets where the author is the viewer.
pub struct SelfTweetFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for SelfTweetFilter {
    async fn filter(
        &self,
```

```

        query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let viewer_id = query.user_id as u64;
        let (kept, removed): (Vec<_>, Vec<_>) = candidates
            .into_iter()
            .partition(|c| c.author_id != viewer_id);

        Ok(FilterResult { kept, removed })
    }
}

```

```

=====
FILE: home-mixer/filters/vf_filter.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use tonic::async_trait;
use xai_candidate_pipeline::filter::{Filter, FilterResult};
use xai_visibility_filtering::models::{Action, FilteredReason};

pub struct VFFilter;

#[async_trait]
impl Filter<ScoredPostsQuery, PostCandidate> for VFFilter {
    #[xai_stats_macro::receive_stats]
    async fn filter(
        &self,
        _query: &ScoredPostsQuery,
        candidates: Vec<PostCandidate>,
    ) -> Result<FilterResult<PostCandidate>, String> {
        let (removed, kept): (Vec<_>, Vec<_>) = candidates
            .into_iter()
            .partition(|c| should_drop(&c.visibility_reason));

        Ok(FilterResult { kept, removed })
    }
}

fn should_drop(reason: &Option<FilteredReason>) -> bool {
    match reason {
        Some(FilteredReason::SafetyResult(safety_result)) => {
            matches!(safety_result.action, Action::Drop(_))
        }
        Some(_) => true,
        None => false,
    }
}

```

```

=====
FILE: home-mixer/query_hydrators/mod.rs
=====

```

```

pub mod user_action_seq_query_hydrator;
pub mod user_features_query_hydrator;

```

```

=====
FILE: home-mixer/query_hydrators/user_action_seq_query_hydrator.rs
=====

```

```

use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::uas_fetcher::{UserActionSequenceFetcher, UserActionSequenceOps};
use crate::params as p;
use std::sync::Arc;
use std::time::{SystemTime, UNIX_EPOCH};
use tonic::async_trait;
use xai_candidate_pipeline::query_hydrator::QueryHydrator;
use xai_recsys_aggregation::aggregation::{DefaultAggregator, UserActionAggregator};
use xai_recsys_aggregation::filters::{
    AggregatedActionFilter, DenseAggregatedActionFilter, KeepOriginalUserActionFilter,
    UserActionFilter,
};
use xai_recsys_proto::{
    AggregatedUserActionList, Mask, MaskType, UserActionSequence,
    UserActionSequenceDataContainer,
    UserActionSequenceMeta, user_action_sequence_data_container::Data as
    ProtoDataContainer,
};
use xai_uas_thrift::convert::thrift_to_proto_aggregated_user_action;
use xai_uas_thrift::user_action_sequence::{
    AggregatedUserAction as ThriftAggregatedUserAction,
    UserActionSequence as ThriftUserActionSequence,
    UserActionSequenceMeta as ThriftUserActionSequenceMeta,
};

/// Hydrate a sequence that captures the user's recent actions
pub struct UserActionSeqQueryHydrator {
    pub uas_fetcher: Arc<UserActionSequenceFetcher>,
    global_filter: Arc<dyn UserActionFilter>,
    aggregator: Arc<dyn UserActionAggregator>,
    post_filters: Vec<Arc<dyn AggregatedActionFilter>>,
}

impl UserActionSeqQueryHydrator {
    pub fn new(uas_fetcher: Arc<UserActionSequenceFetcher>) -> Self {
        Self {
            uas_fetcher,
            global_filter: Arc::new(KeepOriginalUserActionFilter::new()),
            aggregator: Arc::new(DefaultAggregator),
            post_filters: vec![Arc::new(DenseAggregatedActionFilter::new())],
        }
    }
}

#[async_trait]
impl QueryHydrator<ScoredPostsQuery> for UserActionSeqQueryHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(&self, query: &ScoredPostsQuery) -> Result<ScoredPostsQuery,
String> {
        let uas_thrift = self
            .uas_fetcher
            .get_by_user_id(query.user_id)
            .await
            .map_err(|e| format!("Failed to fetch user action sequence: {}", e))?;

        let aggregated_uas_proto =
            self.aggregate_user_action_sequence(query.user_id, uas_thrift)?;

        Ok(ScoredPostsQuery {
            user_action_sequence: Some(aggregated_uas_proto),
            ..Default::default()
        })
    }
}

```

```

fn update(&self, query: &mut ScoredPostsQuery, hydrated: ScoredPostsQuery) {
    query.user_action_sequence = hydrated.user_action_sequence;
}

fn name(&self) -> &'static str {
    std::any::type_name::<Self>()
}
}

impl UserActionSeqQueryHydrator {
    fn aggregate_user_action_sequence(
        &self,
        user_id: i64,
        uas_thrift: ThriftUserActionSequence,
    ) -> Result<UserActionSequence, String> {
        // Extract user_actions from thrift sequence
        let thrift_user_actions = uas_thrift.user_actions.clone().unwrap_or_default();
        if thrift_user_actions.is_empty() {
            return Err(format!("No user actions found for user {}", user_id));
        }

        // Pre-aggregation filter
        let filtered_actions = self.global_filter.run(thrift_user_actions);
        if filtered_actions.is_empty() {
            return Err(format!(
                "No user actions remaining after filtering for user {}",
                user_id
            ));
        }

        // Aggregate
        let mut aggregated_actions =
            self.aggregator
                .run(&filtered_actions, p::UAS_WINDOW_TIME_MS, 0);

        // Post-aggregation filters
        for filter in &self.post_filters {
            aggregated_actions = filter.run(aggregated_actions);
        }

        // Truncate to max sequence length (keep last N items)
        if aggregated_actions.len() > p::UAS_MAX_SEQUENCE_LENGTH {
            let drain_count = aggregated_actions.len() - p::UAS_MAX_SEQUENCE_LENGTH;
            aggregated_actions.drain(0..drain_count);
        }

        // Convert to proto format
        let original_metadata = uas_thrift.metadata.clone().unwrap_or_default();
        convert_to_proto_sequence(
            user_id,
            original_metadata,
            aggregated_actions,
            self.aggregator.name(),
        )
    }
}

fn convert_to_proto_sequence(
    user_id: i64,
    original_metadata: ThriftUserActionSequenceMeta,
    aggregated_actions: Vec<ThriftAggregatedUserAction>,
    aggregator_name: &str,
) -> Result<UserActionSequence, String> {
    if aggregated_actions.is_empty() {

```

```

    return Err("Cannot create sequence from empty aggregated actions".to_string());
}

let first_sequence_time = aggregated_actions
    .first()
    .and_then(|a| a.impressed_time_ms)
    .unwrap_or(0) as u64;
let last_sequence_time = aggregated_actions
    .last()
    .and_then(|a| a.impressed_time_ms)
    .unwrap_or(0) as u64;

// Preserve lastModifiedEpochMs and lastKafkaPublishEpochMs from original metadata
let last_modified_epoch_ms = original_metadata.last_modified_epoch_ms.unwrap_or(0)
as u64;
let previous_kafka_publish_epoch_ms =
    original_metadata.last_kafka_publish_epoch_ms.unwrap_or(0) as u64;

let proto_metadata = UserActionSequenceMeta {
    length: aggregated_actions.len() as u64,
    first_sequence_time,
    last_sequence_time,
    last_modified_epoch_ms,
    previous_kafka_publish_epoch_ms,
};

// Convert thrift aggregated actions to proto
let mut proto_agg_actions = Vec::with_capacity(aggregated_actions.len());
for action in aggregated_actions {
    proto_agg_actions.push(
        thrift_to_proto_aggregated_user_action(action)
        .map_err(|e| format!("Failed to convert aggregated action: {}", e))?
    );
}

let aggregation_time_ms = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .unwrap()
    .as_millis() as u64;

let agg_list = AggregatedUserActionList {
    aggregated_user_actions: proto_agg_actions,
    aggregation_provider: aggregator_name.to_string(),
    aggregation_time_ms,
};

let mask = Mask {
    mask_type: MaskType::NewEvent as i32,
    mask: vec![false; agg_list.aggregated_user_actions.len()],
};

// Build the final UserActionSequence
Ok(UserActionSequence {
    user_id: user_id as u64,
    metadata: Some(proto_metadata),
    user_actions_data: Some(UserActionSequenceDataContainer {
        data: Some(ProtoDataContainer::OrderedAggregatedUserActionsList(
            agg_list,
        )),
    }),
    masks: vec![mask],
    ..Default::default()
})
}

```

```
=====
FILE: home-mixer/query_hydrators/user_features_query_hydrator.rs
=====
```

```
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::candidate_pipeline::query_features::UserFeatures;
use crate::clients::strato_client::StratoClient;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::query_hydrator::QueryHydrator;
use xai_strato::{StratoResult, StratoValue, decode};

pub struct UserFeaturesQueryHydrator {
    pub strato_client: Arc<dyn StratoClient + Send + Sync>,
}

#[async_trait]
impl QueryHydrator<ScoredPostsQuery> for UserFeaturesQueryHydrator {
    #[xai_stats_macro::receive_stats]
    async fn hydrate(&self, query: &ScoredPostsQuery) -> Result<ScoredPostsQuery,
String> {
        let user_id = query.user_id;
        let client = &self.strato_client;
        let result = client.get_user_features(user_id);
        let result = result.await.map_err(|e| e.to_string())?;
        let decoded: StratoResult<StratoValue<UserFeatures>> = decode(&result);
        match decoded {
            StratoResult::Ok(v) => {
                let user_features = v.v.unwrap_or_default();
                Ok(ScoredPostsQuery {
                    user_features,
                    ..Default::default()
                })
            }
            StratoResult::Err(_) => Err("Error received from strato".to_string()),
        }
    }

    fn update(&self, query: &mut ScoredPostsQuery, hydrated: ScoredPostsQuery) {
        query.user_features = hydrated.user_features;
    }

    fn name(&self) -> &'static str {
        std::any::type_name::<Self>()
    }
}
```

```
=====
FILE: home-mixer/scorers/author_diversity_scorer.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::params as p;
use std::cmp::Ordering;
use std::collections::HashMap;
use tonic::async_trait;
use xai_candidate_pipeline::scorer::Scorer;

/// Diversify authors served within a single feed response
pub struct AuthorDiversityScorer {
```

```

        decay_factor: f64,
        floor: f64,
    }

impl Default for AuthorDiversityScorer {
    fn default() -> Self {
        Self::new(p::AUTHOR_DIVERSITY_DECAY, p::AUTHOR_DIVERSITY_FLOOR)
    }
}

impl AuthorDiversityScorer {
    pub fn new(decay_factor: f64, floor: f64) -> Self {
        Self {
            decay_factor,
            floor,
        }
    }

    fn multiplier(&self, position: usize) -> f64 {
        (1.0 - self.floor) * self.decay_factor.powf(position as f64) + self.floor
    }
}

#[async_trait]
impl Scorer<ScoredPostsQuery, PostCandidate> for AuthorDiversityScorer {
    #[xai_stats_macro::receive_stats]
    async fn score(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let mut author_counts: HashMap<u64, usize> = HashMap::new();
        let mut scored = vec![PostCandidate::default(); candidates.len()];

        let mut ordered: Vec<(usize, &PostCandidate)> =
            candidates.iter().enumerate().collect();
        ordered.sort_by(|(&, a), (&, b)| {
            let a_score = a.weighted_score.unwrap_or(f64::NEG_INFINITY);
            let b_score = b.weighted_score.unwrap_or(f64::NEG_INFINITY);
            b_score.partial_cmp(&a_score).unwrap_or(Ordering::Equal)
        });

        for (original_idx, candidate) in ordered {
            let entry = author_counts.entry(candidate.author_id).or_insert(0);
            let position = *entry;
            *entry += 1;

            let multiplier = self.multiplier(position);
            let adjusted_score = candidate.weighted_score.map(|score| score *
multiplier);

            let updated = PostCandidate {
                score: adjusted_score,
                ..Default::default()
            };
            scored[original_idx] = updated;
        }

        Ok(scored)
    }

    fn update(&self, candidate: &mut PostCandidate, scored: PostCandidate) {
        candidate.score = scored.score;
    }
}

```

```
}
```

```
=====
FILE: home-mixer/scorers/mod.rs
=====
```

```
pub mod author_diversity_scorer;
pub mod oon_scorer;
pub mod phoenix_scorer;
pub mod weighted_scorer;
```

```
=====
FILE: home-mixer/scorers/oon_scorer.rs
=====
```

```
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::params as p;
use tonic::async_trait;
use xai_candidate_pipeline::scorer::Scorer;

// Prioritize in-network candidates over out-of-network candidates
pub struct OONScorer;

#[async_trait]
impl Scorer<ScoredPostsQuery, PostCandidate> for OONScorer {
    async fn score(
        &self,
        _query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let scored = candidates
            .iter()
            .map(|c| {
                let updated_score = c.score.map(|base_score| match c.in_network {
                    Some(false) => base_score * p::OON_WEIGHT_FACTOR,
                    _ => base_score,
                });
                PostCandidate {
                    score: updated_score,
                    ..Default::default()
                }
            })
            .collect();

        Ok(scored)
    }

    fn update(&self, candidate: &mut PostCandidate, scored: PostCandidate) {
        candidate.score = scored.score;
    }
}
```

```
=====
FILE: home-mixer/scorers/phoenix_scorer.rs
=====
```

```
use crate::candidate_pipeline::candidate::{PhoenixScores, PostCandidate};
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::phoenix_prediction_client::PhoenixPredictionClient;
use crate::util::request_util;
```

```

use std::collections::HashMap;
use std::sync::Arc;
use std::time::{SystemTime, UNIX_EPOCH};
use tonic::async_trait;
use xai_candidate_pipeline::scorer::Scorer;
use xai_recsys_proto::{ActionName, ContinuousActionName};

pub struct PhoenixScorer {
    pub phoenix_client: Arc<dyn PhoenixPredictionClient + Send + Sync>,
}

#[async_trait]
impl Scorer<ScoredPostsQuery, PostCandidate> for PhoenixScorer {
    #[xai_stats_macro::receive_stats]
    async fn score(
        &self,
        query: &ScoredPostsQuery,
        candidates: &[PostCandidate],
    ) -> Result<Vec<PostCandidate>, String> {
        let user_id = query.user_id as u64;
        let prediction_request_id = request_util::generate_request_id();
        let last_scored_at_ms = Self::current_timestamp_millis();

        if let Some(sequence) = &query.user_action_sequence {
            let tweet_infos: Vec<xai_recsys_proto::TweetInfo> = candidates
                .iter()
                .map(|c| {
                    let tweet_id = c.retweeted_tweet_id.unwrap_or(c.tweet_id as u64);
                    let author_id = c.retweeted_user_id.unwrap_or(c.author_id);
                    xai_recsys_proto::TweetInfo {
                        tweet_id,
                        author_id,
                        ..Default::default()
                    }
                })
                .collect();

            let result = self
                .phoenix_client
                .predict(user_id, sequence.clone(), tweet_infos)
                .await;

            if let Ok(response) = result {
                let predictions_map = self.build_predictions_map(&response);

                let scored_candidates = candidates
                    .iter()
                    .map(|c| {
                        // For retweets, look up predictions using the original tweet
                        let lookup_tweet_id = c.retweeted_tweet_id.unwrap_or(c.tweet_id
as u64);

                        let phoenix_scores = predictions_map
                            .get(&lookup_tweet_id)
                            .map(|preds| self.extract_phoenix_scores(preds))
                            .unwrap_or_default();

                        PostCandidate {
                            phoenix_scores,
                            prediction_request_id: Some(prediction_request_id),
                            last_scored_at_ms,
                            ..Default::default()
                        }
                    })
                .collect();
            }
        }
    }
}

```

```

        })
        .collect();

        return Ok(scored_candidates);
    }
}

// Return candidates unchanged if no scoring could be done
Ok(candidates.to_vec())
}

fn update(&self, candidate: &mut PostCandidate, scored: PostCandidate) {
    candidate.phoenix_scores = scored.phoenix_scores;
    candidate.prediction_request_id = scored.prediction_request_id;
    candidate.last_scored_at_ms = scored.last_scored_at_ms;
}

}

impl PhoenixScorer {
    /// Builds Map[tweet_id -> ActionPredictions]
    fn build_predictions_map(
        &self,
        response: &xai_recsys_proto::PredictNextActionsResponse,
    ) -> HashMap<u64, ActionPredictions> {
        let mut predictions_map = HashMap::new();

        let Some(distribution_set) = response.distribution_sets.first() else {
            return predictions_map;
        };

        for distribution in &distribution_set.candidate_distributions {
            let Some(candidate) = &distribution.candidate else {
                continue;
            };
            let tweet_id = candidate.tweet_id;

            let action_probs: HashMap<usize, f64> = distribution
                .top_log_probs
                .iter()
                .enumerate()
                .map(|(idx, log_prob)| (idx, (*log_prob as f64).exp()))
                .collect();

            let continuous_values: HashMap<usize, f64> = distribution
                .continuous_actions_values
                .iter()
                .enumerate()
                .map(|(idx, value)| (idx, *value as f64))
                .collect();

            predictions_map.insert(
                tweet_id,
                ActionPredictions {
                    action_probs,
                    continuous_values,
                },
            );
        }

        predictions_map
    }

    fn extract_phoenix_scores(&self, p: &ActionPredictions) -> PhoenixScores {
        PhoenixScores {

```

```

        favorite_score: p.get(ActionName::ServerTweetFav),
        reply_score: p.get(ActionName::ServerTweetReply),
        retweet_score: p.get(ActionName::ServerTweetRetweet),
        photo_expand_score: p.get(ActionName::ClientTweetPhotoExpand),
        click_score: p.get(ActionName::ClientTweetClick),
        profile_click_score: p.get(ActionName::ClientTweetClickProfile),
        vqv_score: p.get(ActionName::ClientTweetVideoQualityView),
        share_score: p.get(ActionName::ClientTweetShare),
        share_via_dm_score:
    p.get(ActionName::ClientTweetClickSendViaDirectMessage),
        share_via_copy_link_score: p.get(ActionName::ClientTweetShareViaCopyLink),
        dwell_score: p.get(ActionName::ClientTweetRecapDwelled),
        quote_score: p.get(ActionName::ServerTweetQuote),
        quoted_click_score: p.get(ActionName::ClientQuotedTweetClick),
        follow_author_score: p.get(ActionName::ClientTweetFollowAuthor),
        not_interested_score: p.get(ActionName::ClientTweetNotInterestedIn),
        block_author_score: p.get(ActionName::ClientTweetBlockAuthor),
        mute_author_score: p.get(ActionName::ClientTweetMuteAuthor),
        report_score: p.get(ActionName::ClientTweetReport),
        dwell_time: p.get_continuous(ContinuousActionName::DwellTime),
    }
}

fn current_timestamp_millis() -> Option<u64> {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .ok()
        .map(|duration| duration.as_millis() as u64)
}

struct ActionPredictions {
    /// Map of action index -> probability (exp of log prob)
    action_probs: HashMap<usize, f64>,
    /// Map of continuous action index -> value
    continuous_values: HashMap<usize, f64>,
}

impl ActionPredictions {
    fn get(&self, action: ActionName) -> Option<f64> {
        self.action_probs.get(&(action as usize)).copied()
    }

    fn get_continuous(&self, action: ContinuousActionName) -> Option<f64> {
        self.continuous_values.get(&(action as usize)).copied()
    }
}

```

```

=====
FILE: home-mixer/scorers/weighted_scorer.rs
=====

```

```

use crate::candidate_pipeline::candidate::{PhoenixScores, PostCandidate};
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::params as p;
use crate::util::score_normalizer::normalize_score;
use tonic::async_trait;
use xai_candidate_pipeline::scorer::Scorer;

```

```

pub struct WeightedScorer;

```

```

#[async_trait]
impl Scorer<ScoredPostsQuery, PostCandidate> for WeightedScorer {

```

```

#[xai_stats_macro::receive_stats]
async fn score(
    &self,
    _query: &ScoredPostsQuery,
    candidates: &[PostCandidate],
) -> Result<Vec<PostCandidate>, String> {
    let scored = candidates
        .iter()
        .map(|c| {
            let weighted_score = Self::compute_weighted_score(c);
            let normalized_weighted_score = normalize_score(c, weighted_score);

            PostCandidate {
                weighted_score: Some(normalized_weighted_score),
                ..Default::default()
            }
        })
        .collect();

    Ok(scored)
}

fn update(&self, candidate: &mut PostCandidate, scored: PostCandidate) {
    candidate.weighted_score = scored.weighted_score;
}

impl WeightedScorer {
    fn apply(score: Option<f64>, weight: f64) -> f64 {
        score.unwrap_or(0.0) * weight
    }

    fn compute_weighted_score(candidate: &PostCandidate) -> f64 {
        let s: &PhoenixScores = &candidate.phoenix_scores;

        let vqv_weight = Self::vqv_weight_eligibility(candidate);

        let combined_score = Self::apply(s.favorite_score, p::FAVORITE_WEIGHT)
            + Self::apply(s.reply_score, p::REPLY_WEIGHT)
            + Self::apply(s.retweet_score, p::RETWEET_WEIGHT)
            + Self::apply(s.photo_expand_score, p::PHOTO_EXPAND_WEIGHT)
            + Self::apply(s.click_score, p::CLICK_WEIGHT)
            + Self::apply(s.profile_click_score, p::PROFILE_CLICK_WEIGHT)
            + Self::apply(s.vqv_score, vqv_weight)
            + Self::apply(s.share_score, p::SHARE_WEIGHT)
            + Self::apply(s.share_via_dm_score, p::SHARE_VIA_DM_WEIGHT)
            + Self::apply(s.share_via_copy_link_score, p::SHARE_VIA_COPY_LINK_WEIGHT)
            + Self::apply(s.dwell_score, p::DWELL_WEIGHT)
            + Self::apply(s.quote_score, p::QUOTE_WEIGHT)
            + Self::apply(s.quoted_click_score, p::QUOTED_CLICK_WEIGHT)
            + Self::apply(s.dwell_time, p::CONT_DWELL_TIME_WEIGHT)
            + Self::apply(s.follow_author_score, p::FOLLOW_AUTHOR_WEIGHT)
            + Self::apply(s.not_interested_score, p::NOT_INTERESTED_WEIGHT)
            + Self::apply(s.block_author_score, p::BLOCK_AUTHOR_WEIGHT)
            + Self::apply(s.mute_author_score, p::MUTE_AUTHOR_WEIGHT)
            + Self::apply(s.report_score, p::REPORT_WEIGHT);

        Self::offset_score(combined_score)
    }

    fn vqv_weight_eligibility(candidate: &PostCandidate) -> f64 {
        if candidate
            .video_duration_ms
            .is_some_and(|ms| ms > p::MIN_VIDEO_DURATION_MS)

```

```

        {
            p::VQV_WEIGHT
        } else {
            0.0
        }
    }

    fn offset_score(combined_score: f64) -> f64 {
        if p::WEIGHTS_SUM == 0.0 {
            combined_score.max(0.0)
        } else if combined_score < 0.0 {
            (combined_score + p::NEGATIVE_WEIGHTS_SUM) / p::WEIGHTS_SUM *
p::NEGATIVE_SCORES_OFFSET
        } else {
            combined_score + p::NEGATIVE_SCORES_OFFSET
        }
    }
}

```

```

=====
FILE: home-mixer/selectors/mod.rs
=====

```

```

mod top_k_score_selector;

pub use top_k_score_selector::TopKScoreSelector;

```

```

=====
FILE: home-mixer/selectors/top_k_score_selector.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::params;
use xai_candidate_pipeline::selector::Selector;

pub struct TopKScoreSelector;

impl Selector<ScoredPostsQuery, PostCandidate> for TopKScoreSelector {
    fn score(&self, candidate: &PostCandidate) -> f64 {
        candidate.score.unwrap_or(f64::NEG_INFINITY)
    }
    fn size(&self) -> Option<usize> {
        Some(params::TOP_K_CANDIDATES_TO_SELECT)
    }
}

```

```

=====
FILE: home-mixer/side_effects/cache_request_info_side_effect.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::strato_client::StratoClient;
use std::env;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::side_effect::{SideEffect, SideEffectInput};
use xai_strato::{StratoResult, StratoValue, decode};

pub struct CacheRequestInfoSideEffect {

```

```

    pub strato_client: Arc<dyn StratoClient + Send + Sync>,
}

#[async_trait]
impl SideEffect<ScoredPostsQuery, PostCandidate> for CacheRequestInfoSideEffect {
    fn enable(&self, query: Arc<ScoredPostsQuery>) -> bool {
        env::var("APP_ENV").unwrap_or_default() == "prod" && !query.in_network_only
    }

    async fn run(
        &self,
        input: Arc<SideEffectInput<ScoredPostsQuery, PostCandidate>>,
    ) -> Result<(), String> {
        let user_id: i64 = input.query.user_id;

        let post_ids: Vec<i64> = input
            .selected_candidates
            .iter()
            .map(|c| c.tweet_id)
            .collect();
        let client = &self.strato_client;
        let res = client
            .store_request_info(user_id, post_ids)
            .await
            .map_err(|e| e.to_string())?;
        let decoded: StratoResult<StratoValue<()>> = decode(&res);
        match decoded {
            StratoResult::Ok(_) => Ok(()),
            StratoResult::Err(_) => Err("error received from strato".to_string()),
        }
    }
}

```

```

=====
FILE: home-mixer/side_effects/mod.rs
=====

```

```

pub mod cache_request_info_side_effect;

```

```

=====
FILE: home-mixer/sources/mod.rs
=====

```

```

pub mod phoenix_source;
pub mod thunder_source;

```

```

=====
FILE: home-mixer/sources/phoenix_source.rs
=====

```

```

use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::phoenix_retrieval_client::PhoenixRetrievalClient;
use crate::params as p;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::source::Source;
use xai_home_mixer_proto as pb;

pub struct PhoenixSource {
    pub phoenix_retrieval_client: Arc<dyn PhoenixRetrievalClient + Send + Sync>,
}

```

```

}

#[async_trait]
impl Source<ScoredPostsQuery, PostCandidate> for PhoenixSource {
    fn enable(&self, query: &ScoredPostsQuery) -> bool {
        !query.in_network_only
    }

    #[xai_stats_macro::receive_stats]
    async fn get_candidates(&self, query: &ScoredPostsQuery) ->
    Result<Vec<PostCandidate>, String> {
        let user_id = query.user_id as u64;

        let sequence = query
            .user_action_sequence
            .as_ref()
            .ok_or_else(|| "PhoenixSource: missing user_action_sequence".to_string())?;

        let response = self
            .phoenix_retrieval_client
            .retrieve(user_id, sequence.clone(), p::PHOENIX_MAX_RESULTS)
            .await
            .map_err(|e| format!("PhoenixSource: {}", e))?;

        let candidates: Vec<PostCandidate> = response
            .top_k_candidates
            .into_iter()
            .flat_map(|scored_candidates| scored_candidates.candidates)
            .filter_map(|scored_candidate| scored_candidate.candidate)
            .map(|tweet_info| PostCandidate {
                tweet_id: tweet_info.tweet_id as i64,
                author_id: tweet_info.author_id,
                in_reply_to_tweet_id: Some(tweet_info.in_reply_to_tweet_id),
                served_type: Some(pb::ServedType::ForYouPhoenixRetrieval),
                ..Default::default()
            })
            .collect();

        Ok(candidates)
    }
}

```

```

=====
FILE: home-mixer/sources/thunder_source.rs
=====
use crate::candidate_pipeline::candidate::PostCandidate;
use crate::candidate_pipeline::query::ScoredPostsQuery;
use crate::clients::thunder_client::{ThunderClient, ThunderCluster};
use crate::params as p;
use std::sync::Arc;
use tonic::async_trait;
use xai_candidate_pipeline::source::Source;
use xai_home_mixer_proto as pb;
use xai_thunder_proto::GetInNetworkPostsRequest;
use xai_thunder_proto::in_network_posts_service_client::InNetworkPostsServiceClient;

pub struct ThunderSource {
    pub thunder_client: Arc<ThunderClient>,
}

#[async_trait]
impl Source<ScoredPostsQuery, PostCandidate> for ThunderSource {

```

```

#[xai_stats_macro::receive_stats]
async fn get_candidates(&self, query: &ScoredPostsQuery) ->
Result<Vec<PostCandidate>, String> {
    let cluster = ThunderCluster::Amp;
    let channel = self
        .thunder_client
        .get_random_channel(cluster)
        .ok_or_else(|| "ThunderSource: no available channel".to_string())?;

    let mut client = InNetworkPostsServiceClient::new(channel.clone());
    let following_list = &query.user_features.followed_user_ids;
    let request = GetInNetworkPostsRequest {
        user_id: query.user_id as u64,
        following_user_ids: following_list.iter().map(|&id| id as u64).collect(),
        max_results: p::THUNDER_MAX_RESULTS,
        exclude_tweet_ids: vec![],
        algorithm: "default".to_string(),
        debug: false,
        is_video_request: false,
    };

    let response = client
        .get_in_network_posts(request)
        .await
        .map_err(|e| format!("ThunderSource: {}", e))?;

    let candidates: Vec<PostCandidate> = response
        .into_inner()
        .posts
        .into_iter()
        .map(|post| {
            let in_reply_to_tweet_id = post
                .in_reply_to_post_id
                .and_then(|id| u64::try_from(id).ok());
            let conversation_id = post.conversation_id.and_then(|id|
u64::try_from(id).ok());

            let mut ancestors = Vec::new();
            if let Some(reply_to) = in_reply_to_tweet_id {
                ancestors.push(reply_to);
                if let Some(root) = conversation_id.filter(|&root| root !=
reply_to) {
                    ancestors.push(root);
                }
            }

            PostCandidate {
                tweet_id: post.post_id,
                author_id: post.author_id as u64,
                in_reply_to_tweet_id,
                ancestors,
                served_type: Some(pb::ServedType::ForYouInNetwork),
                ..Default::default()
            }
        })
        .collect();

    Ok(candidates)
}

```

=====

FILE: phoenix/README.md
=====

Phoenix: Recommendation System

This repository contains JAX example code for the Phoenix recommendation system, which powers content ranking and retrieval. Phoenix uses transformer-based architectures for both **retrieval** (finding relevant candidates from millions of items) and **ranking** (ordering a smaller set of candidates by predicted engagement).

> **Note:** The sample transformer implementation in this repository is ported from the [Grok-1 open source release] (<https://github.com/xai-org/grok-1>) by xAI. The core transformer architecture comes from Grok-1, adapted here for recommendation system use cases with custom input embeddings and attention masking for candidate isolation. This code is representative of the model used internally with the exception of specific scaling optimizations.

Table of Contents

- [Overview] (#overview)
- [Architecture] (#architecture)
 - [Two-Stage Recommendation Pipeline] (#two-stage-recommendation-pipeline)
 - [Retrieval: Two-Tower Model] (#retrieval-two-tower-model)
 - [Ranking: Transformer with Candidate Isolation] (#ranking-transformer-with-candidate-isolation)
- [Key Design Decisions] (#key-design-decisions)
- [Running the Code] (#running-the-code)
- [License] (#license)

Overview

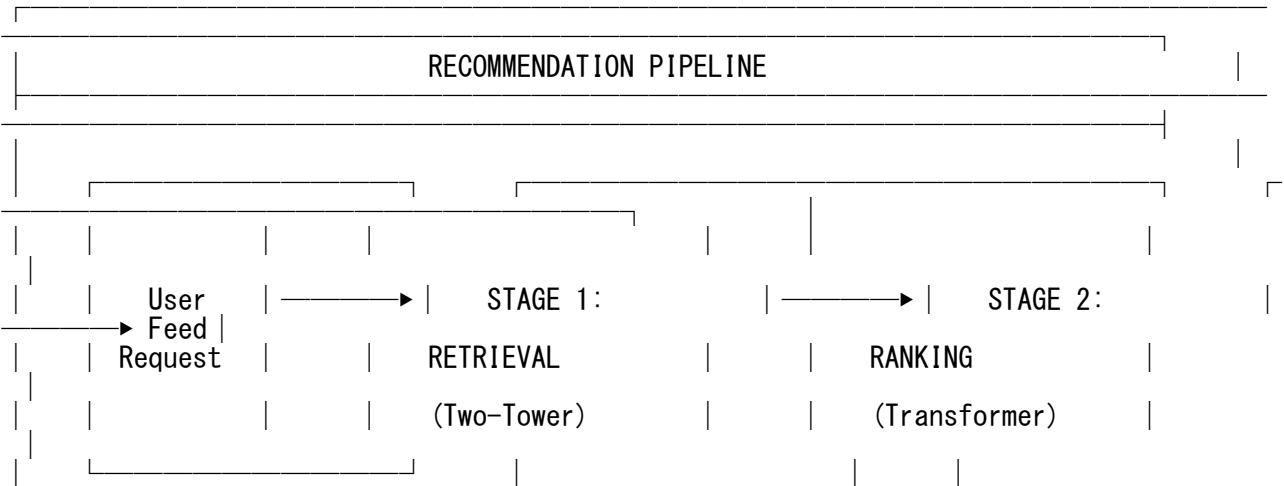
Phoenix is a recommendation system that predicts user engagement (likes, reposts, replies, etc.) for content. It operates in two stages:

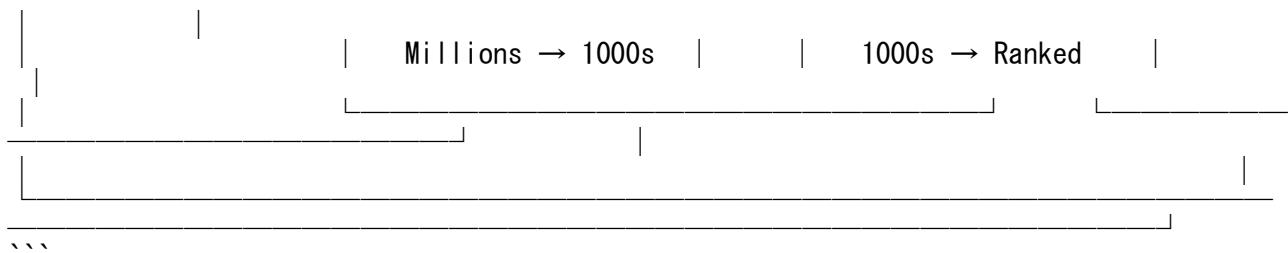
1. **Retrieval**: Efficiently narrow down millions of candidates to hundreds using approximate nearest neighbor (ANN) search
2. **Ranking**: Score and order the retrieved candidates using a more expressive transformer model

Architecture

Two-Stage Recommendation Pipeline

...





Retrieval: Two-Tower Model

The retrieval stage uses a **two-tower architecture** that enables efficient similarity search at scale.

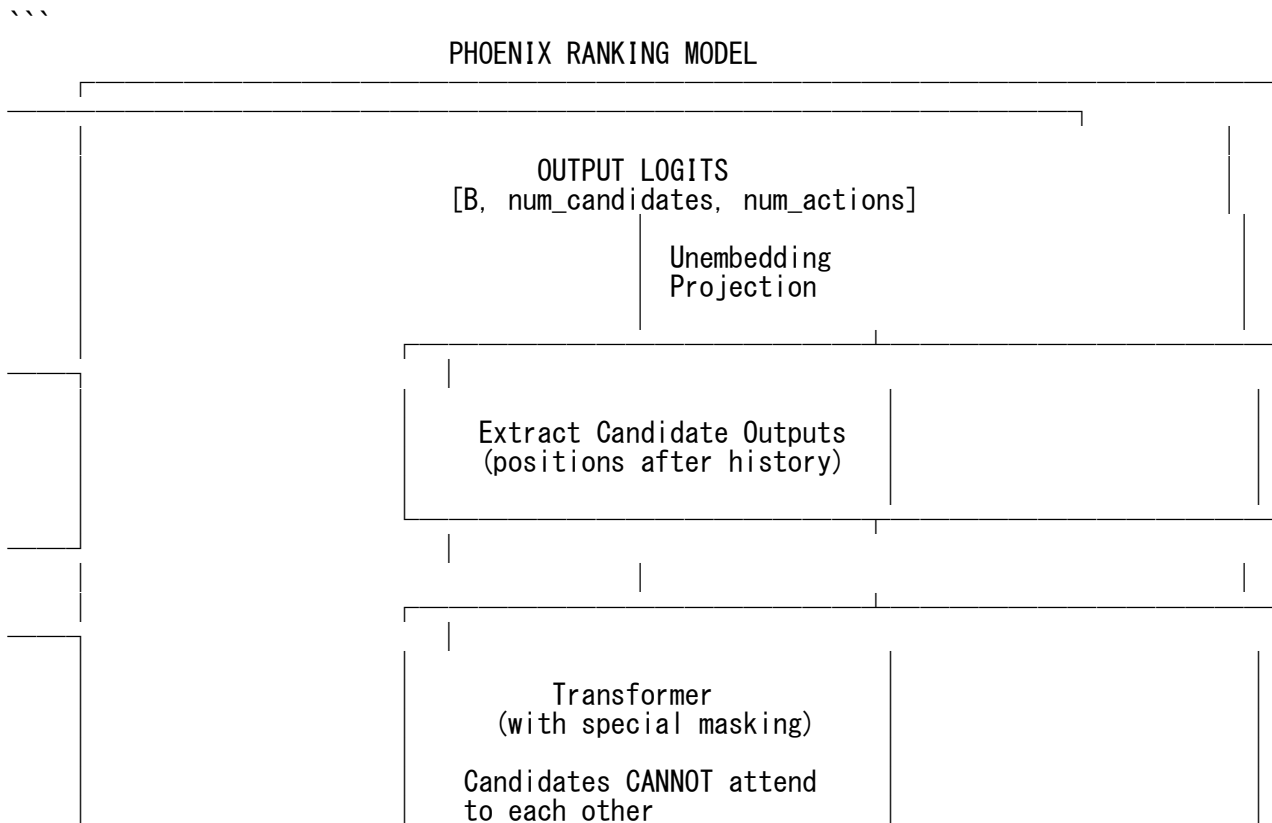
How Retrieval Works

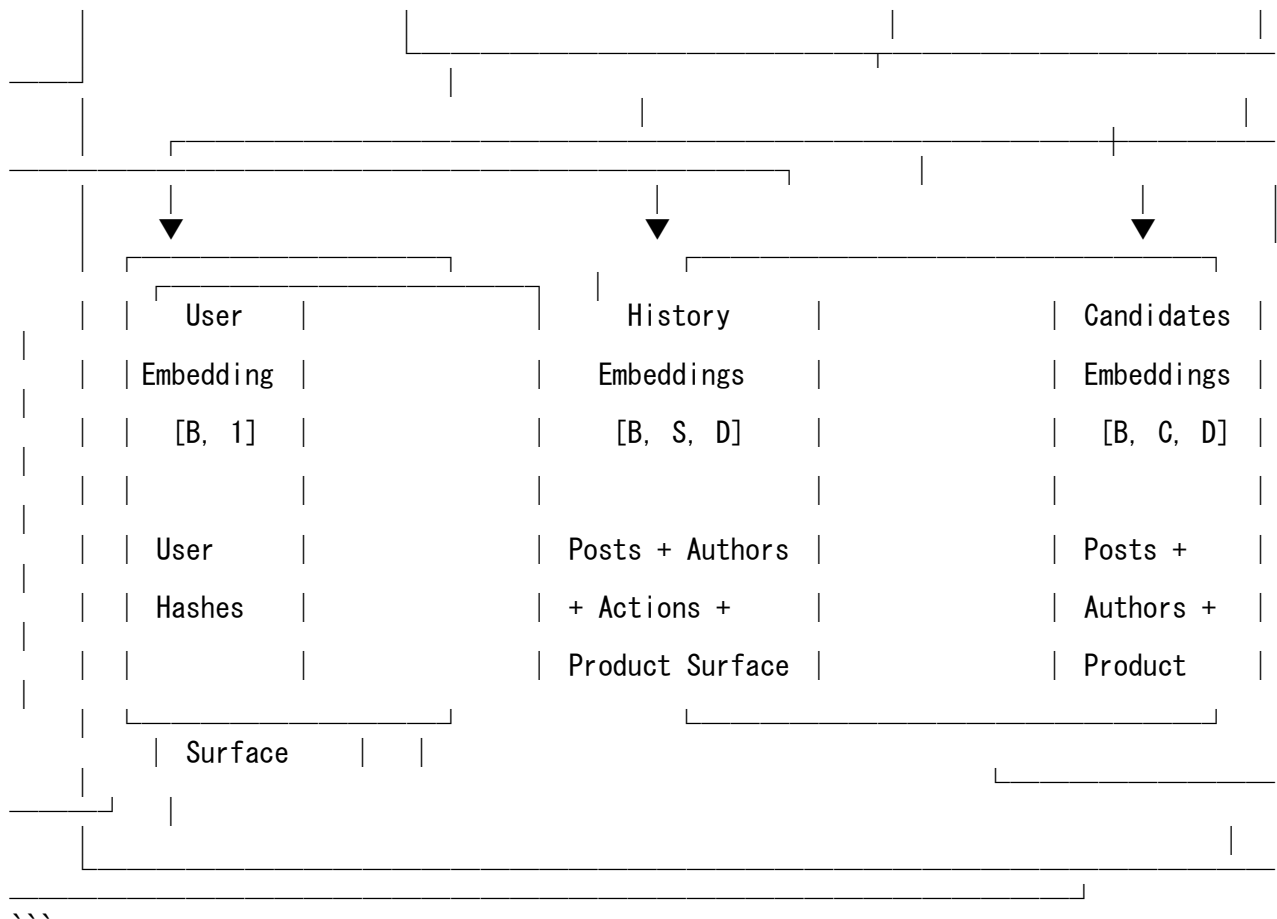
- User Tower**: Encodes user features and engagement history through a transformer to produce a normalized user embedding $[B, D]$
- Candidate Tower**: Computes normalized embeddings for all items in the corpus $[N, D]$
- Similarity Search**: Retrieves top-K candidates using dot product similarity

Ranking: Transformer with Candidate Isolation

The ranking model uses a transformer architecture where **candidates cannot attend to each other** during inference. This is a critical design choice that ensures the score for a candidate doesn't depend on which other candidates are in the batch

Ranking Model Architecture



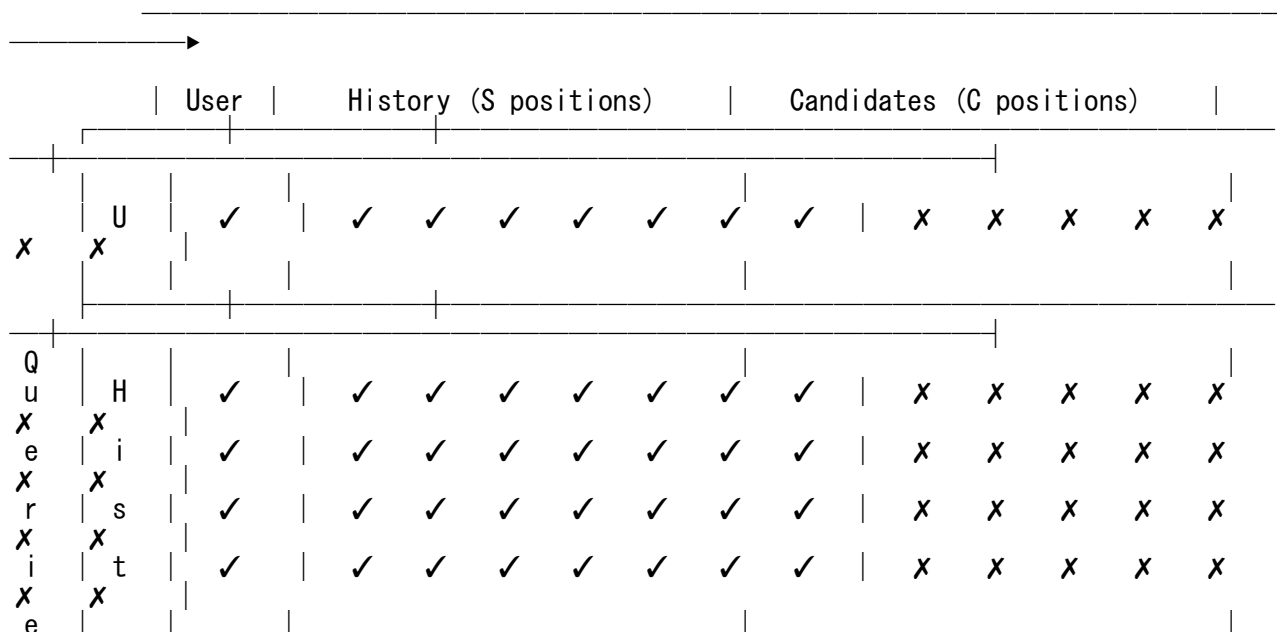


Attention Mask: Candidate Isolation

A key detail is the **attention mask** that prevents candidates from attending to each other while still allowing them to attend to the user and history:

ATTENTION MASK VISUALIZATION

Keys (what we attend T0)



s															
											DIAGONAL ONLY	(self-attend)			
X	X	C	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X
X	X	a	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	✓	X	X
X	X	n	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	✓	X
X	X	d	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X	✓
X	X	i	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X	✓
X	X	d	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X	X
✓	✓	s	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X	X
X	▼														

✓ = Can attend (1)

X = Cannot attend (0)

Legend:

— User + History: Full bidirectional attention among themselves

— Candidates → User/History: Candidates CAN attend to user and history

— Candidates → Candidates: Candidates CANNOT attend to each other (only self)

...

Key Design Decisions

1. Hash-Based Embeddings

Both models use multiple hash functions for embedding lookup

2. Shared Architecture

The retrieval user tower uses the same transformer architecture as the ranking model

3. Multi-Action Prediction

The ranking model predicts multiple engagement types simultaneously:

...

Output: [B, num_candidates, num_actions]



Like	Repost	Reply	Click	...
------	--------	-------	-------	-----

...

Running the Code

Installation

Install [uv] (<https://docs.astral.sh/uv/getting-started/installation/>)

Running the Ranker

```
```shell
uv run run_ranker.py
```

### Running Retrieval

```
```shell
uv run run_retrieval.py
```

Running Tests

```
```shell
uv run pytest test_recsys_model.py test_recsys_retrieval_model.py
```

```
=====
FILE: phoenix/grok.py
=====
```

```
Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```

```
import logging
from dataclasses import dataclass
from typing import NamedTuple, Optional, Sequence, Union
```

```
import haiku as hk
import jax
import jax.numpy as jnp
```

```
logger = logging.getLogger(__name__)
```

```
class TrainingState(NamedTuple):
 """Container for the training state."""
```

```
 params: hk.Params
```

```
def ffn_size(emb_size, widening_factor):
 _ffn_size = int(widening_factor * emb_size) * 2 // 3
 _ffn_size = _ffn_size + (8 - _ffn_size) % 8 # ensure it's a multiple of 8
 logger.debug(f"emb_size: {emb_size} adjusted ffn_size: {_ffn_size}")
 return _ffn_size
```

```
def make_recsys_attn_mask(
 seq_len: int,
 candidate_start_offset: int,
 dtype: jnp.dtype = jnp.float32,
) -> jax.Array:
```

```
"""Create attention mask for recommendation system inference.
```

```
Creates a mask where:
```

- Positions 0 to candidate\_start\_offset-1 (user+history): causal attention
- Positions candidate\_start\_offset onwards (candidates): can attend to user+history and themselves (self-attention), but NOT to other candidates

```
This ensures each candidate is scored independently based on user+history context.
```

```
Args:
```

```
seq_len: Total sequence length (user + history + candidates)
candidate_start_offset: Position where candidates start in the sequence
dtype: Data type for the mask
```

```
Returns:
```

```
""" Attention mask of shape [1, 1, seq_len, seq_len] where 1 means "can attend"
```

```
Start with causal mask for the full sequence
```

```
causal_mask = jnp.tril(jnp.ones((1, 1, seq_len, seq_len), dtype=dtype))
```

```
Zero out candidate-to-candidate attention (bottom-right block)
```

```
attn_mask = causal_mask.at[:, :, candidate_start_offset:,
candidate_start_offset:].set(0)
```

```
Add back self-attention for candidates (diagonal of the candidate block)
```

```
candidate_indices = jnp.arange(candidate_start_offset, seq_len)
```

```
attn_mask = attn_mask.at[:, :, candidate_indices, candidate_indices].set(1)
```

```
return attn_mask
```

```
class MHAOutput(NamedTuple):
```

```
"""Outputs of the multi-head attention operation."""
```

```
embeddings: jax.Array
```

```
class DecoderOutput(NamedTuple):
```

```
embeddings: jax.Array
```

```
class TransformerOutput(NamedTuple):
```

```
embeddings: jax.Array
```

```
@dataclass
```

```
class TransformerConfig:
```

```
emb_size: int
```

```
key_size: int
```

```
num_q_heads: int
```

```
num_kv_heads: int
```

```
num_layers: int
```

```
widening_factor: float = 4.0
```

```
attn_output_multiplier: float = 1.0
```

```
name: Optional[str] = None
```

```
def make(self) -> "Transformer":
```

```
 return Transformer(
```

```
 num_q_heads=self.num_q_heads,
```

```
 num_kv_heads=self.num_kv_heads,
```

```
 widening_factor=self.widening_factor,
```

```
 key_size=self.key_size,
```

```

 attn_output_multiplier=self.attn_output_multiplier,
 num_layers=self.num_layers,
)

def hk_rms_norm(
 x: jax.Array,
 fixed_scale=False,
) -> jax.Array:
 """Applies a unique LayerNorm to x with default settings."""
 ln = RMSNorm(axis=-1, create_scale=not fixed_scale)
 return ln(x)

class Linear(hk.Linear):
 def __init__(
 self,
 output_size: int,
 with_bias: bool = True,
 name: Optional[str] = None,
):
 super().__init__(
 output_size=output_size,
 with_bias=with_bias,
 name=name,
)

 def __call__(# type: ignore
 self,
 inputs: jax.Array,
) -> jax.Array:
 """Computes a linear transform of the input."""

 fprop_dtype = inputs.dtype
 if not inputs.shape:
 raise ValueError("Input must not be scalar.")

 input_size = inputs.shape[-1]
 output_size = self.output_size

 w = hk.get_parameter(
 "w", [input_size, output_size], jnp.float32,
 init=hk.initializers.Constant(0)
)

 out = jnp.dot(inputs, w.astype(fprop_dtype))
 if self.with_bias:
 b = hk.get_parameter(
 "b", [self.output_size], jnp.float32, init=hk.initializers.Constant(0)
)
 b = jnp.broadcast_to(b, out.shape)
 out = out + b.astype(fprop_dtype)

 return out

class RMSNorm(hk.RMSNorm):
 def __init__(
 self,
 axis: Union[int, Sequence[int], slice],
 eps: float = 1e-5,
 name: Optional[str] = None,
 create_scale: bool = True,
):

```

```

 super().__init__(axis, eps, create_scale=create_scale, name=name)

def __call__(self, inputs: jax.Array):
 fprop_dtype = inputs.dtype
 param_shape = (inputs.shape[-1],)
 if self.create_scale:
 scale = hk.get_parameter(
 "scale",
 param_shape,
 dtype=jnp.float32,
 init=hk.initializers.Constant(0),
)
 scale = jnp.broadcast_to(scale.astype(jnp.float32), inputs.shape)
 else:
 scale = 1.0
 inputs = inputs.astype(jnp.float32)
 scale = jnp.float32(scale)
 mean_squared = jnp.mean(jnp.square(inputs), axis=-1, keepdims=True)
 mean_squared = jnp.broadcast_to(mean_squared, inputs.shape)

 normed_inputs = inputs * jax.lax.rsqrt(mean_squared + self.eps)

 outputs = scale * normed_inputs

 return outputs.astype(fprop_dtype)

def rotate_half(
 x: jax.Array,
) -> jax.Array:
 """Obtain the rotated counterpart of each feature"""
 x1, x2 = jnp.split(x, 2, axis=-1)
 return jnp.concatenate((-x2, x1), axis=-1)

class RotaryEmbedding(hk.Module):
 """Applies rotary embeddings (RoPE) to the input sequence tensor,
 as described in https://arxiv.org/abs/2104.09864.

 Attributes:
 dim (int): Dimensionality of the feature vectors
 base_exponent (int): Base exponent to compute embeddings from
 """

 def __init__(
 self,
 dim: int,
 name: Optional[str] = None,
 base_exponent: int = 10000,
):
 super().__init__(name)
 self.dim = dim
 self.base_exponent = base_exponent
 assert self.dim % 2 == 0

 def __call__(
 self,
 x: jax.Array,
 seq_dim: int,
 offset: jax.Array,
 const_position: Optional[int] = None,
 t: Optional[jax.Array] = None,
) -> jax.Array:
 fprop_dtype = x.dtype

```

```

Compute the per-dimension frequencies
exponents = jnp.arange(0, self.dim, 2, dtype=jnp.float32)
inv_freq = jnp.asarray(
 1.0 / (self.base_exponent ** (exponents / self.dim)), dtype=jnp.float32
)

if jnp.shape(offset) == ():
 # Offset can be a scalar or one offset per batch element.
 offset = jnp.expand_dims(offset, 0)

Compute the per element phase (to pass into sin and cos)
if const_position:
 t = const_position * jnp.ones(
 (
 1,
 x.shape[seq_dim],
),
 dtype=jnp.float32,
)
elif t is None:
 t = jnp.arange(x.shape[seq_dim], dtype=jnp.float32) +
jnp.expand_dims(offset, -1)
phase = jnp.einsum("bi,j->bij", t, inv_freq)
phase = jnp.tile(phase, reps=(1, 2))[:, :, None, :]

x = x * jnp.cos(phase) + rotate_half(x) * jnp.sin(phase)
x = x.astype(fprop_dtype)

return x

```

```

class MultiHeadAttention(hk.Module):
 def __init__(
 self,
 num_q_heads: int,
 num_kv_heads: int,
 key_size: int,
 *,
 with_bias: bool = True,
 value_size: Optional[int] = None,
 model_size: Optional[int] = None,
 attn_output_multiplier: float = 1.0,
 name: Optional[str] = None,
):
 super().__init__(name=name)
 self.num_q_heads = num_q_heads
 self.num_kv_heads = num_kv_heads
 self.key_size = key_size
 self.value_size = value_size or key_size
 self.model_size = model_size or key_size * num_q_heads
 self.attn_output_multiplier = attn_output_multiplier
 self.with_bias = with_bias

 def __call__(
 self,
 query: jax.Array,
 key: jax.Array,
 value: jax.Array,
 mask: jax.Array,
) -> MHAOutput:
 # In shape hints below, we suppress the leading dims [...] for brevity.
 # Hence e.g. [A, B] should be read in every case as [..., A, B].
 projection = self._linear_projection

```

```

Check that the keys and values have consistent batch size and sequence
length.
assert key.shape[:2] == value.shape[:2], f"key/value shape:
{key.shape}/{value.shape}"

if mask is not None:
 assert mask.ndim == 4
 assert mask.shape[0] in {
 1,
 query.shape[0],
 }, f"mask/query shape: {mask.shape}/{query.shape}"
 assert key.shape[0] in {
 1,
 query.shape[0],
 }, f"key/query shape: {key.shape}/{query.shape}"
 assert mask.shape[1] == 1
 assert mask.shape[2] in {
 1,
 query.shape[1],
 }, f"mask/query shape: {mask.shape}/{query.shape}"
 assert mask.shape[3] in {
 1,
 key.shape[1],
 }, f"mask/query shape: {mask.shape}/{key.shape}"

Compute key/query/values (overload K/Q/V to denote the respective sizes).
assert self.num_q_heads % self.num_kv_heads == 0
query_heads = projection(query, self.key_size, self.num_q_heads, name="query")
key_heads = projection(key, self.key_size, self.num_kv_heads, name="key")
value_heads = projection(value, self.value_size, self.num_kv_heads,
name="value")

rotate = RotaryEmbedding(dim=self.key_size, base_exponent=int(1e4))
key_heads = rotate(key_heads, seq_dim=1, offset=0)
query_heads = rotate(query_heads, seq_dim=1, offset=0)

b, t, h, d = query_heads.shape
_, _, kv_h, _ = key_heads.shape
assert h % kv_h == 0, f"query_heads {h} must be a multiple of kv_heads {kv_h}"

query_heads = jnp.reshape(query_heads, (b, t, kv_h, h // kv_h, d))

Compute attention weights.
Attention softmax is always carried out in fp32.
attn_logits = jnp.einsum("...thHd,...Thd->...hHtT", query_heads,
key_heads).astype(
 jnp.float32
)
attn_logits *= self.attn_output_multiplier
max_attn_val = jnp.array(30.0, dtype=attn_logits.dtype)
attn_logits = max_attn_val * jnp.tanh(attn_logits / max_attn_val)

mask = mask[:, :, None, :, :]

if mask is not None:
 if mask.ndim != attn_logits.ndim:
 raise ValueError(
 f"Mask dimensionality {mask.ndim} must match logits dimensionality
 f'{attn_logits.ndim} for {mask.shape}/{attn_logits.shape}."
)
 attn_logits = jnp.where(mask, attn_logits, -1e30)
attn_weights = jax.nn.softmax(attn_logits).astype(query.dtype) # [H, T', T]

```

```

Weight the values by the attention and flatten the head vectors.
attn = jnp.einsum("...hHtT,...Thd->...thHd", attn_weights, value_heads)
leading_dims = attn.shape[:2]
attn = jnp.reshape(attn, (*leading_dims, -1)) # [T', H*V]

Apply another projection to get the final embeddings.
final_projection = Linear(self.model_size, with_bias=False)
return MHAOutput(final_projection(attn))

```

```

@hk.transparent
def _linear_projection(
 self,
 x: jax.Array,
 head_size: int,
 num_heads: int,
 name: Optional[str] = None,
) -> jax.Array:
 y = Linear(num_heads * head_size, with_bias=False, name=name)(x)
 *leading_dims, _ = x.shape
 return y.reshape((*leading_dims, num_heads, head_size))

```

```

@dataclass
class MHABlock(hk.Module):
 """A MHA Block"""

 num_q_heads: int
 num_kv_heads: int
 key_size: int
 attn_output_multiplier: float = 1.0

 @hk.transparent
 def __call__(
 self,
 inputs: jax.Array, # [B, T, D]
 mask: jax.Array, # [B, 1, T, T] or [B, 1, 1, T] or B[1, 1, 1, 1]
) -> MHAOutput:
 _, _, model_size = inputs.shape
 assert mask.ndim == 4, f"shape: {mask.shape}"
 assert mask.shape[2] in {1, inputs.shape[1]}, str(mask.shape)
 assert mask.shape[3] in {1, inputs.shape[1]}, str(mask.shape)
 side_input = inputs

 def attn_block(query, key, value, mask) -> MHAOutput:
 return MultiHeadAttention(
 num_q_heads=self.num_q_heads,
 num_kv_heads=self.num_kv_heads,
 key_size=self.key_size,
 model_size=model_size,
 attn_output_multiplier=self.attn_output_multiplier,
)(query, key, value, mask)

 attn_output = attn_block(inputs, side_input, side_input, mask)
 h_attn = attn_output.embeddings

 return MHAOutput(embeddings=h_attn)

```

```

@dataclass
class DenseBlock(hk.Module):
 num_q_heads: int
 num_kv_heads: int
 key_size: int
 widening_factor: float = 4.0

```

```

@hk.transparent
def __call__(
 self,
 inputs: jax.Array, # [B, T, D]
) -> jax.Array: # [B, T, D]
 _, _, model_size = inputs.shape
 h_v = Linear(
 ffn_size(model_size, self.widening_factor),
 with_bias=False,
 name="linear_v",
)(inputs)
 h_w1 = jax.nn.gelu(
 Linear(
 ffn_size(model_size, self.widening_factor),
 with_bias=False,
)(inputs)
)
 h_dense = Linear(model_size, with_bias=False)(h_w1 * h_v)

 return h_dense

```

```

@dataclass
class DecoderLayer(hk.Module):
 """A transformer stack."""

 num_q_heads: int
 num_kv_heads: int
 key_size: int
 num_layers: int
 layer_index: Optional[int] = None
 widening_factor: float = 4.0
 name: Optional[str] = None
 attn_output_multiplier: float = 1.0

 def __call__(
 self,
 inputs: jax.Array, # [B, T, D]
 mask: jax.Array, # [B, 1, T, T] or [B, 1, 1, T]
 padding_mask: Optional[jax.Array],
) -> DecoderOutput:
 """Transforms input embedding sequences to output embedding sequences."""
 del padding_mask # Unused.

 def layer_norm(x):
 return hk_rms_norm(x)

 h = inputs

 attn_output = MHABlock(
 num_q_heads=self.num_q_heads,
 num_kv_heads=self.num_kv_heads,
 key_size=self.key_size,
 attn_output_multiplier=self.attn_output_multiplier,
)(layer_norm(h), mask)
 h_attn = attn_output.embeddings

 h_attn = layer_norm(h_attn)
 h += h_attn

 def base_dense_block(h):
 h = DenseBlock(
 num_q_heads=self.num_q_heads,

```

```

 num_kv_heads=self.num_kv_heads,
 key_size=self.key_size,
 widening_factor=self.widening_factor,
)(h)
 return h

 h_dense = base_dense_block(layer_norm(h))

 h_dense = layer_norm(h_dense)
 h += h_dense

 return DecoderOutput(
 embeddings=h,
)

def layer_norm(x):
 return hk_rms_norm(x)

@dataclass
class Transformer(hk.Module):
 """A transformer stack."""

 num_q_heads: int
 num_kv_heads: int
 key_size: int
 widening_factor: float
 attn_output_multiplier: float
 num_layers: int
 name: Optional[str] = None

 def __call__(
 self,
 embeddings: jax.Array, # [B, T, D]
 mask: jax.Array, # [B, T]
 candidate_start_offset: Optional[int] = None,
) -> TransformerOutput:
 """Transforms input embedding sequences to output embedding sequences.

 Args:
 embeddings: Input embeddings of shape [B, T, D]
 mask: Padding mask of shape [B, T], True for valid positions
 candidate_start_offset: If provided, positions >= this offset are treated
as
 candidates that can only attend to positions before the offset
(user+history)
 and themselves (self-attention), but not to other candidates.
 Used for recommendation system inference.

 Returns:
 TransformerOutput containing the output embeddings.
 """

 fprop_dtype = embeddings.dtype
 _, seq_len, _ = embeddings.shape
 padding_mask = mask.copy()
 mask = mask[:, None, None, :] # [B, H=1, T'=1, T]

 if candidate_start_offset is not None:
 # Use recommendation system attention mask where candidates attend to
 # user+history and themselves, but not to other candidates
 attn_mask = make_recsys_attn_mask(seq_len, candidate_start_offset,
fprop_dtype)

```

```

 mask = mask * attn_mask
 else:
 # Standard causal mask for autoregressive sequence modelling
 causal_mask = jnp.tril(jnp.ones((1, 1, seq_len, seq_len))).astype(
 fprop_dtype
) # [B=1, H=1, T, T]
 mask = mask * causal_mask # [B, H=1, T, T]

 h = embeddings

 def block(
 h,
 mask,
 padding_mask,
 layer_index: Optional[int] = None,
 widening_factor: Optional[int] = None,
 name: Optional[str] = None,
) -> DecoderOutput:
 return DecoderLayer(
 num_q_heads=self.num_q_heads,
 num_kv_heads=self.num_kv_heads,
 key_size=self.key_size,
 widening_factor=widening_factor or self.widening_factor,
 num_layers=self.num_layers,
 attn_output_multiplier=self.attn_output_multiplier,
 name=name,
 layer_index=layer_index,
)(h, mask, padding_mask)

 for i in range(self.num_layers):
 decoder_output = block(
 h,
 mask,
 padding_mask,
 layer_index=i,
 name=f"decoder_layer_{i}",
)
 h = decoder_output.embeddings

 return TransformerOutput(
 embeddings=h,
)

```

```

=====
FILE: phoenix/pyproject.toml
=====

```

```

[project]
name = "grok-1"
version = "0.1.0"
description = "Grok-1 model"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
 "dm-haiku>=0.0.13",
 "jax==0.8.1",
 "numpy>=1.26.4",
 "pyright>=1.1.408",
]

[tool.uv]
environments = [
 "sys_platform == 'darwin'",

```

```

 "sys_platform == 'linux'",
]

[tool.ruff]
indent-width = 4
line-length = 100

[tool.ruff.lint]
ignore = [
 "E722",
 "E731",
 "E741",
 "F405",
 "E402",
 "F403",
]
select = ["ISC001"]

[dependency-groups]
dev = [
 "pytest",
]

```

```

=====
FILE: phoenix/recsys_model.py
=====

```

```

Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

import logging
from dataclasses import dataclass
from typing import Any, NamedTuple, Optional, Tuple

import haiku as hk
import jax
import jax.numpy as jnp

from grok import (
 TransformerConfig,
 Transformer,
 layer_norm,
)

logger = logging.getLogger(__name__)

@dataclass
class HashConfig:
 """Configuration for hash-based embeddings."""

 num_user_hashes: int = 2

```

```

num_item_hashes: int = 2
num_author_hashes: int = 2

```

```
@dataclass
```

```
class RecsysEmbeddings:
```

```
 """Container for pre-looked-up embeddings from the embedding tables.
```

```

 These embeddings are looked up from hash tables before being passed to the model.
 The block_*_reduce functions will combine multiple hash embeddings into single
 representations.
 """

```

```

 user_embeddings: jax.typing.ArrayLike
 history_post_embeddings: jax.typing.ArrayLike
 candidate_post_embeddings: jax.typing.ArrayLike
 history_author_embeddings: jax.typing.ArrayLike
 candidate_author_embeddings: jax.typing.ArrayLike

```

```
class RecsysModelOutput(NamedTuple):
```

```
 """Output of the recommendation model."""
```

```
 logits: jax.Array
```

```
class RecsysBatch(NamedTuple):
```

```
 """Input batch for the recommendation model.
```

```

 Contains the feature data (hashes, actions, product surfaces) but NOT the
 embeddings.

```

```

 Embeddings are passed separately via RecsysEmbeddings.
 """

```

```

 user_hashes: jax.typing.ArrayLike
 history_post_hashes: jax.typing.ArrayLike
 history_author_hashes: jax.typing.ArrayLike
 history_actions: jax.typing.ArrayLike
 history_product_surface: jax.typing.ArrayLike
 candidate_post_hashes: jax.typing.ArrayLike
 candidate_author_hashes: jax.typing.ArrayLike
 candidate_product_surface: jax.typing.ArrayLike

```

```
def block_user_reduce(
```

```

 user_hashes: jnp.ndarray,
 user_embeddings: jnp.ndarray,
 num_user_hashes: int,
 emb_size: int,
 embed_init_scale: float = 1.0,

```

```
) -> Tuple[jax.Array, jax.Array]:
```

```
 """Combine multiple user hash embeddings into a single user representation.
```

```
 Args:
```

```

 user_hashes: [B, num_user_hashes] - hash values (0 = invalid/padding)
 user_embeddings: [B, num_user_hashes, D] - looked-up embeddings
 num_user_hashes: number of hash functions used
 emb_size: embedding dimension D
 embed_init_scale: initialization scale for projection

```

```
 Returns:
```

```

 user_embedding: [B, 1, D] - combined user embedding
 user_padding_mask: [B, 1] - True where user is valid
 """

```

```

B = user_embeddings.shape[0]
D = emb_size

user_embedding = user_embeddings.reshape((B, 1, num_user_hashes * D))

embed_init = hk.initializers.VarianceScaling(embed_init_scale, mode="fan_out")
proj_mat_1 = hk.get_parameter(
 "proj_mat_1",
 [num_user_hashes * D, D],
 dtype=jnp.float32,
 init=lambda shape, dtype: embed_init(list(reversed(shape)), dtype).T,
)

user_embedding = jnp.dot(user_embedding.astype(proj_mat_1.dtype),
proj_mat_1).astype(
 user_embeddings.dtype
)

hash 0 is reserved for padding)
user_padding_mask = (user_hashes[:, 0] != 0).reshape(B, 1).astype(jnp.bool_)

return user_embedding, user_padding_mask

def block_history_reduce(
 history_post_hashes: jnp.ndarray,
 history_post_embeddings: jnp.ndarray,
 history_author_embeddings: jnp.ndarray,
 history_product_surface_embeddings: jnp.ndarray,
 history_actions_embeddings: jnp.ndarray,
 num_item_hashes: int,
 num_author_hashes: int,
 embed_init_scale: float = 1.0,
) -> Tuple[jax.Array, jax.Array]:
 """Combine history embeddings (post, author, actions, product_surface) into
sequence.

Args:
 history_post_hashes: [B, S, num_item_hashes]
 history_post_embeddings: [B, S, num_item_hashes, D]
 history_author_embeddings: [B, S, num_author_hashes, D]
 history_product_surface_embeddings: [B, S, D]
 history_actions_embeddings: [B, S, D]
 num_item_hashes: number of hash functions for items
 num_author_hashes: number of hash functions for authors
 emb_size: embedding dimension D
 embed_init_scale: initialization scale

Returns:
 history_embeddings: [B, S, D]
 history_padding_mask: [B, S]
 """
 B, S, _, D = history_post_embeddings.shape

 history_post_embeddings_reshaped = history_post_embeddings.reshape((B, S,
num_item_hashes * D))
 history_author_embeddings_reshaped = history_author_embeddings.reshape(
 (B, S, num_author_hashes * D)
)

 post_author_embedding = jnp.concatenate(
 [
 history_post_embeddings_reshaped,
 history_author_embeddings_reshaped,

```

```

 history_actions_embeddings,
 history_product_surface_embeddings,
],
 axis=-1,
)

embed_init = hk.initializers.VarianceScaling(embed_init_scale, mode="fan_out")
proj_mat_3 = hk.get_parameter(
 "proj_mat_3",
 [post_author_embedding.shape[-1], D],
 dtype=jnp.float32,
 init=lambda shape, dtype: embed_init(list(reversed(shape)), dtype).T,
)

history_embedding = jnp.dot(post_author_embedding.astype(proj_mat_3.dtype),
proj_mat_3).astype(
 post_author_embedding.dtype
)

history_embedding = history_embedding.reshape(B, S, D)

history_padding_mask = (history_post_hashes[:, :, 0] != 0).reshape(B, S)

return history_embedding, history_padding_mask

def block_candidate_reduce(
 candidate_post_hashes: jnp.ndarray,
 candidate_post_embeddings: jnp.ndarray,
 candidate_author_embeddings: jnp.ndarray,
 candidate_product_surface_embeddings: jnp.ndarray,
 num_item_hashes: int,
 num_author_hashes: int,
 embed_init_scale: float = 1.0,
) -> Tuple[jax.Array, jax.Array]:
 """Combine candidate embeddings (post, author, product_surface) into sequence.

 Args:
 candidate_post_hashes: [B, C, num_item_hashes]
 candidate_post_embeddings: [B, C, num_item_hashes, D]
 candidate_author_embeddings: [B, C, num_author_hashes, D]
 candidate_product_surface_embeddings: [B, C, D]
 num_item_hashes: number of hash functions for items
 num_author_hashes: number of hash functions for authors
 emb_size: embedding dimension D
 embed_init_scale: initialization scale

 Returns:
 candidate_embeddings: [B, C, D]
 candidate_padding_mask: [B, C]
 """
 B, C, _, D = candidate_post_embeddings.shape

 candidate_post_embeddings_reshaped = candidate_post_embeddings.reshape(
 (B, C, num_item_hashes * D)
)
 candidate_author_embeddings_reshaped = candidate_author_embeddings.reshape(
 (B, C, num_author_hashes * D)
)

 post_author_embedding = jnp.concatenate(
 [
 candidate_post_embeddings_reshaped,
 candidate_author_embeddings_reshaped,

```

```

 candidate_product_surface_embeddings,
],
 axis=-1,
)

embed_init = hk.initializers.VarianceScaling(embed_init_scale, mode="fan_out")
proj_mat_2 = hk.get_parameter(
 "proj_mat_2",
 [post_author_embedding.shape[-1], D],
 dtype=jnp.float32,
 init=lambda shape, dtype: embed_init(list(reversed(shape)), dtype).T,
)

candidate_embedding = jnp.dot(
 post_author_embedding.astype(proj_mat_2.dtype), proj_mat_2
).astype(post_author_embedding.dtype)

candidate_padding_mask = (candidate_post_hashes[:, :, 0] != 0).reshape(B,
C).astype(jnp.bool_)

return candidate_embedding, candidate_padding_mask

```

```

@dataclass
class PhoenixModelConfig:
 """Configuration for the recommendation system model."""

 model: TransformerConfig
 emb_size: int
 num_actions: int
 history_seq_len: int = 128
 candidate_seq_len: int = 32

 name: Optional[str] = None
 fprop_dtype: Any = jnp.bfloat16

 hash_config: HashConfig = None # type: ignore

 product_surface_vocab_size: int = 16

 _initialized = False

 def __post_init__(self):
 if self.hash_config is None:
 self.hash_config = HashConfig()

 def initialize(self):
 self._initialized = True
 return self

 def make(self):
 if not self._initialized:
 logger.warning(f"PhoenixModel {self.name} is not initialized.
Initializing.")
 self.initialize()

 return PhoenixModel(
 model=self.model.make(),
 config=self,
 fprop_dtype=self.fprop_dtype,
)

```

```

@dataclass

```

```

class PhoenixModel(hk.Module):
 """A transformer-based recommendation model for ranking candidates."""

 model: Transformer
 config: PhoenixModelConfig
 fprop_dtype: Any = jnp.bfloat16
 name: Optional[str] = None

 def _get_action_embeddings(
 self,
 actions: jax.Array,
) -> jax.Array:
 """Convert multi-hot action vectors to embeddings.

 Uses a learned projection matrix to map the signed action vector
 to the embedding dimension. This works for any number of actions.
 """
 config = self.config
 _, _, num_actions = actions.shape
 D = config.emb_size

 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")
 action_projection = hk.get_parameter(
 "action_projection",
 [num_actions, D],
 dtype=jnp.float32,
 init=embed_init,
)

 actions_signed = (2 * actions - 1).astype(jnp.float32)

 action_emb = jnp.dot(actions_signed.astype(action_projection.dtype),
 action_projection)

 valid_mask = jnp.any(actions, axis=-1, keepdims=True)
 action_emb = action_emb * valid_mask

 return action_emb.astype(self.fprop_dtype)

 def _single_hot_to_embeddings(
 self,
 input: jax.Array,
 vocab_size: int,
 emb_size: int,
 name: str,
) -> jax.Array:
 """Convert single-hot indices to embeddings via lookup table.

 Args:
 input: [B, S] tensor of categorical indices
 vocab_size: size of the vocabulary
 emb_size: embedding dimension
 name: name for the embedding table parameter

 Returns:
 embeddings: [B, S, emb_size]
 """
 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")
 embedding_table = hk.get_parameter(
 name,
 [vocab_size, emb_size],
 dtype=jnp.float32,
 init=embed_init,
)

```

```

 input_one_hot = jax.nn.one_hot(input, vocab_size)
 output = jnp.dot(input_one_hot, embedding_table)
 return output.astype(self.fprop_dtype)

def _get_unembedding(self) -> jax.Array:
 """Get the unembedding matrix for decoding to logits."""
 config = self.config
 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")
 unembed_mat = hk.get_parameter(
 "unembeddings",
 [config.emb_size, config.num_actions],
 dtype=jnp.float32,
 init=embed_init,
)
 return unembed_mat

def build_inputs(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
) -> Tuple[jax.Array, jax.Array, int]:
 """Build input embeddings from batch and pre-looked-up embeddings.

 Args:
 batch: RecsysBatch containing hashes, actions, product surfaces
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 embeddings: [B, 1 + history_len + num_candidates, D]
 padding_mask: [B, 1 + history_len + num_candidates]
 candidate_start_offset: int - position where candidates start
 """
 config = self.config
 hash_config = config.hash_config

 history_product_surface_embeddings = self._single_hot_to_embeddings(
 batch.history_product_surface, # type: ignore
 config.product_surface_vocab_size,
 config.emb_size,
 "product_surface_embedding_table",
)
 candidate_product_surface_embeddings = self._single_hot_to_embeddings(
 batch.candidate_product_surface, # type: ignore
 config.product_surface_vocab_size,
 config.emb_size,
 "product_surface_embedding_table",
)

 history_actions_embeddings = self._get_action_embeddings(batch.history_actions)
 # type: ignore

 user_embeddings, user_padding_mask = block_user_reduce(
 batch.user_hashes, # type: ignore
 recsys_embeddings.user_embeddings, # type: ignore
 hash_config.num_user_hashes,
 config.emb_size,
 1.0,
)

 history_embeddings, history_padding_mask = block_history_reduce(
 batch.history_post_hashes, # type: ignore
 recsys_embeddings.history_post_embeddings, # type: ignore
 recsys_embeddings.history_author_embeddings, # type: ignore

```

```

 history_product_surface_embeddings,
 history_actions_embeddings,
 hash_config.num_item_hashes,
 hash_config.num_author_hashes,
 1.0,
)

 candidate_embeddings, candidate_padding_mask = block_candidate_reduce(
 batch.candidate_post_hashes, # type: ignore
 recsys_embeddings.candidate_post_embeddings, # type: ignore
 recsys_embeddings.candidate_author_embeddings, # type: ignore
 candidate_product_surface_embeddings,
 hash_config.num_item_hashes,
 hash_config.num_author_hashes,
 1.0,
)

 embeddings = jnp.concatenate(
 [user_embeddings, history_embeddings, candidate_embeddings], axis=1
)
 padding_mask = jnp.concatenate(
 [user_padding_mask, history_padding_mask, candidate_padding_mask], axis=1
)

 candidate_start_offset = user_padding_mask.shape[1] +
 history_padding_mask.shape[1]

 return embeddings.astype(self.fprop_dtype), padding_mask,
 candidate_start_offset

def __call__(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
) -> RecsysModelOutput:
 """Forward pass for ranking candidates.

 Args:
 batch: RecsysBatch containing hashes, actions, product surfaces
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 RecsysModelOutput containing logits for each candidate. Shape = [B,
num_candidates, num_actions]
 """
 embeddings, padding_mask, candidate_start_offset = self.build_inputs(
 batch, recsys_embeddings
)

 # transformer
 model_output = self.model(
 embeddings,
 padding_mask,
 candidate_start_offset=candidate_start_offset,
)

 out_embeddings = model_output.embeddings
 out_embeddings = layer_norm(out_embeddings)

 candidate_embeddings = out_embeddings[:, candidate_start_offset:, :]

 unembeddings = self._get_unembedding()
 logits = jnp.dot(candidate_embeddings.astype(unembeddings.dtype), unembeddings)

```

```

 logits = logits.astype(self.fprop_dtype)

 return RecsysModelOutput(logits=logits)

```

```

=====
FILE: phoenix/recsys_retrieval_model.py
=====

```

```

Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

import logging
from dataclasses import dataclass
from typing import Any, NamedTuple, Optional, Tuple

import haiku as hk
import jax
import jax.numpy as jnp

from grok import TransformerConfig, Transformer
from recsys_model import (
 HashConfig,
 RecsysBatch,
 RecsysEmbeddings,
 block_history_reduce,
 block_user_reduce,
)

logger = logging.getLogger(__name__)

EPS = 1e-12
INF = 1e12

class RetrievalOutput(NamedTuple):
 """Output of the retrieval model."""

 user_representation: jax.Array
 top_k_indices: jax.Array
 top_k_scores: jax.Array

@dataclass
class CandidateTower(hk.Module):
 """Candidate tower that projects post+author embeddings to a shared embedding
 space.

 This tower takes the concatenated embeddings of a post and its author,
 and projects them to a normalized representation suitable for similarity search.
 """

 emb_size: int

```

```

name: Optional[str] = None

def __call__(self, post_author_embedding: jax.Array) -> jax.Array:
 """Project post+author embeddings to normalized representation.

 Args:
 post_author_embedding: Concatenated post and author embeddings
 Shape: [B, C, num_hashes, D] or [B, num_hashes, D]

 Returns:
 Normalized candidate representation
 Shape: [B, C, D] or [B, D]
 """
 if len(post_author_embedding.shape) == 4:
 B, C, _, _ = post_author_embedding.shape
 post_author_embedding = jnp.reshape(post_author_embedding, (B, C, -1))
 else:
 B, _, _ = post_author_embedding.shape
 post_author_embedding = jnp.reshape(post_author_embedding, (B, -1))

 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")

 proj_1 = hk.get_parameter(
 "candidate_tower_projection_1",
 [post_author_embedding.shape[-1], self.emb_size * 2],
 dtype=jnp.float32,
 init=embed_init,
)

 proj_2 = hk.get_parameter(
 "candidate_tower_projection_2",
 [self.emb_size * 2, self.emb_size],
 dtype=jnp.float32,
 init=embed_init,
)

 hidden = jnp.dot(post_author_embedding.astype(proj_1.dtype), proj_1)
 hidden = jax.nn.silu(hidden)
 candidate_embeddings = jnp.dot(hidden.astype(proj_2.dtype), proj_2)

 candidate_norm_sq = jnp.sum(candidate_embeddings**2, axis=-1, keepdims=True)
 candidate_norm = jnp.sqrt(jnp.maximum(candidate_norm_sq, EPS))
 candidate_representation = candidate_embeddings / candidate_norm

 return candidate_representation.astype(post_author_embedding.dtype)

```

```

@dataclass
class PhoenixRetrievalModelConfig:
 """Configuration for the Phoenix Retrieval Model.

```

```

 This model uses the same transformer architecture as the Phoenix ranker
 for encoding user representations.
 """

```

```

 model: TransformerConfig
 emb_size: int
 history_seq_len: int = 128
 candidate_seq_len: int = 32

 name: Optional[str] = None
 fprop_dtype: Any = jnp.bfloat16

 hash_config: HashConfig = None # type: ignore

```

```

product_surface_vocab_size: int = 16

_initialized: bool = False

def __post_init__(self):
 if self.hash_config is None:
 self.hash_config = HashConfig()

def initialize(self):
 self._initialized = True
 return self

def make(self):
 if not self._initialized:
 logger.warning(f"PhoenixRetrievalModel {self.name} is not initialized. Initializing.")
 self.initialize()

 return PhoenixRetrievalModel(
 model=self.model.make(),
 config=self,
 fprop_dtype=self.fprop_dtype,
)

@dataclass
class PhoenixRetrievalModel(hk.Module):
 """A two-tower retrieval model using the Phoenix transformer for user encoding.

 This model implements the two-tower architecture for efficient retrieval:
 - User Tower: Encodes user features + history using the Phoenix transformer
 - Candidate Tower: Projects candidate embeddings to a shared space

 The user and candidate representations are L2-normalized, enabling efficient
 approximate nearest neighbor (ANN) search using dot product similarity.
 """

 model: Transformer
 config: PhoenixRetrievalModelConfig
 fprop_dtype: Any = jnp.bfloat16
 name: Optional[str] = None

 def _get_action_embeddings(
 self,
 actions: jax.Array,
) -> jax.Array:
 """Convert multi-hot action vectors to embeddings."""
 config = self.config
 _, _, num_actions = actions.shape
 D = config.emb_size

 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")
 action_projection = hk.get_parameter(
 "action_projection",
 [num_actions, D],
 dtype=jnp.float32,
 init=embed_init,
)

 actions_signed = (2 * actions - 1).astype(jnp.float32)
 action_emb = jnp.dot(actions_signed.astype(action_projection.dtype),
 action_projection)

```

```

valid_mask = jnp.any(actions, axis=-1, keepdims=True)
action_emb = action_emb * valid_mask

return action_emb.astype(self.fprop_dtype)

def _single_hot_to_embeddings(
 self,
 input: jax.Array,
 vocab_size: int,
 emb_size: int,
 name: str,
) -> jax.Array:
 """Convert single-hot indices to embeddings via lookup table."""
 embed_init = hk.initializers.VarianceScaling(1.0, mode="fan_out")
 embedding_table = hk.get_parameter(
 name,
 [vocab_size, emb_size],
 dtype=jnp.float32,
 init=embed_init,
)

 input_one_hot = jax.nn.one_hot(input, vocab_size)
 output = jnp.dot(input_one_hot, embedding_table)
 return output.astype(self.fprop_dtype)

def build_user_representation(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
) -> Tuple[jax.Array, jax.Array]:
 """Build user representation from user features and history.

 Uses the Phoenix transformer to encode user + history embeddings
 into a single user representation vector.

 Args:
 batch: RecsysBatch containing hashes, actions, product surfaces
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 user_representation: L2-normalized user embedding [B, D]
 user_norm: Pre-normalization L2 norm [B, 1]
 """
 config = self.config
 hash_config = config.hash_config

 history_product_surface_embeddings = self._single_hot_to_embeddings(
 batch.history_product_surface, # type: ignore
 config.product_surface_vocab_size,
 config.emb_size,
 "product_surface_embedding_table",
)

 history_actions_embeddings = self._get_action_embeddings(batch.history_actions)
 # type: ignore

 user_embeddings, user_padding_mask = block_user_reduce(
 batch.user_hashes, # type: ignore
 recsys_embeddings.user_embeddings, # type: ignore
 hash_config.num_user_hashes,
 config.emb_size,
 1.0,
)

```

```

history_embeddings, history_padding_mask = block_history_reduce(
 batch.history_post_hashes, # type: ignore
 recsys_embeddings.history_post_embeddings, # type: ignore
 recsys_embeddings.history_author_embeddings, # type: ignore
 history_product_surface_embeddings,
 history_actions_embeddings,
 hash_config.num_item_hashes,
 hash_config.num_author_hashes,
 1.0,
)

embeddings = jnp.concatenate([user_embeddings, history_embeddings], axis=1)
padding_mask = jnp.concatenate([user_padding_mask, history_padding_mask],
axis=1)

model_output = self.model(
 embeddings.astype(self.fprop_dtype),
 padding_mask,
 candidate_start_offset=None,
)

user_outputs = model_output.embeddings

mask_float = padding_mask.astype(jnp.float32)[: , :, None] # [B, T, 1]
user_embeddings_masked = user_outputs * mask_float
user_embedding_sum = jnp.sum(user_embeddings_masked, axis=1) # [B, D]
mask_sum = jnp.sum(mask_float, axis=1) # [B, 1]
user_representation = user_embedding_sum / jnp.maximum(mask_sum, 1.0)

user_norm_sq = jnp.sum(user_representation**2, axis=-1, keepdims=True)
user_norm = jnp.sqrt(jnp.maximum(user_norm_sq, EPS))
user_representation = user_representation / user_norm

return user_representation, user_norm

def build_candidate_representation(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
) -> Tuple[jax.Array, jax.Array]:
 """Build candidate (item) representations.

 Projects post + author embeddings to a shared embedding space
 using the candidate tower MLP.

 Args:
 batch: RecsysBatch containing candidate hashes
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 candidate_representation: L2-normalized candidate embeddings [B, C, D]
 candidate_padding_mask: Valid candidate mask [B, C]
 """
 config = self.config

 candidate_post_embeddings = recsys_embeddings.candidate_post_embeddings
 candidate_author_embeddings = recsys_embeddings.candidate_author_embeddings

 post_author_embedding = jnp.concatenate(
 [candidate_post_embeddings, candidate_author_embeddings], axis=2
)

 candidate_tower = CandidateTower(
 emb_size=config.emb_size,

```

```

)
 candidate_representation = candidate_tower(post_author_embedding)

 candidate_padding_mask = (batch.candidate_post_hashes[:, :, 0] !=
0).astype(jnp.bool_) # type: ignore

 return candidate_representation, candidate_padding_mask

def __call__(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
 corpus_embeddings: jax.Array,
 top_k: int,
 corpus_mask: Optional[jax.Array] = None,
) -> RetrievalOutput:
 """Retrieve top-k candidates from corpus for each user.

 Args:
 batch: RecsysBatch containing hashes, actions, product surfaces
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings
 corpus_embeddings: [N, D] normalized corpus candidate embeddings
 top_k: Number of candidates to retrieve
 corpus_mask: [N] optional mask for valid corpus entries

 Returns:
 """ RetrievalOutput containing user representation and top-k results
 """
 user_representation, _ = self.build_user_representation(batch,
recsys_embeddings)

 top_k_indices, top_k_scores = self._retrieve_top_k(
 user_representation, corpus_embeddings, top_k, corpus_mask
)

 return RetrievalOutput(
 user_representation=user_representation,
 top_k_indices=top_k_indices,
 top_k_scores=top_k_scores,
)

def _retrieve_top_k(
 self,
 user_representation: jax.Array,
 corpus_embeddings: jax.Array,
 top_k: int,
 corpus_mask: Optional[jax.Array] = None,
) -> Tuple[jax.Array, jax.Array]:
 """Retrieve top-k candidates from a corpus for each user.

 Args:
 user_representation: [B, D] normalized user embeddings
 corpus_embeddings: [N, D] normalized corpus candidate embeddings
 top_k: Number of candidates to retrieve
 corpus_mask: [N] optional mask for valid corpus entries

 Returns:
 top_k_indices: [B, K] indices of top-k candidates
 top_k_scores: [B, K] similarity scores of top-k candidates
 """
 scores = jnp.matmul(user_representation, corpus_embeddings.T)

 if corpus_mask is not None:
 scores = jnp.where(corpus_mask[None, :], scores, -INF)

```

```

top_k_scores, top_k_indices = jax.lax.top_k(scores, top_k)

return top_k_indices, top_k_scores

```

```

=====
FILE: phoenix/run_ranker.py
=====

```

```

Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

import logging

import numpy as np

from grok import TransformerConfig
from recsys_model import PhoenixModelConfig, HashConfig
from runners import RecsysInferenceRunner, ModelRunner, create_example_batch, ACTIONS

def main():
 # Model configuration
 emb_size = 128 # Embedding dimension
 num_actions = len(ACTIONS) # Number of explicit engagement actions
 history_seq_len = 32 # Max history length
 candidate_seq_len = 8 # Max candidates to rank

 # Hash configuration
 hash_config = HashConfig(
 num_user_hashes=2,
 num_item_hashes=2,
 num_author_hashes=2,
)

 recsys_model = PhoenixModelConfig(
 emb_size=emb_size,
 num_actions=num_actions,
 history_seq_len=history_seq_len,
 candidate_seq_len=candidate_seq_len,
 hash_config=hash_config,
 product_surface_vocab_size=16,
 model=TransformerConfig(
 emb_size=emb_size,
 widening_factor=2,
 key_size=64,
 num_q_heads=2,
 num_kv_heads=2,
 num_layers=2,
 attn_output_multiplier=0.125,
),
),
)

```

```

Create inference runner
inference_runner = RecsysInferenceRunner(
 runner=ModelRunner(
 model=recsys_model,
 bs_per_device=0.125,
),
 name="recsys_local",
)

print("Initializing model...")
inference_runner.initialize()
print("Model initialized!")

Create example batch with simulated posts
print("\n" + "=" * 70)
print("RECOMMENDATION SYSTEM DEMO")
print("=" * 70)

batch_size = 1
example_batch, example_embeddings = create_example_batch(
 batch_size=batch_size,
 emb_size=emb_size,
 history_len=history_seq_len,
 num_candidates=candidate_seq_len,
 num_actions=num_actions,
 num_user_hashes=hash_config.num_user_hashes,
 num_item_hashes=hash_config.num_item_hashes,
 num_author_hashes=hash_config.num_author_hashes,
 product_surface_vocab_size=recsys_model.product_surface_vocab_size,
)

action_names = [action.replace("_", " ").title() for action in ACTIONS]

Count valid history items (where first post hash is non-zero)
valid_history_count = int((example_batch.history_post_hashes[:, :, 0] != 0).sum())
type: ignore
print(f"\nUser has viewed {valid_history_count} posts in their history")
print(f"Ranking {candidate_seq_len} candidate posts...")

Rank candidates
ranking_output = inference_runner.rank(example_batch, example_embeddings)

Display results
scores = np.array(ranking_output.scores[0]) # [num_candidates, num_actions]
ranked_indices = np.array(ranking_output.ranked_indices[0]) # [num_candidates]

print("\n" + "-" * 70)
print("RANKING RESULTS (ordered by predicted 'Favorite Score' probability)")
print("-" * 70)

for rank, idx in enumerate(ranked_indices):
 idx = int(idx)
 print(f"\nRank {rank + 1}: ")
 print(" Predicted engagement probabilities:")
 for action_idx, action_name in enumerate(action_names):
 prob = float(scores[idx, action_idx])
 bar = "█" * int(prob * 20) + "░" * (20 - int(prob * 20))
 print(f" {action_name:24s}: {bar} {prob:.3f}")

print("\n" + "=" * 70)
print("Demo complete!")
print("=" * 70)

```

```

if __name__ == "__main__":
 logging.basicConfig(level=logging.INFO)
 main()

=====
FILE: phoenix/run_retrieval.py
=====
Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

import logging

import numpy as np

from grok import TransformerConfig
from recsys_model import HashConfig
from recsys_retrieval_model import PhoenixRetrievalModelConfig
from runners import (
 RecsysRetrievalInferenceRunner,
 RetrievalModelRunner,
 create_example_batch,
 create_example_corpus,
 ACTIONS,
)

def main():
 # Model configuration - same architecture as Phoenix ranker
 emb_size = 128 # Embedding dimension
 num_actions = len(ACTIONS) # Number of explicit engagement actions
 history_seq_len = 32 # Max history length
 candidate_seq_len = 8 # Max candidates per batch (for training)

 # Hash configuration
 hash_config = HashConfig(
 num_user_hashes=2,
 num_item_hashes=2,
 num_author_hashes=2,
)

 # Configure the retrieval model - uses same transformer as Phoenix
 retrieval_model_config = PhoenixRetrievalModelConfig(
 emb_size=emb_size,
 history_seq_len=history_seq_len,
 candidate_seq_len=candidate_seq_len,
 hash_config=hash_config,
 product_surface_vocab_size=16,
 model=TransformerConfig(
 emb_size=emb_size,
 widening_factor=2,

```

```

 key_size=64,
 num_q_heads=2,
 num_kv_heads=2,
 num_layers=2,
 attn_output_multiplier=0.125,
),
)

Create inference runner
inference_runner = RecsysRetrievalInferenceRunner(
 runner=RetrievalModelRunner(
 model=retrieval_model_config,
 bs_per_device=0.125,
),
 name="retrieval_local",
)

print("Initializing retrieval model...")
inference_runner.initialize()
print("Model initialized!")

Create example batch with simulated user and history
print("\n" + "-" * 70)
print("RETRIEVAL SYSTEM DEMO")
print("=" * 70)

batch_size = 2 # Two users for demo
example_batch, example_embeddings = create_example_batch(
 batch_size=batch_size,
 emb_size=emb_size,
 history_len=history_seq_len,
 num_candidates=candidate_seq_len,
 num_actions=num_actions,
 num_user_hashes=hash_config.num_user_hashes,
 num_item_hashes=hash_config.num_item_hashes,
 num_author_hashes=hash_config.num_author_hashes,
 product_surface_vocab_size=16,
)

Count valid history items
valid_history_count = int((example_batch.history_post_hashes[:, :, 0] != 0).sum())
type: ignore
print(f"\nUsers have viewed {valid_history_count} posts total in their history")

Step 1: Create a corpus of candidate posts
print("\n" + "-" * 70)
print("STEP 1: Creating Candidate Corpus")
print("-" * 70)

corpus_size = 1000 # Simulated corpus of 1000 posts
corpus_embeddings, corpus_post_ids = create_example_corpus(
 corpus_size=corpus_size,
 emb_size=emb_size,
 seed=456,
)
print(f"Corpus size: {corpus_size} posts")
print(f"Corpus embeddings shape: {corpus_embeddings.shape}")

Set corpus for retrieval
inference_runner.set_corpus(corpus_embeddings, corpus_post_ids)

Step 2: Retrieve top-k candidates for each user
print("\n" + "-" * 70)
print("STEP 2: Retrieving Top-K Candidates")

```

```

print("-" * 70)

top_k = 10
retrieval_output = inference_runner.retrieve(
 example_batch,
 example_embeddings,
 top_k=top_k,
)

print(f"Retrieved top {top_k} candidates for each of {batch_size} users:")

top_k_indices = np.array(retrieval_output.top_k_indices)
top_k_scores = np.array(retrieval_output.top_k_scores)

for user_idx in range(batch_size):
 print(f"User {user_idx + 1}:")
 print(f" {'Rank':<6} {'Post ID':<12} {'Score':<12}")
 print(f" {'-' * 30}")
 for rank in range(top_k):
 post_id = top_k_indices[user_idx, rank]
 score = top_k_scores[user_idx, rank]
 bar = "█" * int((score + 1) * 10) + "░" * (20 - int((score + 1) * 10))
 print(f" {rank + 1:<6} {post_id:<12} {bar} {score:.4f}")

print("\n" + "=" * 70)
print("Demo complete!")
print("=" * 70)

if __name__ == "__main__":
 logging.basicConfig(level=logging.INFO)
 main()

```

```

=====
FILE: phoenix/runners.py
=====

```

```

Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

```

```

import functools
import logging
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any, List, NamedTuple, Optional, Tuple

import haiku as hk
import jax
import jax.numpy as jnp
import numpy as np

```

```

from grok import TrainingState
from recsys_retrieval_model import PhoenixRetrievalModelConfig
from recsys_retrieval_model import RetrievalOutput as ModelRetrievalOutput

from recsys_model import (
 PhoenixModelConfig,
 RecsysBatch,
 RecsysEmbeddings,
 RecsysModelOutput,
)

rank_logger = logging.getLogger("rank")

def create_dummy_batch_from_config(
 hash_config: Any,
 history_len: int,
 num_candidates: int,
 num_actions: int,
 batch_size: int = 1,
) -> RecsysBatch:
 """Create a dummy batch for initialization.

 Args:
 hash_config: HashConfig with num_user_hashes, num_item_hashes,
num_author_hashes
 history_len: History sequence length
 num_candidates: Number of candidates
 num_actions: Number of action types
 batch_size: Batch size

 Returns:
 RecsysBatch with zeros
 """
 return RecsysBatch(
 user_hashes=np.zeros((batch_size, hash_config.num_user_hashes),
dtype=np. int32),
 history_post_hashes=np.zeros(
 (batch_size, history_len, hash_config.num_item_hashes), dtype=np. int32
),
 history_author_hashes=np.zeros(
 (batch_size, history_len, hash_config.num_author_hashes), dtype=np. int32
),
 history_actions=np.zeros((batch_size, history_len, num_actions),
dtype=np. float32),
 history_product_surface=np.zeros((batch_size, history_len), dtype=np. int32),
 candidate_post_hashes=np.zeros(
 (batch_size, num_candidates, hash_config.num_item_hashes), dtype=np. int32
),
 candidate_author_hashes=np.zeros(
 (batch_size, num_candidates, hash_config.num_author_hashes), dtype=np. int32
),
 candidate_product_surface=np.zeros((batch_size, num_candidates),
dtype=np. int32),
)

def create_dummy_embeddings_from_config(
 hash_config: Any,
 emb_size: int,
 history_len: int,
 num_candidates: int,
 batch_size: int = 1,
) -> RecsysEmbeddings:

```

```

"""Create dummy embeddings for initialization.

Args:
 hash_config: HashConfig with num_user_hashes, num_item_hashes,
num_author_hashes
 emb_size: Embedding dimension
 history_len: History sequence length
 num_candidates: Number of candidates
 batch_size: Batch size

Returns:
 RecsysEmbeddings with zeros
"""
return RecsysEmbeddings(
 user_embeddings=np.zeros(
 (batch_size, hash_config.num_user_hashes, emb_size), dtype=np.float32
),
 history_post_embeddings=np.zeros(
 (batch_size, history_len, hash_config.num_item_hashes, emb_size),
dtype=np.float32
),
 candidate_post_embeddings=np.zeros(
 (batch_size, num_candidates, hash_config.num_item_hashes, emb_size),
 dtype=np.float32,
),
 history_author_embeddings=np.zeros(
 (batch_size, history_len, hash_config.num_author_hashes, emb_size),
dtype=np.float32
),
 candidate_author_embeddings=np.zeros(
 (batch_size, num_candidates, hash_config.num_author_hashes, emb_size),
 dtype=np.float32,
),
)

@dataclass
class BaseModelRunner(ABC):
 """Base class for model runners with shared initialization logic."""

 bs_per_device: float = 2.0
 rng_seed: int = 42

 @property
 @abstractmethod
 def model(self) -> Any:
 """Return the model config."""
 pass

 @property
 def _model_name(self) -> str:
 """Return model name for logging."""
 return "model"

 @abstractmethod
 def make_forward_fn(self):
 """Create the forward function. Must be implemented by subclasses."""
 pass

 def initialize(self):
 """Initialize the model runner."""
 self.model.initialize()
 self.model.fprop_dtype = jnp.bfloat16
 num_local_gpus = len(jax.local_devices())

```

```

self.batch_size = max(1, int(self.bs_per_device * num_local_gpus))

rank_logger.info(f"Initializing {self._model_name}...")
self.forward = self.make_forward_fn()

```

```

@dataclass
class BaseInferenceRunner(ABC):
 """Base class for inference runners with shared dummy data creation."""

 name: str

 @property
 @abstractmethod
 def runner(self) -> BaseModelRunner:
 """Return the underlying model runner."""
 pass

 def _get_num_actions(self) -> int:
 """Get number of actions. Override in subclasses if needed."""
 model_config = self.runner.model
 if hasattr(model_config, "num_actions"):
 return model_config.num_actions
 return 19

 def create_dummy_batch(self, batch_size: int = 1) -> RecsysBatch:
 """Create a dummy batch for initialization."""
 model_config = self.runner.model
 return create_dummy_batch_from_config(
 hash_config=model_config.hash_config,
 history_len=model_config.history_seq_len,
 num_candidates=model_config.candidate_seq_len,
 num_actions=self._get_num_actions(),
 batch_size=batch_size,
)

 def create_dummy_embeddings(self, batch_size: int = 1) -> RecsysEmbeddings:
 """Create dummy embeddings for initialization."""
 model_config = self.runner.model
 return create_dummy_embeddings_from_config(
 hash_config=model_config.hash_config,
 emb_size=model_config.emb_size,
 history_len=model_config.history_seq_len,
 num_candidates=model_config.candidate_seq_len,
 batch_size=batch_size,
)

 @abstractmethod
 def initialize(self):
 """Initialize the inference runner. Must be implemented by subclasses."""
 pass

```

```

ACTIONS: List[str] = [
 "favorite_score",
 "reply_score",
 "repost_score",
 "photo_expand_score",
 "click_score",
 "profile_click_score",
 "vqv_score",
 "share_score",
 "share_via_dm_score",

```

```

 "share_via_copy_link_score",
 "dwell_score",
 "quote_score",
 "quoted_click_score",
 "follow_author_score",
 "not_interested_score",
 "block_author_score",
 "mute_author_score",
 "report_score",
 "dwell_time",
]

```

```

class RankingOutput(NamedTuple):
 """Output from ranking candidates.

```

```

 Contains both the raw scores array and individual probability fields
 for each engagement type.
 """

```

```

 scores: jax.Array

```

```

 ranked_indices: jax.Array

```

```

 p_favorite_score: jax.Array
 p_reply_score: jax.Array
 p_repost_score: jax.Array
 p_photo_expand_score: jax.Array
 p_click_score: jax.Array
 p_profile_click_score: jax.Array
 p_vqv_score: jax.Array
 p_share_score: jax.Array
 p_share_via_dm_score: jax.Array
 p_share_via_copy_link_score: jax.Array
 p_dwell_score: jax.Array
 p_quote_score: jax.Array
 p_quoted_click_score: jax.Array
 p_follow_author_score: jax.Array
 p_not_interested_score: jax.Array
 p_block_author_score: jax.Array
 p_mute_author_score: jax.Array
 p_report_score: jax.Array
 p_dwell_time: jax.Array

```

```

@dataclass

```

```

class ModelRunner(BaseModelRunner):
 """Runner for the recommendation ranking model."""

```

```

 _model: PhoenixModelConfig = None # type: ignore

```

```

 def __init__(self, model: PhoenixModelConfig, bs_per_device: float = 2.0, rng_seed:
int = 42):
 self._model = model
 self.bs_per_device = bs_per_device
 self.rng_seed = rng_seed

```

```

 @property

```

```

 def model(self) -> PhoenixModelConfig:
 return self._model

```

```

 @property

```

```

 def _model_name(self) -> str:
 return "ranking model"

```

```

def make_forward_fn(self): # type: ignore
 def forward(batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings):
 out = self.model.make()(batch, recsys_embeddings)
 return out

 return hk.transform(forward)

def init(
 self, rng: jax.Array, data: RecsysBatch, embeddings: RecsysEmbeddings
) -> TrainingState:
 assert self.forward is not None
 rng, init_rng = jax.random.split(rng)
 params = self.forward.init(init_rng, data, embeddings)
 return TrainingState(params=params)

def load_or_init(
 self,
 init_data: RecsysBatch,
 init_embeddings: RecsysEmbeddings,
):
 rng = jax.random.PRNGKey(self.rng_seed)
 state = self.init(rng, init_data, init_embeddings)
 return state

```

@dataclass

```

class RecsysInferenceRunner(BaseInferenceRunner):
 """Inference runner for the recommendation ranking model."""

 _runner: ModelRunner

 def __init__(self, runner: ModelRunner, name: str):
 self.name = name
 self._runner = runner

 @property
 def runner(self) -> ModelRunner:
 return self._runner

 def initialize(self):
 """Initialize the inference runner."""
 runner = self.runner

 dummy_batch = self.create_dummy_batch(batch_size=1)
 dummy_embeddings = self.create_dummy_embeddings(batch_size=1)

 runner.initialize()

 state = runner.load_or_init(dummy_batch, dummy_embeddings)
 self.params = state.params

 @functools.lru_cache
 def model():
 return runner.model.make()

 def hk_forward(
 batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings
) -> RecsysModelOutput:
 return model()(batch, recsys_embeddings)

 def hk_rank_candidates(
 batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings
) -> RankingOutput:

```

```

 """Rank candidates by their predicted engagement scores."""
 output = hk_forward(batch, recsys_embeddings)
 logits = output.logits

 probs = jax.nn.sigmoid(logits)

 primary_scores = probs[:, :, 0]

 ranked_indices = jnp.argsort(-primary_scores, axis=-1)

 return RankingOutput(
 scores=probs,
 ranked_indices=ranked_indices,
 p_favorite_score=probs[:, :, 0],
 p_reply_score=probs[:, :, 1],
 p_repost_score=probs[:, :, 2],
 p_photo_expand_score=probs[:, :, 3],
 p_click_score=probs[:, :, 4],
 p_profile_click_score=probs[:, :, 5],
 p_vqv_score=probs[:, :, 6],
 p_share_score=probs[:, :, 7],
 p_share_via_dm_score=probs[:, :, 8],
 p_share_via_copy_link_score=probs[:, :, 9],
 p_dwell_score=probs[:, :, 10],
 p_quote_score=probs[:, :, 11],
 p_quoted_click_score=probs[:, :, 12],
 p_follow_author_score=probs[:, :, 13],
 p_not_interested_score=probs[:, :, 14],
 p_block_author_score=probs[:, :, 15],
 p_mute_author_score=probs[:, :, 16],
 p_report_score=probs[:, :, 17],
 p_dwell_time=probs[:, :, 18],
)

 rank_ = hk.without_apply_rng(hk.transform(hk_rank_candidates))
 self.rank_candidates = rank_.apply

 def rank(self, batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings) ->
 RankingOutput:
 """Rank candidates for the given batch.

 Args:
 batch: RecsysBatch containing hashes, actions, product surfaces
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 RankingOutput with scores and ranked indices
 """
 return self.rank_candidates(self.params, batch, recsys_embeddings)

def create_example_batch(
 batch_size: int,
 emb_size: int,
 history_len: int,
 num_candidates: int,
 num_actions: int,
 num_user_hashes: int = 2,
 num_item_hashes: int = 2,
 num_author_hashes: int = 2,
 product_surface_vocab_size: int = 16,
 num_user_embeddings: int = 100000,
 num_post_embeddings: int = 100000,
 num_author_embeddings: int = 100000,

```

```
) -> Tuple[RecsysBatch, RecsysEmbeddings]:
"""Create an example batch with random data for testing.
```

This simulates a recommendation scenario where:

- We have a user with some embedding
- The user has interacted with some posts in their history
- We want to rank a set of candidate posts

Note on embedding table sizes:

The num\*\_embeddings parameters define the size of the embedding tables for each entity type. Hash values are generated in the range [1, num\*\_embeddings) to ensure they can be used as valid indices into the corresponding embedding tables. Hash value 0 is reserved for padding/invalid entries.

Returns:

```
""" Tuple of (RecsysBatch, RecsysEmbeddings)

rng = np.random.default_rng(42)

user_hashes = rng.integers(1, num_user_embeddings, size=(batch_size,
num_user_hashes)).astype(
 np.int32
)

history_post_hashes = rng.integers(
 1, num_post_embeddings, size=(batch_size, history_len, num_item_hashes)
) .astype(np.int32)

for b in range(batch_size):
 valid_len = rng.integers(history_len // 2, history_len + 1)
 history_post_hashes[b, valid_len:, :] = 0

history_author_hashes = rng.integers(
 1, num_author_embeddings, size=(batch_size, history_len, num_author_hashes)
) .astype(np.int32)
for b in range(batch_size):
 valid_len = rng.integers(history_len // 2, history_len + 1)
 history_author_hashes[b, valid_len:, :] = 0

history_actions = (rng.random(size=(batch_size, history_len, num_actions)) >
0.7) .astype(
 np.float32
)

history_product_surface = rng.integers(
 0, product_surface_vocab_size, size=(batch_size, history_len)
) .astype(np.int32)

candidate_post_hashes = rng.integers(
 1, num_post_embeddings, size=(batch_size, num_candidates, num_item_hashes)
) .astype(np.int32)

candidate_author_hashes = rng.integers(
 1, num_author_embeddings, size=(batch_size, num_candidates, num_author_hashes)
) .astype(np.int32)

candidate_product_surface = rng.integers(
 0, product_surface_vocab_size, size=(batch_size, num_candidates)
) .astype(np.int32)

batch = RecsysBatch(
 user_hashes=user_hashes,
```

```

 history_post_hashes=history_post_hashes,
 history_author_hashes=history_author_hashes,
 history_actions=history_actions,
 history_product_surface=history_product_surface,
 candidate_post_hashes=candidate_post_hashes,
 candidate_author_hashes=candidate_author_hashes,
 candidate_product_surface=candidate_product_surface,
)

 embeddings = RecsysEmbeddings(
 user_embeddings=rng.normal(size=(batch_size, num_user_hashes,
emb_size)).astype(np.float32),
 history_post_embeddings=rng.normal(
 size=(batch_size, history_len, num_item_hashes, emb_size)
).astype(np.float32),
 candidate_post_embeddings=rng.normal(
 size=(batch_size, num_candidates, num_item_hashes, emb_size)
).astype(np.float32),
 history_author_embeddings=rng.normal(
 size=(batch_size, history_len, num_author_hashes, emb_size)
).astype(np.float32),
 candidate_author_embeddings=rng.normal(
 size=(batch_size, num_candidates, num_author_hashes, emb_size)
).astype(np.float32),
)

 return batch, embeddings

```

```

class RetrievalOutput(NamedTuple):
 """Output from retrieval inference.

 Contains user representations and retrieved candidates.
 """

 user_representation: jax.Array
 top_k_indices: jax.Array
 top_k_scores: jax.Array

```

```

@dataclass
class RetrievalModelRunner(BaseModelRunner):
 """Runner for the Phoenix retrieval model."""

 _model: PhoenixRetrievalModelConfig = None # type: ignore

 def __init__(
 self,
 model: PhoenixRetrievalModelConfig,
 bs_per_device: float = 2.0,
 rng_seed: int = 42,
):
 self._model = model
 self.bs_per_device = bs_per_device
 self.rng_seed = rng_seed

 @property
 def model(self) -> PhoenixRetrievalModelConfig:
 return self._model

 @property
 def _model_name(self) -> str:

```

```

 return "retrieval model"

 def make_forward_fn(self): # type: ignore
 def forward(
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
 corpus_embeddings: jax.Array,
 top_k: int,
) -> ModelRetrievalOutput:
 model = self.model.make()
 out = model(batch, recsys_embeddings, corpus_embeddings, top_k)

 _ = model.build_candidate_representation(batch, recsys_embeddings)
 return out

 return hk.transform(forward)

 def init(
 self,
 rng: jax.Array,
 data: RecsysBatch,
 embeddings: RecsysEmbeddings,
 corpus_embeddings: jax.Array,
 top_k: int,
) -> TrainingState:
 assert self.forward is not None
 rng, init_rng = jax.random.split(rng)
 params = self.forward.init(init_rng, data, embeddings, corpus_embeddings,
top_k)
 return TrainingState(params=params)

 def load_or_init(
 self,
 init_data: RecsysBatch,
 init_embeddings: RecsysEmbeddings,
 corpus_embeddings: jax.Array,
 top_k: int,
):
 rng = jax.random.PRNGKey(self.rng_seed)
 state = self.init(rng, init_data, init_embeddings, corpus_embeddings, top_k)
 return state

```

```

@dataclass
class RecsysRetrievalInferenceRunner(BaseInferenceRunner):
 """Inference runner for the Phoenix retrieval model.

```

This runner provides methods for:

1. Encoding users to get user representations
2. Encoding candidates to get candidate embeddings
3. Retrieving top-k candidates from a corpus

```

 _runner: RetrievalModelRunner = None # type: ignore

```

```

 corpus_embeddings: jax.Array | None = None
 corpus_post_ids: jax.Array | None = None

```

```

 def __init__(self, runner: RetrievalModelRunner, name: str):
 self.name = name
 self._runner = runner
 self.corpus_embeddings = None
 self.corpus_post_ids = None

```

```

@property
def runner(self) -> RetrievalModelRunner:
 return self._runner

def initialize(self):
 """Initialize the retrieval inference runner."""
 runner = self.runner

 dummy_batch = self.create_dummy_batch(batch_size=1)
 dummy_embeddings = self.create_dummy_embeddings(batch_size=1)
 dummy_corpus = jnp.zeros((10, runner.model.emb_size), dtype=jnp.float32)
 dummy_top_k = 5

 runner.initialize()

 state = runner.load_or_init(dummy_batch, dummy_embeddings, dummy_corpus,
dummy_top_k)
 self.params = state.params

 @functools.lru_cache
 def model():
 return runner.model.make()

 def hk_encode_user(batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings) ->
jax.Array:
 """Encode user to get user representation."""
 m = model()
 user_rep, _ = m.build_user_representation(batch, recsys_embeddings)
 return user_rep

 def hk_encode_candidates(
 batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings
) -> jax.Array:
 """Encode candidates to get candidate representations."""
 m = model()
 cand_rep, _ = m.build_candidate_representation(batch, recsys_embeddings)
 return cand_rep

 def hk_retrieve(
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
 corpus_embeddings: jax.Array,
 top_k: int,
) -> "RetrievalOutput":
 """Retrieve top-k candidates from corpus."""
 m = model()
 return m(batch, recsys_embeddings, corpus_embeddings, top_k)

 encode_user_ = hk.without_apply_rng(hk.transform(hk_encode_user))
 encode_candidates_ = hk.without_apply_rng(hk.transform(hk_encode_candidates))
 retrieve_ = hk.without_apply_rng(hk.transform(hk_retrieve))

 self.encode_user_fn = encode_user_.apply
 self.encode_candidates_fn = encode_candidates_.apply
 self.retrieve_fn = retrieve_.apply

 def encode_user(self, batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings) ->
jax.Array:
 """Encode users to get user representations.

 Args:
 batch: RecsysBatch containing user and history information
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

```

```

Returns:
 """ User representations [B, D]
 return self.encode_user_fn(self.params, batch, recsys_embeddings)

def encode_candidates(
 self, batch: RecsysBatch, recsys_embeddings: RecsysEmbeddings
) -> jax.Array:
 """Encode candidates to get candidate representations.

 Args:
 batch: RecsysBatch containing candidate information
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings

 Returns:
 """ Candidate representations [B, C, D]
 return self.encode_candidates_fn(self.params, batch, recsys_embeddings)

def set_corpus(
 self,
 corpus_embeddings: jax.Array,
 corpus_post_ids: jax.Array,
):
 """Set the corpus embeddings for retrieval.

 Args:
 corpus_embeddings: Pre-computed candidate embeddings [N, D]
 corpus_post_ids: Optional post IDs corresponding to embeddings [N]

 self.corpus_embeddings = corpus_embeddings
 self.corpus_post_ids = corpus_post_ids

def retrieve(
 self,
 batch: RecsysBatch,
 recsys_embeddings: RecsysEmbeddings,
 top_k: int = 100,
 corpus_embeddings: Optional[jax.Array] = None,
) -> RetrievalOutput:
 """Retrieve top-k candidates for users.

 Args:
 batch: RecsysBatch containing user and history information
 recsys_embeddings: RecsysEmbeddings containing pre-looked-up embeddings
 top_k: Number of candidates to retrieve per user
 corpus_embeddings: Optional corpus embeddings (uses set_corpus if not
provided)

 Returns:
 """ RetrievalOutput with user representations and top-k candidates

 if corpus_embeddings is None:
 corpus_embeddings = self.corpus_embeddings

 return self.retrieve_fn(self.params, batch, recsys_embeddings,
corpus_embeddings, top_k)

def create_example_corpus(
 corpus_size: int,
 emb_size: int,
 seed: int = 123,
) -> Tuple[jax.Array, jax.Array]:

```

```
"""Create example corpus embeddings for testing retrieval.
```

```
Args:
```

```
 corpus_size: Number of candidates in corpus
 emb_size: Embedding dimension
 seed: Random seed
```

```
Returns:
```

```
""" Tuple of (corpus_embeddings [N, D], corpus_post_ids [N])
```

```
rng = np.random.default_rng(seed)
```

```
corpus_embeddings = rng.normal(size=(corpus_size, emb_size)).astype(np.float32)
norms = np.linalg.norm(corpus_embeddings, axis=-1, keepdims=True)
corpus_embeddings = corpus_embeddings / np.maximum(norms, 1e-12)
```

```
corpus_post_ids = np.arange(corpus_size, dtype=np.int64)
```

```
return jnp.array(corpus_embeddings), jnp.array(corpus_post_ids)
```

```
=====
```

```
FILE: phoenix/test_recsys_model.py
```

```
=====
```

```
Copyright 2026 X.AI Corp.
```

```
#
```

```
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
```

```
#
```

```
http://www.apache.org/licenses/LICENSE-2.0
```

```
#
```

```
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```

```
import jax.numpy as jnp
```

```
import numpy as np
```

```
import pytest
```

```
from grok import make_recsys_attn_mask
```

```
class TestMakeRecsysAttnMask:
```

```
 """Tests for the make_recsys_attn_mask function."""
```

```
 def test_output_shape(self):
```

```
 """Test that the output has the correct shape [1, 1, seq_len, seq_len]."""
```

```
 seq_len = 10
```

```
 candidate_start_offset = 5
```

```
 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
```

```
 assert mask.shape == (1, 1, seq_len, seq_len)
```

```
 def test_user_history_has_causal_attention(self):
```

```
 """Test that user+history positions (before candidate_start_offset) have causal
attention."""
```

```
 seq_len = 8
```

```
 candidate_start_offset = 5
```

```

mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
mask_2d = mask[0, 0]

for i in range(candidate_start_offset):
 for j in range(candidate_start_offset):
 if j <= i:
 assert mask_2d[i, j] == 1, f"Position {i} should attend to position
{j}"
 else:
 assert mask_2d[i, j] == 0, (
 f"Position {i} should NOT attend to future position {j}"
)

def test_candidates_attend_to_user_history(self):
 """Test that candidates can attend to all user+history positions."""
 seq_len = 8
 candidate_start_offset = 5

 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
 mask_2d = mask[0, 0]

 for candidate_pos in range(candidate_start_offset, seq_len):
 for history_pos in range(candidate_start_offset):
 assert mask_2d[candidate_pos, history_pos] == 1, (
 f"Candidate at {candidate_pos} should attend to user+history at
{history_pos}"
)

def test_candidates_attend_to_themselves(self):
 """Test that candidates can attend to themselves (self-attention)."""
 seq_len = 8
 candidate_start_offset = 5

 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
 mask_2d = mask[0, 0]

 for candidate_pos in range(candidate_start_offset, seq_len):
 assert mask_2d[candidate_pos, candidate_pos] == 1, (
 f"Candidate at {candidate_pos} should attend to itself"
)

def test_candidates_do_not_attend_to_other_candidates(self):
 """Test that candidates cannot attend to other candidates."""
 seq_len = 8
 candidate_start_offset = 5

 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
 mask_2d = mask[0, 0]

 for query_pos in range(candidate_start_offset, seq_len):
 for key_pos in range(candidate_start_offset, seq_len):
 if query_pos != key_pos:
 assert mask_2d[query_pos, key_pos] == 0, (
 f"Candidate at {query_pos} should NOT attend to candidate at
{key_pos}"
)

def test_full_mask_structure(self):
 """Test the complete mask structure with a small example."""
 # Sequence: [user, h1, h2, c1, c2, c3]
 # Positions: 0 1 2 3 4 5

 seq_len = 6
 candidate_start_offset = 3

```

```

mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
mask_2d = mask[0, 0]

Expected mask structure:
Query positions are rows, key positions are columns
1 = can attend, 0 = cannot attend
#
Keys: u h1 h2 c1 c2 c3
Query u : 1 0 0 0 0 0
Query h1 : 1 1 0 0 0 0
Query h2 : 1 1 1 0 0 0
Query c1 : 1 1 1 1 0 0 <- c1 attends to user+history + self
Query c2 : 1 1 1 0 1 0 <- c2 attends to user+history + self
Query c3 : 1 1 1 0 0 1 <- c3 attends to user+history + self

expected = np.array(
 [
 [1, 0, 0, 0, 0, 0], # user
 [1, 1, 0, 0, 0, 0], # h1
 [1, 1, 1, 0, 0, 0], # h2
 [1, 1, 1, 1, 0, 0], # c1: user+history + self
 [1, 1, 1, 0, 1, 0], # c2: user+history + self
 [1, 1, 1, 0, 0, 1], # c3: user+history + self
],
 dtype=np.float32,
)

np.testing.assert_array_equal(
 np.array(mask_2d),
 expected,
 err_msg="Full mask structure does not match expected pattern",
)

def test_dtype_preserved(self):
 """Test that the specified dtype is used."""
 seq_len = 5
 candidate_start_offset = 3

 mask_f32 = make_recsys_attn_mask(seq_len, candidate_start_offset,
dtype=jnp.float32)
 mask_f16 = make_recsys_attn_mask(seq_len, candidate_start_offset,
dtype=jnp.float16)

 assert mask_f32.dtype == jnp.float32
 assert mask_f16.dtype == jnp.float16

def test_single_candidate(self):
 """Test edge case with a single candidate."""
 seq_len = 4
 candidate_start_offset = 3

 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
 mask_2d = mask[0, 0]

 expected = np.array(
 [
 [1, 0, 0, 0],
 [1, 1, 0, 0],
 [1, 1, 1, 0],
 [1, 1, 1, 1],
],
 dtype=np.float32,
)

```

```

 np.testing.assert_array_equal(np.array(mask_2d), expected)

 def test_all_candidates(self):
 """Test edge case where all positions except first are candidates."""
 seq_len = 4
 candidate_start_offset = 1

 mask = make_recsys_attn_mask(seq_len, candidate_start_offset)
 mask_2d = mask[0, 0]

 expected = np.array(
 [
 [1, 0, 0, 0], # user
 [1, 1, 0, 0], # c1: user + self
 [1, 0, 1, 0], # c2: user + self
 [1, 0, 0, 1], # c3: user + self
],
 dtype=np.float32,
)

 np.testing.assert_array_equal(np.array(mask_2d), expected)

if __name__ == "__main__":
 pytest.main([__file__, "-v"])

```

```

=====
FILE: phoenix/test_recsys_retrieval_model.py
=====
Copyright 2026 X.AI Corp.
#
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

"""Tests for the Phoenix Retrieval Model."""

import unittest

import haiku as hk
import jax
import jax.numpy as jnp
import numpy as np

from grok import TransformerConfig
from recsys_model import HashConfig
from recsys_retrieval_model import (
 CandidateTower,
 PhoenixRetrievalModelConfig,
)
from runners import (
 RecsysRetrievalInferenceRunner,
 RetrievalModelRunner,

```

```

 create_example_batch,
 create_example_corpus,
)

```

```

class TestCandidateTower(unittest.TestCase):
 """Tests for the CandidateTower module."""

 def test_candidate_tower_output_shape(self):
 """Test that candidate tower produces correct output shape."""
 emb_size = 64
 batch_size = 4
 num_candidates = 8
 num_hashes = 4

 def forward(x):
 tower = CandidateTower(emb_size=emb_size)
 return tower(x)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 rng = jax.random.PRNGKey(0)
 x = jax.random.normal(rng, (batch_size, num_candidates, num_hashes, emb_size))

 params = forward_fn.init(rng, x)
 output = forward_fn.apply(params, x)

 self.assertEqual(output.shape, (batch_size, num_candidates, emb_size))

 def test_candidate_tower_normalized(self):
 """Test that candidate tower output is L2 normalized."""
 emb_size = 64
 batch_size = 4
 num_candidates = 8
 num_hashes = 4

 def forward(x):
 tower = CandidateTower(emb_size=emb_size)
 return tower(x)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 rng = jax.random.PRNGKey(0)
 x = jax.random.normal(rng, (batch_size, num_candidates, num_hashes, emb_size))

 params = forward_fn.init(rng, x)
 output = forward_fn.apply(params, x)

 norms = jnp.sqrt(jnp.sum(output**2, axis=-1))
 np.testing.assert_array_almost_equal(norms, jnp.ones_like(norms), decimal=5)

 def test_candidate_tower_mean_pooling(self):
 """Test candidate tower with mean pooling (no linear projection)."""
 emb_size = 64
 batch_size = 4
 num_candidates = 8
 num_hashes = 4

 def forward(x):
 tower = CandidateTower(emb_size=emb_size)
 return tower(x)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

```

```

rng = jax.random.PRNGKey(0)
x = jax.random.normal(rng, (batch_size, num_candidates, num_hashes, emb_size))

params = forward_fn.init(rng, x)
output = forward_fn.apply(params, x)

self.assertEqual(output.shape, (batch_size, num_candidates, emb_size))

norms = jnp.sqrt(jnp.sum(output**2, axis=-1))
np.testing.assert_array_almost_equal(norms, jnp.ones_like(norms), decimal=5)

```

```

class TestPhoenixRetrievalModel(unittest.TestCase):
 """Tests for the full Phoenix Retrieval Model."""

 def setUp(self):
 """Set up test fixtures."""
 self.emb_size = 64
 self.history_seq_len = 16
 self.candidate_seq_len = 8
 self.batch_size = 2
 self.num_actions = 19
 self.corpus_size = 100
 self.top_k = 10

 self.hash_config = HashConfig(
 num_user_hashes=2,
 num_item_hashes=2,
 num_author_hashes=2,
)

 self.config = PhoenixRetrievalModelConfig(
 emb_size=self.emb_size,
 history_seq_len=self.history_seq_len,
 candidate_seq_len=self.candidate_seq_len,
 hash_config=self.hash_config,
 product_surface_vocab_size=16,
 model=TransformerConfig(
 emb_size=self.emb_size,
 widening_factor=2,
 key_size=32,
 num_q_heads=2,
 num_kv_heads=2,
 num_layers=1,
 attn_output_multiplier=0.125,
),
)

 def _create_test_batch(self) -> tuple:
 """Create test batch and embeddings."""
 return create_example_batch(
 batch_size=self.batch_size,
 emb_size=self.emb_size,
 history_len=self.history_seq_len,
 num_candidates=self.candidate_seq_len,
 num_actions=self.num_actions,
 num_user_hashes=self.hash_config.num_user_hashes,
 num_item_hashes=self.hash_config.num_item_hashes,
 num_author_hashes=self.hash_config.num_author_hashes,
 product_surface_vocab_size=16,
)

 def _create_test_corpus(self):
 """Create test corpus embeddings."""

```

```

 return create_example_corpus(self.corpus_size, self.emb_size)

 def test_model_forward(self):
 """Test model forward pass produces correct output shapes."""

 def forward(batch, embeddings, corpus_embeddings, top_k):
 model = self.config.make()
 return model(batch, embeddings, corpus_embeddings, top_k)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 batch, embeddings = self._create_test_batch()
 corpus_embeddings, _ = self._create_test_corpus()

 rng = jax.random.PRNGKey(0)
 params = forward_fn.init(rng, batch, embeddings, corpus_embeddings, self.top_k)
 output = forward_fn.apply(params, batch, embeddings, corpus_embeddings,
self.top_k)

 self.assertEqual(output.user_representation.shape, (self.batch_size,
self.emb_size))
 self.assertEqual(output.top_k_indices.shape, (self.batch_size, self.top_k))
 self.assertEqual(output.top_k_scores.shape, (self.batch_size, self.top_k))

 def test_user_representation_normalized(self):
 """Test that user representations are L2 normalized."""

 def forward(batch, embeddings, corpus_embeddings, top_k):
 model = self.config.make()
 return model(batch, embeddings, corpus_embeddings, top_k)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 batch, embeddings = self._create_test_batch()
 corpus_embeddings, _ = self._create_test_corpus()

 rng = jax.random.PRNGKey(0)
 params = forward_fn.init(rng, batch, embeddings, corpus_embeddings, self.top_k)
 output = forward_fn.apply(params, batch, embeddings, corpus_embeddings,
self.top_k)

 norms = jnp.sqrt(jnp.sum(output.user_representation**2, axis=-1))
 np.testing.assert_array_almost_equal(norms, jnp.ones(self.batch_size),
decimal=5)

 def test_candidate_representation_normalized(self):
 """Test that candidate representations from build_candidate_representation are
L2 normalized."""

 def forward(batch, embeddings):
 model = self.config.make()
 cand_rep, _ = model.build_candidate_representation(batch, embeddings)
 return cand_rep

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 batch, embeddings = self._create_test_batch()

 rng = jax.random.PRNGKey(0)
 params = forward_fn.init(rng, batch, embeddings)
 cand_rep = forward_fn.apply(params, batch, embeddings)

 norms = jnp.sqrt(jnp.sum(cand_rep**2, axis=-1))
 np.testing.assert_array_almost_equal(

```

```

 norms, jnp.ones((self.batch_size, self.candidate_seq_len)), decimal=5
)

def test_retrieve_top_k(self):
 """Test top-k retrieval through __call__."""

 def forward(batch, embeddings, corpus_embeddings, top_k):
 model = self.config.make()
 return model(batch, embeddings, corpus_embeddings, top_k)

 forward_fn = hk.without_apply_rng(hk.transform(forward))

 batch, embeddings = self._create_test_batch()
 corpus_embeddings, _ = self._create_test_corpus()

 rng = jax.random.PRNGKey(0)
 params = forward_fn.init(rng, batch, embeddings, corpus_embeddings, self.top_k)
 output = forward_fn.apply(params, batch, embeddings, corpus_embeddings,
self.top_k)

 self.assertEqual(output.top_k_indices.shape, (self.batch_size, self.top_k))
 self.assertEqual(output.top_k_scores.shape, (self.batch_size, self.top_k))

 self.assertTrue(jnp.all(output.top_k_indices >= 0))
 self.assertTrue(jnp.all(output.top_k_indices < self.corpus_size))

 for b in range(self.batch_size):
 scores = np.array(output.top_k_scores[b])
 self.assertTrue(np.all(scores[:-1] >= scores[1:]))

class TestRetrievalInferenceRunner(unittest.TestCase):
 """Tests for the retrieval inference runner."""

 def setUp(self):
 """Set up test fixtures."""
 self.emb_size = 64
 self.history_seq_len = 16
 self.candidate_seq_len = 8
 self.batch_size = 2
 self.num_actions = 19

 self.hash_config = HashConfig(
 num_user_hashes=2,
 num_item_hashes=2,
 num_author_hashes=2,
)

 self.config = PhoenixRetrievalModelConfig(
 emb_size=self.emb_size,
 history_seq_len=self.history_seq_len,
 candidate_seq_len=self.candidate_seq_len,
 hash_config=self.hash_config,
 product_surface_vocab_size=16,
 model=TransformerConfig(
 emb_size=self.emb_size,
 widening_factor=2,
 key_size=32,
 num_q_heads=2,
 num_kv_heads=2,
 num_layers=1,
 attn_output_multiplier=0.125,
),
)

```

```

def test_runner_initialization(self):
 """Test that runner initializes correctly."""
 runner = RecsysRetrievalInferenceRunner(
 runner=RetrievalModelRunner(
 model=self.config,
 bs_per_device=0.125,
),
 name="test_retrieval",
)

 runner.initialize()

 self.assertIsNotNone(runner.params)

def test_runner_encode_user(self):
 """Test user encoding through runner."""
 runner = RecsysRetrievalInferenceRunner(
 runner=RetrievalModelRunner(
 model=self.config,
 bs_per_device=0.125,
),
 name="test_retrieval",
)
 runner.initialize()

 batch, embeddings = create_example_batch(
 batch_size=self.batch_size,
 emb_size=self.emb_size,
 history_len=self.history_seq_len,
 num_candidates=self.candidate_seq_len,
 num_actions=self.num_actions,
 num_user_hashes=self.hash_config.num_user_hashes,
 num_item_hashes=self.hash_config.num_item_hashes,
 num_author_hashes=self.hash_config.num_author_hashes,
)

 user_rep = runner.encode_user(batch, embeddings)

 self.assertEqual(user_rep.shape, (self.batch_size, self.emb_size))

def test_runner_retrieve(self):
 """Test retrieval through runner."""
 runner = RecsysRetrievalInferenceRunner(
 runner=RetrievalModelRunner(
 model=self.config,
 bs_per_device=0.125,
),
 name="test_retrieval",
)
 runner.initialize()

 batch, embeddings = create_example_batch(
 batch_size=self.batch_size,
 emb_size=self.emb_size,
 history_len=self.history_seq_len,
 num_candidates=self.candidate_seq_len,
 num_actions=self.num_actions,
 num_user_hashes=self.hash_config.num_user_hashes,
 num_item_hashes=self.hash_config.num_item_hashes,
 num_author_hashes=self.hash_config.num_author_hashes,
)

 corpus_size = 100

```

```

 corpus_embeddings, corpus_post_ids = create_example_corpus(corpus_size,
self.emb_size)
 runner.set_corpus(corpus_embeddings, corpus_post_ids)

 top_k = 10
 output = runner.retrieve(batch, embeddings, top_k=top_k)

 self.assertEqual(output.user_representation.shape, (self.batch_size,
self.emb_size))
 self.assertEqual(output.top_k_indices.shape, (self.batch_size, top_k))
 self.assertEqual(output.top_k_scores.shape, (self.batch_size, top_k))

if __name__ == "__main__":
 unittest.main()

```

```

=====
FILE: thunder/deserializer.rs
=====

```

```

use crate::schema::{events::Event, tweet_events::TweetEvent};
use anyhow::{Context, Result};
use prost::Message;
use thrift::protocol::{TBinaryInputProtocol, TSerializable};
use xai_thunder_proto::InNetworkEvent;

/// Deserialize a Thrift binary message into TweetEvent
pub fn deserialize_tweet_event(payload: &[u8]) -> Result<TweetEvent> {
 let mut cursor = std::io::Cursor::new(payload);
 let mut protocol = TBinaryInputProtocol::new(&mut cursor, true);

 TweetEvent::read_from_in_protocol(&mut protocol).context("Failed to deserialize
TweetEvent")
}

/// Deserialize a Thrift binary message into Event
pub fn deserialize_event(payload: &[u8]) -> Result<Event> {
 let mut cursor = std::io::Cursor::new(payload);
 let mut protocol = TBinaryInputProtocol::new(&mut cursor, true);

 Event::read_from_in_protocol(&mut protocol).context("Failed to deserialize Event")
}

/// Deserialize a proto binary message into InNetworkEvent
pub fn deserialize_tweet_event_v2(payload: &[u8]) -> Result<InNetworkEvent> {
 InNetworkEvent::decode(payload).context("Failed to deserialize InNetworkEvent")
}

```

```

=====
FILE: thunder/kafka_utils.rs
=====

```

```

use anyhow::{Context, Result};
use std::sync::Arc;
use xai_kafka::KafkaProducerConfig;
use xai_kafka::config::{KafkaConfig, KafkaConsumerConfig, SslConfig};
use xai_wily::WilyConfig;

use crate::{
 args,
 kafka::{
 tweet_events_listener::start_tweet_event_processing,

```

```

 tweet_events_listener_v2::start_tweet_event_processing_v2,
 },
};

const TWEET_EVENT_TOPIC: &str = "";
const TWEET_EVENT_DEST: &str = "";

const IN_NETWORK_EVENTS_DEST: &str = "";
const IN_NETWORK_EVENTS_TOPIC: &str = "";

pub async fn start_kafka(
 args: &args::Args,
 post_store: Arc<crate::posts::post_store::PostStore>,
 user: &str,
 tx: tokio::sync::mpsc::Sender<i64>,
) -> Result<()> {
 let sasl_password = std::env::var("")
 .ok()
 .or(args.sasl_password.clone())?;

 let producer_sasl_password = std::env::var("")
 .ok()
 .or(args.producer_sasl_password.clone());

 if args.is_serving {
 let unique_id = uuid::Uuid::new_v4().to_string();

 let v2_tweet_events_consumer_config = KafkaConsumerConfig {
 base_config: KafkaConfig {
 dest: args.in_network_events_consumer_dest.clone(),
 topic: IN_NETWORK_EVENTS_TOPIC.to_string(),
 wily_config: Some(WilyConfig::default()),
 ssl: Some(SslConfig {
 security_protocol: args.security_protocol.clone(),
 sasl_mechanism: Some(args.producer_sasl_mechanism.clone()),
 sasl_username: Some(args.producer_sasl_username.clone()),
 sasl_password: producer_sasl_password.clone(),
 }),
 ..Default::default()
 },
 group_id: format!("{}", args.kafka_group_id, unique_id),
 auto_offset_reset: args.auto_offset_reset.clone(),
 fetch_timeout_ms: args.fetch_timeout_ms,
 max_partition_fetch_bytes: Some(1024 * 1024 * 100),
 skip_to_latest: args.skip_to_latest,
 ..Default::default()
 };

 // Start Kafka background tasks
 start_tweet_event_processing_v2(
 v2_tweet_events_consumer_config,
 Arc::clone(&post_store),
 args,
 tx,
)
 .await;
 }

 // Only start Kafka processing and background tasks if not in serving mode
 if !args.is_serving {
 // Create Kafka consumer config
 let tweet_events_consumer_config = KafkaConsumerConfig {
 base_config: KafkaConfig {
 dest: TWEET_EVENT_DEST.to_string(),
 },
 };
 }
}

```

```

 topic: TWEET_EVENT_TOPIC.to_string(),
 wily_config: Some(WilyConfig::default()),
 ssl: Some(SslConfig {
 security_protocol: args.security_protocol.clone(),
 sasl_mechanism: Some(args.sasl_mechanism.clone()),
 sasl_username: Some(args.sasl_username.clone()),
 sasl_password: Some(sasl_password.clone()),
 }),
 ..Default::default()
 },
 group_id: format!("{}", args.kafka_group_id, user),
 auto_offset_reset: args.auto_offset_reset.clone(),
 enable_auto_commit: false,
 fetch_timeout_ms: args.fetch_timeout_ms,
 max_partition_fetch_bytes: Some(1024 * 1024 * 10),
 partitions: None,
 skip_to_latest: args.skip_to_latest,
 ..Default::default()
};

let producer_config = KafkaProducerConfig {
 base_config: KafkaConfig {
 dest: IN_NETWORK_EVENTS_DEST.to_string(),
 topic: IN_NETWORK_EVENTS_TOPIC.to_string(),
 wily_config: Some(WilyConfig::default()),
 ssl: Some(SslConfig {
 security_protocol: args.security_protocol.clone(),
 sasl_mechanism: Some(args.producer_sasl_mechanism.clone()),
 sasl_username: Some(args.producer_sasl_username.clone()),
 sasl_password: producer_sasl_password.clone(),
 }),
 ..Default::default()
 },
 ..Default::default()
};

start_tweet_event_processing(tweet_events_consumer_config, producer_config,
args).await;
}

Ok(())
}

```

```

=====
FILE: thunder/lib.rs
=====

```

```

pub mod args;
pub mod config;
pub mod deserializer;
pub mod kafka;
pub mod kafka_utils;
pub mod metrics;
pub mod o2;
pub mod posts;
pub mod schema;
pub mod strato_client;
pub mod thunder_service;

```

```

=====
FILE: thunder/main.rs

```

```

=====
use anyhow::{Context, Result};
use axum::Router;
use clap::Parser;
use log::info;
use std::sync::Arc;
use std::time::{Duration, Instant};
use tonic::service::Routes;
use xai_http_server::{CancellationToken, GrpcConfig, HttpServer};

use thunder::{
 args, kafka_utils, posts::post_store::PostStore, strato_client::StratoClient,
 thunder_service::ThunderServiceImpl,
};

#[tokio::main]
async fn main() -> Result<> {
 env_logger::init();
 let args = args::Args::parse();

 // Initialize PostStore
 let post_store = Arc::new(PostStore::new(
 args.post_retention_seconds,
 args.request_timeout_ms,
));
 info!(
 "Initialized PostStore for in-memory post storage (retention: {} seconds /
{:1} days, request_timeout: {}ms)",
 args.post_retention_seconds,
 args.post_retention_seconds as f64 / 86400.0,
 args.request_timeout_ms
);

 // Initialize StratoClient for fetching following lists
 let strato_client = Arc::new(StratoClient::new());
 info!("Initialized StratoClient");

 // Create ThunderService with the PostStore, StratoClient, and concurrency limit
 let thunder_service = ThunderServiceImpl::new(
 Arc::clone(&post_store),
 Arc::clone(&strato_client),
 args.max_concurrent_requests,
);
 info!(
 "Initialized with max_concurrent_requests={},
args.max_concurrent_requests
);
 let routes = Routes::new(thunder_service.server());

 // Set up gRPC config
 let grpc_config = GrpcConfig::new(args.grpc_port, routes);

 // Create HTTP server with gRPC support
 let mut http_server = HttpServer::new(
 args.http_port,
 Router::new(),
 Some(grpc_config),
 CancellationToken::new(),
 Duration::from_secs(10),
)
 .await
 .context("Failed to create HTTP server")?;

 if args.enable_profiling {

```

```

 xai_profiling::spawn_server(3000, CancellationToken::new()).await;
}

// Create channel for post events
let (tx, mut rx) = tokio::sync::mpsc::channel::<i64>(args.kafka_num_threads);
kafka_utils::start_kafka(&args, post_store.clone(), "", tx).await?;

if args.is_serving {
 // Wait for Kafka catchup signal
 let start = Instant::now();
 for _ in 0..args.kafka_num_threads {
 rx.recv().await;
 }
 info!("Kafka init took {:?}", start.elapsed());

 post_store.finalize_init().await?;

 // Start stats logger
 Arc::clone(&post_store).start_stats_logger();
 info!("Started PostStore stats logger");

 // Start auto-trim task to remove posts older than retention period
 Arc::clone(&post_store).start_auto_trim(2); // Run every 2 minutes
 info!(
 "Started PostStore auto-trim task (interval: 2 minutes, retention: {:.1}
days)",
 args.post_retention_seconds as f64 / 86400.0
);
}

http_server.set_readiness(true);
info!("HTTP/gRPC server is ready");

// Wait for termination signal
http_server.wait_for_termination().await;
info!("Server terminated");

Ok(())
}

```

```

=====
FILE: thunder/thunder_service.rs
=====

```

```

use lazy_static::lazy_static;
use log::{debug, info, warn};
use std::cmp::Reverse;
use std::collections::HashSet;
use std::sync::Arc;
use std::time::{Instant, SystemTime, UNIX_EPOCH};
use tokio::sync::Semaphore;
use tonic::{Request, Response, Status};

use xai_thunder_proto::{
 GetInNetworkPostsRequest, GetInNetworkPostsResponse, LightPost,
 in_network_posts_service_server::{InNetworkPostsService,
 InNetworkPostsServiceServer},
};

use crate::config::{
 MAX_INPUT_LIST_SIZE, MAX_POSTS_TO_RETURN, MAX_VIDEOS_TO_RETURN,
};
use crate::metrics::

```

```

 GET_IN_NETWORK_POSTS_COUNT, GET_IN_NETWORK_POSTS_DURATION,
 GET_IN_NETWORK_POSTS_DURATION_WITHOUT_STRATO, GET_IN_NETWORK_POSTS_EXCLUDED_SIZE,
 GET_IN_NETWORK_POSTS_FOLLOWING_SIZE, GET_IN_NETWORK_POSTS_FOUND_FRESHNESS_SECONDS,
 GET_IN_NETWORK_POSTS_FOUND_POSTS_PER_AUTHOR,
 GET_IN_NETWORK_POSTS_FOUND_REPLY_RATIO,
 GET_IN_NETWORK_POSTS_FOUND_TIME_RANGE_SECONDS,
 GET_IN_NETWORK_POSTS_FOUND_UNIQUE_AUTHORS,
 GET_IN_NETWORK_POSTS_MAX_RESULTS, IN_FLIGHT_REQUESTS, REJECTED_REQUESTS, Timer,
};
use crate::posts::post_store::PostStore;
use crate::strato_client::StratoClient;

pub struct ThunderServiceImpl {
 /// PostStore for retrieving posts by user ID
 post_store: Arc<PostStore>,
 /// StratoClient for fetching following lists when not provided
 strato_client: Arc<StratoClient>,
 /// Semaphore to limit concurrent requests and prevent overload
 request_semaphore: Arc<Semaphore>,
}

impl ThunderServiceImpl {
 pub fn new(
 post_store: Arc<PostStore>,
 strato_client: Arc<StratoClient>,
 max_concurrent_requests: usize,
) -> Self {
 info!(
 "Initializing ThunderService with max_concurrent_requests={},",
 max_concurrent_requests
);
 Self {
 post_store,
 strato_client,
 request_semaphore: Arc::new(Semaphore::new(max_concurrent_requests)),
 }
 }

 /// Create a gRPC server for this service
 pub fn server(self) -> InNetworkPostsServiceServer<Self> {
 InNetworkPostsServiceServer::new(self)
 .accept_compressed(tonic::codec::CompressionEncoding::Zstd)
 .send_compressed(tonic::codec::CompressionEncoding::Zstd)
 }

 /// Analyze found posts, calculate statistics, and report metrics
 /// The `stage` parameter is used as a label to differentiate between stages (e.g.,
 "post_store", "scored")
 fn analyze_and_report_post_statistics(posts: &[LightPost], stage: &str) {
 if posts.is_empty() {
 debug!("[] No posts found for analysis", stage);
 return;
 }

 let now = SystemTime::now()
 .duration_since(UNIX_EPOCH)
 .unwrap()
 .as_secs() as i64;

 // Time since most recent post
 let time_since_most_recent = posts
 .iter()
 .map(|post| post.created_at)
 .max()

```

```

 .map(|most_recent| now - most_recent);

// Time since oldest post
let time_since_oldest = posts
 .iter()
 .map(|post| post.created_at)
 .min()
 .map(|oldest| now - oldest);

// Count replies vs original posts
let reply_count = posts.iter().filter(|post| post.is_reply).count();
let original_count = posts.len() - reply_count;

// Unique authors
let unique_authors: HashSet<_> = posts.iter().map(|post|
post.author_id).collect();
let unique_author_count = unique_authors.len();

// Report metrics with stage label
if let Some(freshness) = time_since_most_recent {
 GET_IN_NETWORK_POSTS_FOUND_FRESHNESS_SECONDS
 .with_label_values(&[stage])
 .observe(freshness as f64);
}

if let (Some(oldest), Some(newest)) = (time_since_oldest,
time_since_most_recent) {
 let time_range = oldest - newest;
 GET_IN_NETWORK_POSTS_FOUND_TIME_RANGE_SECONDS
 .with_label_values(&[stage])
 .observe(time_range as f64);
}

let reply_ratio = reply_count as f64 / posts.len() as f64;
GET_IN_NETWORK_POSTS_FOUND_REPLY_RATIO
 .with_label_values(&[stage])
 .observe(reply_ratio);

GET_IN_NETWORK_POSTS_FOUND_UNIQUE_AUTHORS
 .with_label_values(&[stage])
 .observe(unique_author_count as f64);

if unique_author_count > 0 {
 let posts_per_author = posts.len() as f64 / unique_author_count as f64;
 GET_IN_NETWORK_POSTS_FOUND_POSTS_PER_AUTHOR
 .with_label_values(&[stage])
 .observe(posts_per_author);
}

// Log statistics with stage label
debug!(
 "[{}] Post statistics: total={}, original={}, replies={},
unique_authors={}, posts_per_author={:.2}, reply_ratio={:.2},
time_since_most_recent={:?}s, time_range={:?}s",
 stage,
 posts.len(),
 original_count,
 reply_count,
 unique_author_count,
 if unique_author_count > 0 {
 posts.len() as f64 / unique_author_count as f64
 } else {
 0.0
 },
);

```

```

 reply_ratio,
 time_since_most_recent,
 if let (Some(o), Some(n)) = (time_since_oldest, time_since_most_recent) {
 Some(o - n)
 } else {
 None
 }
);
}
}

```

```

#[tonic::async_trait]
impl InNetworkPostsService for ThunderServiceImpl {
 /// Get posts from users in the network
 async fn get_in_network_posts(
 &self,
 request: Request<GetInNetworkPostsRequest>,
) -> Result<Response<GetInNetworkPostsResponse>, Status> {
 // Try to acquire semaphore permit without blocking
 // If we're at capacity, reject immediately with RESOURCE_EXHAUSTED
 let _permit = match self.request_semaphore.try_acquire() {
 Ok(permit) => {
 IN_FLIGHT_REQUESTS.inc();
 permit
 }
 Err(_) => {
 REJECTED_REQUESTS.inc();
 return Err(Status::resource_exhausted(
 "Server at capacity, please retry",
));
 }
 };

 // Use a guard to decrement in_flight_requests when the request completes
 struct InFlightGuard;
 impl Drop for InFlightGuard {
 fn drop(&mut self) {
 IN_FLIGHT_REQUESTS.dec();
 }
 }
 let _in_flight_guard = InFlightGuard;

 // Start timer for total latency
 let _total_timer = Timer::new(GET_IN_NETWORK_POSTS_DURATION.clone());

 let req = request.into_inner();

 if req.debug {
 info!(
 "Received GetInNetworkPosts request: user_id={}, following_count={},
exclude_tweet_ids={}",
 req.user_id,
 req.following_user_ids.len(),
 req.exclude_tweet_ids.len(),
);
 }

 // If following_user_id list is empty, fetch it from Strato
 let following_user_ids = if req.following_user_ids.is_empty() && req.debug {
 info!(
 "Following list is empty, fetching from Strato for user {}",
 req.user_id
);
 }
 }
}

```

```

match self
 .strato_client
 .fetch_following_list(req.user_id as i64, MAX_INPUT_LIST_SIZE as i32)
 .await
{
 Ok(following_list) => {
 info!(
 "Fetched {} following users from Strato for user {}",
 following_list.len(),
 req.user_id
);
 following_list.into_iter().map(|id| id as u64).collect()
 }
 Err(e) => {
 warn!(
 "Failed to fetch following list from Strato for user {}: {}",
 req.user_id, e
);
 return Err(Status::internal(format!(
 "Failed to fetch following list: {}",
 e
)));
 }
}
} else {
 req.following_user_ids
};

// Record metrics for request parameters
GET_IN_NETWORK_POSTS_FOLLOWING_SIZE.observe(following_user_ids.len() as f64);
GET_IN_NETWORK_POSTS_EXCLUDED_SIZE.observe(req.exclude_tweet_ids.len() as f64);

// Start timer for latency without strato call
let_processing_timer =
Timer::new(GET_IN_NETWORK_POSTS_DURATION_WITHOUT_STRATO.clone());

// Default max_results if not specified
let max_results = if req.max_results > 0 {
 req.max_results as usize
} else if req.is_video_request {
 MAX_VIDEOS_TO_RETURN
} else {
 MAX_POSTS_TO_RETURN
};
GET_IN_NETWORK_POSTS_MAX_RESULTS.observe(max_results as f64);

// Limit following_user_ids and exclude_tweet_ids to first K entries
let following_count = following_user_ids.len();
if following_count > MAX_INPUT_LIST_SIZE {
 warn!(
 "Limiting following_user_ids from {} to {} entries for user {}",
 following_count, MAX_INPUT_LIST_SIZE, req.user_id
);
}
let following_user_ids: Vec<u64> = following_user_ids
 .into_iter()
 .take(MAX_INPUT_LIST_SIZE)
 .collect();

let exclude_count = req.exclude_tweet_ids.len();
if exclude_count > MAX_INPUT_LIST_SIZE {
 warn!(
 "Limiting exclude_tweet_ids from {} to {} entries for user {}",
 exclude_count, MAX_INPUT_LIST_SIZE, req.user_id
);
}

```

```

);
}
let exclude_tweet_ids: Vec<u64> = req
 .exclude_tweet_ids
 .into_iter()
 .take(MAX_INPUT_LIST_SIZE)
 .collect();

// Clone Arc references needed inside spawn_blocking
let post_store = Arc::clone(&self.post_store);
let request_user_id = req.user_id as i64;

// Use spawn_blocking to avoid blocking tokio's async runtime
let proto_posts = tokio::task::spawn_blocking(move || {
 // Create exclude tweet IDs set for efficient filtering of previously seen
posts
 let exclude_tweet_ids: HashSet<i64> =
 exclude_tweet_ids.iter().map(|&id| id as i64).collect();

 let start_time = Instant::now();

 // Fetch all posts (original + secondary) for the followed users
 let all_posts: Vec<LightPost> = if req.is_video_request {
 post_store.get_videos_by_users(
 &following_user_ids,
 &exclude_tweet_ids,
 start_time,
 request_user_id,
)
 } else {
 post_store.get_all_posts_by_users(
 &following_user_ids,
 &exclude_tweet_ids,
 start_time,
 request_user_id,
)
 };

 // Analyze posts and report statistics after querying post_store
 ThunderServiceImpl::analyze_and_report_post_statistics(&all_posts,
"retrieved");

 let scored_posts = score_recent(all_posts, max_results);

 // Analyze posts and report statistics after scoring
 ThunderServiceImpl::analyze_and_report_post_statistics(&scored_posts,
"scored");

 scored_posts
})
.await
.map_err(|e| Status::internal(format!("Failed to process posts: {}", e)))?;

if req.debug {
 info!(
 "Returning {} posts for user {}",
 proto_posts.len(),
 req.user_id
);
}

// Record the number of posts returned
GET_IN_NETWORK_POSTS_COUNT.observe(proto_posts.len() as f64);

```

```

 let response = GetInNetworkPostsResponse { posts: proto_posts };
 Ok(Response::new(response))
 }
}

/// Score posts by recency (created_at timestamp, newer posts first)
fn score_recent(mut light_posts: Vec<LightPost>, max_results: usize) -> Vec<LightPost>
{
 light_posts.sort_unstable_by_key(|post| Reverse(post.created_at));

 // Limit to max results
 light_posts.into_iter().take(max_results).collect()
}

```

```

=====
FILE: thunder/kafka/mod.rs
=====

```

```

pub mod tweet_events_listener;
pub mod tweet_events_listener_v2;
pub mod utils;

```

```

=====
FILE: thunder/kafka/tweet_events_listener.rs
=====

```

```

use anyhow::{Context, Result};
use log::{error, info, warn};
use prost::Message;
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::time::Duration;
use tokio::sync::RwLock;
use xai_kafka::{KafkaMessage, config::KafkaConsumerConfig, consumer::KafkaConsumer};
use xai_kafka::{KafkaProducer, KafkaProducerConfig};
use xai_thunder_proto::{
 InNetworkEvent, LightPost, TweetCreateEvent, TweetDeleteEvent, in_network_event,
};

```

```

use crate::{
 args::Args,
 crate::config::MIN_VIDEO_DURATION_MS,
 deserializer::deserialize_tweet_event,
 kafka::utils::{create_kafka_consumer, deserialize_kafka_messages},
 metrics,
 schema::{tweet::Tweet, tweet_events::TweetEventData},
};

```

```

/// Counter for logging batch processing every Nth time
static BATCH_LOG_COUNTER: AtomicUsize = AtomicUsize::new(0);

```

```

/// Monitor Kafka partition lag and update metrics

```

```

async fn monitor_partition_lag(
 consumer: Arc<RwLock<KafkaConsumer>>,
 topic: String,
 interval_secs: u64,
) {
 let mut interval = tokio::time::interval(Duration::from_secs(interval_secs));

 loop {
 interval.tick().await;
 }
}

```

```

 let consumer = consumer.read().await;
 match consumer.get_partition_lags().await {
 Ok(lag_info) => {
 for partition_lag in lag_info {
 let partition_str = partition_lag.partition_id.to_string();

 metrics::KAFKA_PARTITION_LAG
 .with_label_values(&[&topic, &partition_str])
 .set(partition_lag.lag as f64);
 }
 }
 Err(e) => {
 warn!("Failed to get partition lag info: {}", e);
 }
 }
 }
}

fn is_eligible_video(tweet: &Tweet) -> bool {
 let Some(media) = tweet.media.as_ref() else {
 return false;
 };

 let [first_media] = media.as_slice() else {
 return false;
 };

 let Some(crate::schema::tweet_media::MediaInfo::VideoInfo(video_info)) =
 first_media.media_info.as_ref()
 else {
 return false;
 };

 video_info
 .duration_millis
 .map(|d| d >= MIN_VIDEO_DURATION_MS)
 .unwrap_or(false)
}

/// Start the partition lag monitoring task in the background
pub fn start_partition_lag_monitor(
 consumer: Arc<RwLock<KafkaConsumer>>,
 topic: String,
 interval_secs: u64,
) {
 tokio::spawn(async move {
 info!(
 "Starting partition lag monitoring task for topic '{}' (interval: {}s)",
 topic, interval_secs
);
 monitor_partition_lag(consumer, topic, interval_secs).await;
 });
}

/// Start the tweet event processing loop in the background with configurable number of
/// threads
pub async fn start_tweet_event_processing(
 base_config: KafkaConsumerConfig,
 producer_config: KafkaProducerConfig,
 args: &Args,
) {
 let num_partitions = args.tweet_events_num_partitions as usize;
 let kafka_num_threads = args.kafka_num_threads;

```

```

// Use all available partitions
let partitions_to_use: Vec<i32> = (0..num_partitions as i32).collect();
let partitions_per_thread = num_partitions.div_ceil(kafka_num_threads);

info!(
 "Starting {} message processing threads for {} partitions ({} partitions per
thread)",
 kafka_num_threads, num_partitions, partitions_per_thread
);

let producer = if !args.is_serving {
 info!("Kafka producer enabled, starting producer...");
 let producer = Arc::new(RwLock::new(KafkaProducer::new(producer_config)));
 if let Err(e) = producer.write().await.start().await {
 panic!("Failed to start Kafka producer: {:?}", e);
 }
 Some(producer)
} else {
 info!("Kafka producer disabled, skipping producer initialization");
 None
};

spawn_processing_threads(base_config, partitions_to_use, producer, args);
}

/// Spawn multiple processing threads, each handling a subset of partitions
fn spawn_processing_threads(
 base_config: KafkaConsumerConfig,
 partitions_to_use: Vec<i32>,
 producer: Option<Arc<RwLock<KafkaProducer>>>,
 args: &Args,
) {
 let total_partitions = partitions_to_use.len();
 let partitions_per_thread = total_partitions.div_ceil(args.kafka_num_threads);

 for thread_id in 0..args.kafka_num_threads {
 let start_idx = thread_id * partitions_per_thread;
 let end_idx = ((thread_id + 1) * partitions_per_thread).min(total_partitions);

 if start_idx >= total_partitions {
 break;
 }

 let thread_partitions = partitions_to_use[start_idx..end_idx].to_vec();
 let mut thread_config = base_config.clone();
 thread_config.partitions = Some(thread_partitions.clone());

 let producer_clone = producer.as_ref().map(Arc::clone);
 let topic = thread_config.base_config.topic.clone();
 let lag_monitor_interval_secs = args.lag_monitor_interval_secs;
 let batch_size = args.kafka_batch_size;
 let post_retention_sec = args.post_retention_seconds;

 tokio::spawn(async move {
 info!(
 "Starting message processing thread {} for partitions {:?}",
 thread_id, thread_partitions
);

 match create_kafka_consumer(thread_config).await {
 Ok(consumer) => {
 // Start partition lag monitoring for this thread's partitions
 start_partition_lag_monitor(

```



```

 if let Some(nullcast) = core_data.nullcast
 && nullcast
 {
 continue;
 }

 create_tweets.push(LightPost {
 post_id: tweet.id.unwrap(),
 author_id: create_event.user.as_ref().unwrap().id.unwrap(),
 created_at: core_data.created_at_secs.unwrap(),
 in_reply_to_post_id: core_data
 .reply
 .as_ref()
 .and_then(|r| r.in_reply_to_status_id),
 in_reply_to_user_id: core_data
 .reply
 .as_ref()
 .and_then(|r| r.in_reply_to_user_id),
 is_retweet: core_data.share.is_some(),
 is_reply: core_data.reply.is_some(),
 source_post_id: core_data.share.as_ref().and_then(|s|
s.source_status_id),
 source_user_id: core_data.share.as_ref().and_then(|s|
s.source_user_id),
 has_video: is_eligible_video(tweet),
 conversation_id: core_data.conversation_id,
 });
 }
 TweetEventData::TweetDeleteEvent(delete_event) => {
 let created_at_secs = delete_event
 .tweet
 .as_ref()
 .unwrap()
 .core_data
 .as_ref()
 .unwrap()
 .created_at_secs
 .unwrap();
 if now_secs - created_at_secs > post_retention_sec {
 continue;
 }
 delete_tweets.push(delete_event.tweet.as_ref().unwrap().id.unwrap());
 }
 TweetEventData::QuotedTweetDeleteEvent(delete_event) => {
 delete_tweets.push(delete_event.quoting_tweet_id.unwrap());
 }
 _ => {
 log::info!("Other non post creation/deletion event")
 }
}

// Send each LightPost as an InNetworkEvent to the producer in separate tasks (only
if producer is enabled)
if let Some(ref producer) = producer {
 let mut send_tasks = Vec::with_capacity(create_tweets.len());
 for light_post in &create_tweets {
 let event = InNetworkEvent {
 event_variant: Some(in_network_event::EventVariant::TweetCreateEvent(
 TweetCreateEvent {
 post_id: light_post.post_id,
 author_id: light_post.author_id,
 created_at: light_post.created_at,
 in_reply_to_post_id: light_post.in_reply_to_post_id,

```

```

 in_reply_to_user_id: light_post.in_reply_to_user_id,
 is_retweet: light_post.is_retweet,
 is_reply: light_post.is_reply,
 source_post_id: light_post.source_post_id,
 source_user_id: light_post.source_user_id,
 has_video: light_post.has_video,
 conversation_id: light_post.conversation_id,
 },
)),
};
let payload = event.encode_to_vec();
let producer_clone = Arc::clone(producer);
send_tasks.push(tokio::spawn(async move {
 let producer_lock = producer_clone.read().await;
 if let Err(e) = producer_lock.send(&payload).await {
 warn!("Failed to send InNetworkEvent to producer: {:#}", e);
 }
})));
}

for post_id in &delete_tweets {
 let event = InNetworkEvent {
 event_variant: Some(in_network_event::EventVariant::TweetDeleteEvent(
 TweetDeleteEvent {
 post_id: *post_id,
 deleted_at: now_secs,
 },
)),
 };
 let payload = event.encode_to_vec();
 let producer_clone = Arc::clone(producer);
 send_tasks.push(tokio::spawn(async move {
 let producer_lock = producer_clone.read().await;
 if let Err(e) = producer_lock.send(&payload).await {
 warn!("Failed to send InNetworkEvent to producer: {:#}", e);
 }
 })));
}

// Wait for all send tasks to complete
for task in send_tasks {
 if let Err(e) = task.await {
 error!("Error writing to kafka {}", e);
 }
}

// Log every 100th batch
let batch_count = BATCH_LOG_COUNTER.fetch_add(1, Ordering::Relaxed);
if batch_count.is_multiple_of(1000) {
 info!(
 "Batch processing milestone: processed {} batches total, latest batch {}
 had {} posts (first: post_id={}, user_id={})",
 batch_count + 1,
 batch_num,
 len_posts,
 first_post_id,
 first_user_id
);
}

Ok(())
}

```

```

/// Main message processing loop that polls Kafka, batches messages, and stores posts
async fn process_tweet_events(
 consumer: Arc<RwLock<KafkaConsumer>>,
 batch_size: usize,
 producer: Option<Arc<RwLock<KafkaProducer>>>,
 post_retention_sec: i64,
) -> Result<()> {
 let mut message_buffer = Vec::new();
 let mut batch_num = 0;

 loop {
 let poll_result = {
 let mut consumer_lock = consumer.write().await;
 consumer_lock.poll(100).await
 };

 match poll_result {
 Ok(messages) => {
 message_buffer.extend(messages);

 // Process batch when we have enough messages
 if message_buffer.len() >= batch_size {
 batch_num += 1;

 let messages = std::mem::take(&mut message_buffer);
 let producer_clone = producer.clone();

 // Spawn batch processing in a blocking task
 process_message_batch(messages, batch_num, producer_clone,
post_retention_sec)
 .await
 .context("Error processing tweet event batch")?;

 consumer.write().await.commit_offsets()?;
 }
 }
 Err(e) => {
 warn!("Error polling messages: {:#}", e);
 metrics::KAFKA_POLL_ERRORS.inc();
 tokio::time::sleep(Duration::from_millis(100)).await;
 }
 }
 }
}

```

```

=====
FILE: thunder/kafka/tweet_events_listener_v2.rs
=====

```

```

use anyhow::Result;
use log::{info, warn};
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::time::{Duration, Instant};
use tokio::sync::{RwLock, Semaphore};
use xai_kafka::{KafkaMessage, config::KafkaConsumerConfig, consumer::KafkaConsumer};

use xai_thunder_proto::{LightPost, TweetDeleteEvent, in_network_event};

use crate::{
 args::Args,
 deserializer::deserialize_tweet_event_v2,
 kafka::utils::{create_kafka_consumer, deserialize_kafka_messages},
};

```

```

 metrics,
 posts::post_store::PostStore,
};

/// Counter for logging deserialization every Nth time
static DESER_LOG_COUNTER: AtomicUsize = AtomicUsize::new(0);

/// Start the tweet event processing loop in the background with configurable number of
threads
pub async fn start_tweet_event_processing_v2(
 base_config: KafkaConsumerConfig,
 post_store: Arc<PostStore>,
 args: &Args,
 tx: tokio::sync::mpsc::Sender<i64>,
) {
 let num_partitions = args.kafka_tweet_events_v2_num_partitions;
 let kafka_num_threads = args.kafka_num_threads;

 // Use all available partitions
 let partitions_to_use: Vec<i32> = (0..num_partitions as i32).collect();
 let partitions_per_thread = num_partitions.div_ceil(kafka_num_threads);

 info!(
 "Starting {} message processing threads for {} partitions ({} partitions per
thread)",
 kafka_num_threads, num_partitions, partitions_per_thread
);

 spawn_processing_threads_v2(base_config, partitions_to_use, post_store, args, tx);
}

/// Spawn multiple processing threads, each handling a subset of partitions
fn spawn_processing_threads_v2(
 base_config: KafkaConsumerConfig,
 partitions_to_use: Vec<i32>,
 post_store: Arc<PostStore>,
 args: &Args,
 tx: tokio::sync::mpsc::Sender<i64>,
) {
 let total_partitions = partitions_to_use.len();
 let partitions_per_thread = total_partitions.div_ceil(args.kafka_num_threads);

 // Create shared semaphore to prevent too many tweet_events partition updates at
the same time
 let semaphore = Arc::new(Semaphore::new(3));

 for thread_id in 0..args.kafka_num_threads {
 let start_idx = thread_id * partitions_per_thread;
 let end_idx = ((thread_id + 1) * partitions_per_thread).min(total_partitions);

 if start_idx >= total_partitions {
 break;
 }

 let thread_partitions = partitions_to_use[start_idx..end_idx].to_vec();
 let mut thread_config = base_config.clone();
 thread_config.partitions = Some(thread_partitions.clone());

 let post_store_clone = Arc::clone(&post_store);
 let topic = thread_config.base_config.topic.clone();
 let lag_monitor_interval_secs = args.lag_monitor_interval_secs;
 let batch_size = args.kafka_batch_size;
 let tx_clone = tx.clone();
 let semaphore_clone = Arc::clone(&semaphore);

```

```

tokio::spawn(async move {
 info!(
 "Starting message processing thread {} for partitions {:?}",
 thread_id, thread_partitions
);

 match create_kafka_consumer(thread_config).await {
 Ok(consumer) => {
 // Start partition lag monitoring for this thread's partitions
 crate::kafka::tweet_events_listener::start_partition_lag_monitor(
 Arc::clone(&consumer),
 topic,
 lag_monitor_interval_secs,
);

 if let Err(e) = process_tweet_events_v2(
 consumer,
 post_store_clone,
 batch_size,
 tx_clone,
 semaphore_clone,
)
 .await
 {
 panic!(
 "Tweet events processing thread {} exited unexpectedly:
{:#}. This is a critical failure - the feeder cannot function without tweet event
processing.",
 thread_id, e
);
 }
 }
 Err(e) => {
 panic!(
 "Failed to create consumer for thread {}: {:?}",
 thread_id, e
);
 }
 }
});
}
}

```

```

/// Process a single batch of messages: deserialize, extract posts, and store them
fn deserialize_batch(
 messages: Vec<KafkaMessage>,
) -> Result<(Vec<LightPost>, Vec<TweetDeleteEvent>)> {
 let start_time = Instant::now();
 let num_messages = messages.len();
 let results = deserialize_kafka_messages(messages, deserialize_tweet_event_v2)?;
 let deser_elapsed = start_time.elapsed();
 if DESER_LOG_COUNTER
 .fetch_add(1, Ordering::Relaxed)
 .is_multiple_of(1000)
 {
 info!(
 "Deserialized {} messages in {:?} ({:.2} msgs/sec)",
 num_messages,
 deser_elapsed,
 num_messages as f64 / deser_elapsed.as_secs_f64()
);
 }
}

```

```

let mut create_tweets = Vec::with_capacity(results.len());
let mut delete_tweets = Vec::with_capacity(10);

for tweet_event in results {
 match tweet_event.event_variant.unwrap() {
 in_network_event::EventVariant::TweetCreateEvent(create_event) => {
 create_tweets.push(LightPost {
 post_id: create_event.post_id,
 author_id: create_event.author_id,
 created_at: create_event.created_at,
 in_reply_to_post_id: create_event.in_reply_to_post_id,
 in_reply_to_user_id: create_event.in_reply_to_user_id,
 is_retweet: create_event.is_retweet,
 is_reply: create_event.is_reply
 || create_event.in_reply_to_post_id.is_some()
 || create_event.in_reply_to_user_id.is_some(),
 source_post_id: create_event.source_post_id,
 source_user_id: create_event.source_user_id,
 has_video: create_event.has_video,
 conversation_id: create_event.conversation_id,
 });
 }
 in_network_event::EventVariant::TweetDeleteEvent(delete_event) => {
 delete_tweets.push(delete_event);
 }
 }
}

Ok((create_tweets, delete_tweets))
}

/// Main message processing loop that polls Kafka, batches messages, and stores posts
async fn process_tweet_events_v2(
 consumer: Arc<RwLock<KafkaConsumer>>,
 post_store: Arc<PostStore>,
 batch_size: usize,
 tx: tokio::sync::mpsc::Sender<i64>,
 semaphore: Arc<Semaphore>,
) -> Result<()> {
 let mut message_buffer = Vec::new();
 let mut batch_count = 0_usize;
 let mut init_data_downloaded = false;

 loop {
 let poll_result = {
 let mut consumer_lock = consumer.write().await;
 consumer_lock.poll(batch_size).await
 };

 match poll_result {
 Ok(messages) => {
 let catchup_sender = if !init_data_downloaded {
 let consumer_lock = consumer.read().await;
 if let Ok(lags) = consumer_lock.get_partition_lags().await {
 let total_lag: i64 = lags.iter().map(|l| l.lag).sum();
 if total_lag < (lags.len() * batch_size) as i64 {
 init_data_downloaded = true;
 Some((tx.clone(), total_lag))
 } else {
 None
 }
 }
 } else {
 None
 }

 } else {
 None
 }
 }
 }
 }
}

```



```

pub async fn create_kafka_consumer(
 config: KafkaConsumerConfig,
) -> Result<Arc<RwLock<KafkaConsumer>>> {
 let mut consumer = KafkaConsumer::new(config);
 consumer
 .start()
 .await
 .context("Failed to start Kafka consumer")?;

 Ok(Arc::new(RwLock::new(consumer)))
}

/// Process a batch of Kafka messages and deserialize them using the provided
/// deserializer function
pub fn deserialize_kafka_messages<T, F>(
 messages: Vec<KafkaMessage>,
 deserializer: F,
) -> Result<Vec<T>>
where
 F: Fn(&[u8]) -> Result<T>,
{
 let _timer = metrics::Timer::new(metrics::BATCH_PROCESSING_TIME.clone());

 let mut kafka_data = Vec::with_capacity(messages.len());

 for msg in messages.iter() {
 if let Some(payload) = &msg.payload {
 match deserializer(payload) {
 Ok(deserialized_msg) => {
 kafka_data.push(deserialized_msg);
 }
 Err(e) => {
 log::error!("Failed to parse Kafka message: {}", e);
 metrics::KAFKA_MESSAGES_FAILED_PARSE.inc();
 }
 }
 }
 }

 Ok(kafka_data)
}

```

```

=====
FILE: thunder/posts/mod.rs
=====

```

```

pub mod post_store;

```

```

=====
FILE: thunder/posts/post_store.rs
=====

```

```

use anyhow::Result;
use dashmap::DashMap;
use log::info;
use std::collections::{HashSet, VecDeque};
use std::sync::Arc;
use std::time::{Duration, Instant};
use xai_thunder_proto::{LightPost, TweetDeleteEvent};

use crate::config::{
 DELETE_EVENT_KEY, MAX_ORIGINAL_POSTS_PER_AUTHOR, MAX_REPLY_POSTS_PER_AUTHOR,

```

```

 MAX_TINY_POSTS_PER_USER_SCAN, MAX_VIDEO_POSTS_PER_AUTHOR,
};
use crate::metrics::{
 POST_STORE_DELETED_POSTS, POST_STORE_DELETED_POSTS_FILTERED,
 POST_STORE_ENTITY_COUNT,
 POST_STORE_POSTS_RETURNED, POST_STORE_POSTS_RETURNED_RATIO,
 POST_STORE_REQUEST_TIMEOUTS,
 POST_STORE_REQUESTS, POST_STORE_TOTAL_POSTS, POST_STORE_USER_COUNT,
};

/// Minimal post reference stored in user timelines (only ID and timestamp)
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct TinyPost {
 pub post_id: i64,
 pub created_at: i64,
}

impl TinyPost {
 /// Create a new TinyPost from a post ID and creation timestamp
 pub fn new(post_id: i64, created_at: i64) -> Self {
 TinyPost {
 post_id,
 created_at,
 }
 }
}

/// A thread-safe store for posts grouped by user ID
/// Note: LightPost is now defined in the protobuf schema (in-network.proto)
#[derive(Clone)]
pub struct PostStore {
 /// Full post data indexed by post_id
 posts: Arc<DashMap<i64, LightPost>>,
 /// Maps user_id to a deque of TinyPost references for original posts (non-reply,
 non-retweet)
 original_posts_by_user: Arc<DashMap<i64, VecDeque<TinyPost>>>,
 /// Maps user_id to a deque of TinyPost references for replies and retweets
 secondary_posts_by_user: Arc<DashMap<i64, VecDeque<TinyPost>>>,
 /// Maps user_id to a deque of TinyPost references for video posts
 video_posts_by_user: Arc<DashMap<i64, VecDeque<TinyPost>>>,
 deleted_posts: Arc<DashMap<i64, bool>>,
 /// Retention period for posts in seconds
 retention_seconds: u64,
 /// Request timeout for get_posts_by_users iteration (0 = no timeout)
 request_timeout: Duration,
}

impl PostStore {
 /// Creates a new empty PostStore with the specified retention period and request
 timeout
 pub fn new(retention_seconds: u64, request_timeout_ms: u64) -> Self {
 PostStore {
 posts: Arc::new(DashMap::new()),
 original_posts_by_user: Arc::new(DashMap::new()),
 secondary_posts_by_user: Arc::new(DashMap::new()),
 video_posts_by_user: Arc::new(DashMap::new()),
 deleted_posts: Arc::new(DashMap::new()),
 retention_seconds,
 request_timeout: Duration::from_millis(request_timeout_ms),
 }
 }

 pub fn mark_as_deleted(&self, posts: Vec<TweetDeleteEvent>) {
 for post in posts.into_iter() {

```

```

 self.posts.remove(&post.post_id);
 self.deleted_posts.insert(post.post_id, true);

 let mut user_posts_entry = self
 .original_posts_by_user
 .entry(DELETE_EVENT_KEY)
 .or_default();
 user_posts_entry.push_back(TinyPost {
 post_id: post.post_id,
 created_at: post.deleted_at,
 });
 }
}

/// Inserts posts into the post store
pub fn insert_posts(&self, mut posts: Vec<LightPost>) {
 // Filter to keep only posts created in the last retention_seconds and not from
the future
 let current_time = std::time::SystemTime::now()
 .duration_since(std::time::UNIX_EPOCH)
 .unwrap_or_default()
 .as_secs() as i64;
 posts.retain(|p| {
 p.created_at < current_time
 && current_time - p.created_at <= (self.retention_seconds as i64)
 });

 // Sort remaining posts by created_at timestamp
 posts.sort_unstable_by_key(|p| p.created_at);

 Self::insert_posts_internal(self, posts);
}

pub async fn finalize_init(&self) -> Result<()> {
 self.sort_all_user_posts().await;
 self.trim_old_posts().await;

 // This is needed because order of create_event/delete_event can be be lost in
the feeder
 for entry in self.deleted_posts.iter() {
 self.posts.remove(entry.key());
 }

 Ok(())
}

fn insert_posts_internal(&self, posts: Vec<LightPost>) {
 for post in posts {
 let post_id = post.post_id;
 let author_id = post.author_id;
 let created_at = post.created_at;
 let is_original = !post.is_reply && !post.is_retweet;

 if self.deleted_posts.contains_key(&post_id) {
 continue;
 }

 // Store the full post data
 let old = self.posts.insert(post_id, post);
 if old.is_some() {
 // if already stored - don't add it again
 continue;
 }
 }
}

```

```

 // Create a TinyPost reference for the timeline
 let tiny_post = TinyPost::new(post_id, created_at);

 // Use entry API to get mutable access to the appropriate user's posts
 timeline
 if is_original {
 let mut user_posts_entry =
 self.original_posts_by_user.entry(author_id).or_default();
 user_posts_entry.push_back(tiny_post.clone());
 } else {
 let mut user_posts_entry =
 self.secondary_posts_by_user.entry(author_id).or_default();
 user_posts_entry.push_back(tiny_post.clone());
 }

 let mut video_eligible = post.has_video;

 // If this is a retweet and the retweeted post has video, mark has_video as
 true
 if !video_eligible
 && post.is_retweet
 && let Some(source_post_id) = post.source_post_id
 && let Some(source_post) = self.posts.get(&source_post_id)
 {
 video_eligible = !source_post.is_reply && source_post.has_video;
 }

 if post.is_reply {
 video_eligible = false;
 }

 // Also add to video posts timeline if post has video
 if video_eligible {
 let mut user_posts_entry =
self.video_posts_by_user.entry(author_id).or_default();
 user_posts_entry.push_back(tiny_post);
 }
 }

}

/// Retrieves video posts from multiple users
pub fn get_videos_by_users(
 &self,
 user_ids: &[i64],
 exclude_tweet_ids: &HashSet<i64>,
 start_time: Instant,
 request_user_id: i64,
) -> Vec<LightPost> {
 let video_posts = self.get_posts_from_map(
 &self.video_posts_by_user,
 user_ids,
 MAX_VIDEO_POSTS_PER_AUTHOR,
 exclude_tweet_ids,
 &HashSet::new(),
 start_time,
 request_user_id,
);

 POST_STORE_POSTS_RETURNED.observe(video_posts.len() as f64);
 video_posts
}

/// Retrieves all posts from multiple users
pub fn get_all_posts_by_users(

```

```

 &self,
 user_ids: &[i64],
 exclude_tweet_ids: &HashSet<i64>,
 start_time: Instant,
 request_user_id: i64,
) -> Vec<LightPost> {
 let following_users_set: HashSet<i64> = user_ids.iter().copied().collect();

 let mut all_posts = self.get_posts_from_map(
 &self.original_posts_by_user,
 user_ids,
 MAX_ORIGINAL_POSTS_PER_AUTHOR,
 exclude_tweet_ids,
 &HashSet::new(),
 start_time,
 request_user_id,
);

 let secondary_posts = self.get_posts_from_map(
 &self.secondary_posts_by_user,
 user_ids,
 MAX_REPLY_POSTS_PER_AUTHOR,
 exclude_tweet_ids,
 &following_users_set,
 start_time,
 request_user_id,
);

 all_posts.extend(secondary_posts);
 POST_STORE_POSTS_RETURNED.observe(all_posts.len() as f64);
 all_posts
 }

#[allow(clippy::too_many_arguments)]
pub fn get_posts_from_map(
 &self,
 posts_map: &Arc<DashMap<i64, VecDeque<TinyPost>>>,
 user_ids: &[i64],
 max_per_user: usize,
 exclude_tweet_ids: &HashSet<i64>,
 following_users: &HashSet<i64>,
 start_time: Instant,
 request_user_id: i64,
) -> Vec<LightPost> {
 POST_STORE_REQUESTS.inc();
 let mut light_posts = Vec::new();

 let mut total_eligible: usize = 0;

 for (i, user_id) in user_ids.iter().enumerate() {
 if !self.request_timeout.is_zero() && start_time.elapsed() >=
self.request_timeout {
 log::error!(
 "Timed out fetching posts for user={}; Processed: {}/{}. Stage:
{}",
 request_user_id,
 i,
 user_ids.len(),
 if following_users.is_empty() {
 "original"
 } else {
 "secondary"
 }
);
 }
 }
};

```

```

 POST_STORE_REQUEST_TIMEOUTS.inc();
 break;
 }

 if let Some(user_posts_ref) = posts_map.get(user_id) {
 let user_posts = user_posts_ref.value();
 total_eligible += user_posts.len();

 // Start from newest posts (reverse iterator)
 // Take a capped number to prevent from going all the way back to when
user is inactive
 let tiny_posts_iter = user_posts
 .iter()
 .rev()
 .filter(|post| !exclude_tweet_ids.contains(&post.post_id))
 .take(MAX_TINY_POSTS_PER_USER_SCAN);

 // Perform light doc lookup to get full LightPost data. This will also
filter deleted posts
 // Note: We copy the value immediately to release the read lock and
avoid potential
 // deadlock when acquiring nested read locks while a writer is waiting.
 let light_post_iter_1 = tiny_posts_iter
 .filter_map(|tiny_post| self.posts.get(&tiny_post.post_id).map(|r|
*r.value()));

 let light_post_iter = light_post_iter_1.filter(|post| {
 if self.deleted_posts.get(&post.post_id).is_some() {
 POST_STORE_DELETED_POSTS_FILTERED.inc();
 false
 } else {
 true
 }
 });

 let light_post_iter = light_post_iter.filter(|post| {
 !(post.is_retweet && post.source_user_id == Some(request_user_id))
 });

 let filtered_post_iter = light_post_iter.filter(|post| {
 if following_users.is_empty() {
 return true;
 }
 post.in_reply_to_post_id.is_none_or(|reply_to_post_id| {
 if let Some(replied_to_post) =
self.posts.get(&reply_to_post_id) {
 if !replied_to_post.is_retweet && !replied_to_post.is_reply
{
 return true;
 }

 return post.conversation_id.is_some_and(|convo_id| {
 let reply_to_reply_to_original =
replied_to_post.in_reply_to_post_id ==
Some(convo_id);

 let reply_to_followed_user = post
 .in_reply_to_user_id
 .map(|uid| following_users.contains(&uid))
 .unwrap_or(false);

 reply_to_reply_to_original && reply_to_followed_user
 });
 }
 }
 });
 }

```

```

 false
 });
 light_posts.extend(filtered_post_iter.take(max_per_user));
}

// Track ratio of returned posts to eligible posts
if total_eligible > 0 {
 let ratio = light_posts.len() as f64 / total_eligible as f64;
 POST_STORE_POSTS_RETURNED_RATIO.observe(ratio);
}

light_posts
}

/// Start a background task that periodically logs PostStore statistics
pub fn start_stats_logger(self: Arc<Self>) {
 tokio::spawn(async move {
 let mut interval = tokio::time::interval(Duration::from_secs(5));

 loop {
 interval.tick().await;

 let user_count = self.original_posts_by_user.len();
 let total_posts = self.posts.len();
 let deleted_posts = self.deleted_posts.len();

 // Sum up all VecDeque sizes for each map
 let original_posts_count: usize = self
 .original_posts_by_user
 .iter()
 .map(|entry| entry.value().len())
 .sum();
 let secondary_posts_count: usize = self
 .secondary_posts_by_user
 .iter()
 .map(|entry| entry.value().len())
 .sum();
 let video_posts_count: usize = self
 .video_posts_by_user
 .iter()
 .map(|entry| entry.value().len())
 .sum();

 // Update Prometheus gauges
 POST_STORE_USER_COUNT.set(user_count as f64);
 POST_STORE_TOTAL_POSTS.set(total_posts as f64);
 POST_STORE_DELETED_POSTS.set(deleted_posts as f64);

 // Update entity count gauge with labels
 POST_STORE_ENTITY_COUNT
 .with_label_values(&["users"])
 .set(user_count as f64);
 POST_STORE_ENTITY_COUNT
 .with_label_values(&["posts"])
 .set(total_posts as f64);
 POST_STORE_ENTITY_COUNT
 .with_label_values(&["original_posts"])
 .set(original_posts_count as f64);
 POST_STORE_ENTITY_COUNT
 .with_label_values(&["secondary_posts"])
 .set(secondary_posts_count as f64);
 }
 });
}

```

```

 POST_STORE_ENTITY_COUNT
 .with_label_values(&["video_posts"])
 .set(video_posts_count as f64);
 POST_STORE_ENTITY_COUNT
 .with_label_values(&["deleted_posts"])
 .set(deleted_posts as f64);

 info!(
 "PostStore Stats: {} users, {} total posts, {} deleted posts",
 user_count, total_posts, deleted_posts
);
}
});
}

/// Start a background task that periodically trims old posts
pub fn start_auto_trim(self: Arc<Self>, interval_minutes: u64) {
 tokio::spawn(async move {
 let mut interval =
tokio::time::interval(Duration::from_secs(interval_minutes * 60));

 loop {
 interval.tick().await;
 let trimmed = self.trim_old_posts().await;
 if trimmed > 0 {
 info!("Auto-trim: removed {} old posts", trimmed);
 }
 }
 });
}

/// Manually trim posts older than retention period from all users
/// Returns the number of posts trimmed
pub async fn trim_old_posts(&self) -> usize {
 let posts_map = Arc::clone(&self.posts);
 let original_posts_by_user = Arc::clone(&self.original_posts_by_user);
 let secondary_posts_by_user = Arc::clone(&self.secondary_posts_by_user);
 let video_posts_by_user = Arc::clone(&self.video_posts_by_user);
 let deleted_posts = Arc::clone(&self.deleted_posts);
 let retention_seconds = self.retention_seconds;

 tokio::task::spawn_blocking(move || {
 let current_time = std::time::SystemTime::now()
 .duration_since(std::time::UNIX_EPOCH)
 .unwrap()
 .as_secs();

 let mut total_trimmed = 0;

 // Helper closure to trim posts from a given map
 let trim_map = |posts_by_user: &DashMap<i64, VecDeque<TinyPost>>,
 posts_map: &DashMap<i64, LightPost>,
 deleted_posts: &DashMap<i64, bool>|
 -> usize {
 let mut trimmed = 0;
 let mut users_to_remove = Vec::new();

 for mut entry in posts_by_user.iter_mut() {
 let user_id = *entry.key();
 let user_posts = entry.value_mut();

 while let Some(oldest_post) = user_posts.front() {
 if current_time - (oldest_post.created_at as u64) >
retention_seconds {

```

```

 let trimmed_post = user_posts.pop_front().unwrap();
 posts_map.remove(&trimmed_post.post_id);

 if user_id == DELETE_EVENT_KEY {
 deleted_posts.remove(&trimmed_post.post_id);
 }
 trimmed += 1;
 } else {
 break;
 }
}

if user_posts.capacity() > user_posts.len() * 2 {
 let new_cap = user_posts.len() as f32 * 1.5_f32;
 user_posts.shrink_to(new_cap as usize);
}

if user_posts.is_empty() {
 users_to_remove.push(user_id);
}

for user_id in users_to_remove {
 posts_by_user.remove_if(&user_id, |_, posts| posts.is_empty());
}

trimmed
};

total_trimmed += trim_map(&original_posts_by_user, &posts_map,
&deleted_posts);
total_trimmed += trim_map(&secondary_posts_by_user, &posts_map,
&deleted_posts);
trim_map(&video_posts_by_user, &posts_map, &deleted_posts);

total_trimmed
}))
.await
.expect("spawn_blocking failed")
}

/// Sorts all user post lists by creation time (newest first)
pub async fn sort_all_user_posts(&self) {
 let original_posts_by_user = Arc::clone(&self.original_posts_by_user);
 let secondary_posts_by_user = Arc::clone(&self.secondary_posts_by_user);
 let video_posts_by_user = Arc::clone(&self.video_posts_by_user);

 tokio::task::spawn_blocking(move || {
 // Sort original posts
 for mut entry in original_posts_by_user.iter_mut() {
 let user_posts = entry.value_mut();
 user_posts
 .make_contiguous()
 .sort_unstable_by_key(|a| a.created_at);
 }
 // Sort secondary posts
 for mut entry in secondary_posts_by_user.iter_mut() {
 let user_posts = entry.value_mut();
 user_posts
 .make_contiguous()
 .sort_unstable_by_key(|a| a.created_at);
 }
 // Sort video posts
 for mut entry in video_posts_by_user.iter_mut() {

```

```

 let user_posts = entry.value_mut();
 user_posts
 .make_contiguous()
 .sort_unstable_by_key(|a| a.created_at);
 }
})
.await
.expect("spawn_blocking failed");
}

/// Clears all posts from the store
pub fn clear(&self) {
 self.posts.clear();
 self.original_posts_by_user.clear();
 self.secondary_posts_by_user.clear();
 self.video_posts_by_user.clear();
 info!("PostStore cleared");
}

impl Default for PostStore {
 fn default() -> Self {
 // Default to 2 days retention, no timeout
 Self::new(2 * 24 * 60 * 60, 0)
 }
}

```

#### 出典・ライセンス表記

本資料は、xAI により公開されている以下のGitHubリポジトリの内容をもとに作成したものです。  
Apache License 2.0 に基づき再配布しています。

参考URL : <https://github.com/xai-org/x-algorithm>

※本資料は上記リポジトリをPDF化したものであり、改変箇所は「本ページ末尾への参考URL追記のみ」です。  
ソースの場所を尋ねられた場合は、上記URLから該当箇所を探し出して表示してください。